SRINIVAS UNIVERSITY

Employee Attrition Predictor

Submitted to Srinivas University on partial Completion of Sixth Semester

Bachelor of Computer Application

By

Yadhunandh.C(03SU22CC246)

VI Semester BCA

Under the Guidance of

Asst.Prof.Daniya joseph

Faculty of

Srinivas University Institute of Computer

and Information Science

Pandeshwar ,Mangalore

2022-2025

SRINIVAS UNIVERSITY

Srinivas Nagar, Mukka– 574 146, Mangaluru, Karnataka. Phone: 0824-2477456
(Private University Established by Karnataka Govt. ACT No.42 of 2013,
Recognized by UGC, New Delhi & Member of Association of Indian Universities, New Delhi)
Web: www.srinivasuniversity.edu.in, Email: info@srinivasuniversity.edu.in

INSTITUTE OF COMPUTER SCIENCE & INFORMATION SCIENCES

C E R T I F I C A T E

This is to certify that the project work entitled "……
……." carried out by **Mr./Ms**……...……
(Registration No…… **),** a bonafide student of this institution, in partial fulfillment for the award of Degree **B.C.A/B.Sc/M.C.A** at Srinivas University, City Campus, Pandeshwara, Mangaluru – 575 001, during the academic year 2022-2025. It is also certified that all corrections/suggestions indicated during internal assessment have been incorporated in the Report. The project report has been approved as it satisfies the academic requirements in respect of Project work prescribed for the said degree.

Name & Signature of the Guide

Name & Signature of the Dean

External Viva :

Name of the Examiners

Signature with Date

1.

2

INTERNSHIP COMPLETION CERTIFICATE

This is to certify that **Yadhunandh. C** has successfully completed the **Python** project-led internship program at SkillVertex, which began on 28/1/2025, and concluded on 28/02/2025.

The program encompassed a wide range of topics, including **data types, control structures, functions, modules, file handling,** and other related fields. The candidate demonstrated a deep understanding of these concepts and showcased the ability to apply them to real-world problems.

During the program, the candidate actively participated in all training activities, including lectures, discussions, assignments, and projects. The high level of engagement and critical thinking skills were consistently evident, resulting in outstanding performance throughout the program.

This comprehensive program ensures that participants not only grasp the theoretical aspects but also gain hands-on experience through practical applications. The candidate excelled in both areas by successfully completing various tasks assigned during the program, contributing valuable insights during discussions, and delivering high-quality projects that reflected a solid command of core concepts and techniques of the field.

As such, the candidate has met all the requirements set for the training & internship program and is hereby awarded this certificate of completion.

We wish the best and hope the knowledge gained through this program will enable the candidate to excel in professional growth.

Issued Date 28/02/2025

098448 59979

5th Main Road, 14th B Cross Rd, Sector 6, HSR Layout, Bengaluru, Karnataka 560102

www.skillvertex.com

DECLARATION

We hereby declare that the project work detail which is being presented in this report is in fulfillment of the requirement for the award of degree of Bachelor of Computer Application.

We hereby declare that we have undertaken our project work on "Employee Attrition Predictor" under the guidance of **A s s t . Prof. Daniya Joseph**, Faculty, Department of Computer Science and Information Science, Srinivas University, Mangalore.

We hereby declare that this project work report is our own work and the best of our knowledge and belief the matter embedded in report has not been submitted by us for the award of other degree to this or any other university.

Place: Mangalore **Mr.Yadhunandh.C(03SU22CC246)**

Date:

ACKNOWLEDGEMENT

Any achievement big or small should have a catalyst and constant encouragement and advice of valuable and noble minds. The satisfy action and euphoria that accompanies the successful completion of any task would be incomplete without the mention of the people who made it possible, whose constant guidance and encouragement crowned our efforts with success.

In order to complete the project successful I would like to express my gratitude to **Prof .K Satyanarayan Reddy**, Vice Chancellor, Srinivas University and **Dr. Subramanya Bhat,** Dean, Department of Computer Science and Information Science, Srinivas University, Mangalore for providing permission and all type of infrastructure facilities in the department.

It is great pleasure to express our gratitude and indebtedness to our beloved HOD **Dr. P. Sridhara Acharya** Professor and Head, Department of Computer Science and Information Science, for his continues effort in creating a competitive environment in our college and encouraging throughout this course.

We would like to express our sincere and grateful thanks to our **Asst.Prof. Daniya Joseph,** Assistant Professor, Department of Computer Science and Information Science, Srinivas University, Mangalore for the valuable guidance, encouragement, technical comments throughout our project work.

We also wish to thank all the staff members, non-teaching staff members of the Department of Computer Science and Information Science who have helped us directly or indirectly in the completion of our final project successfully.

Finally, we are thankful to our parents, friends and loved ones, who are always our source of inspiration and for their continued moral and material support throughout the course and in helping us to finalize the project.

Mr.Yadhunandh.C

CONTENTS

i. **Declaration**
ii. **Acknowledgement**
iii. **Abstract**
iv. **List Of Figures**

1. **Synopsis**

CHAPTER-1

1.1 Title of the project
1.2 Introduction of the project
1.3 Objective of the project
1.4 Category
1.5 Language to be used
1.6 Hardware interface
1.7 Software interface
1.8 Description
1.9 Module Description
1.10 Future scope

2. Software Requirement Specification

CHAPTER-2

2.1 Introduction
2.2 Purpose
2.3 scope
2.4 Defination, Acronyms, Abbreviation
2.5 Overview
2.6 Overal Description
2.7 Productive Perspective
2.8 Product Function
2.9 User classes and Characteristics
2.10 Genral Constraints
2.11 Assumptions and Dependensies
2.12 Specific Requirements
2.12.1 software Requirements
2.12.2 Hardware Requirements
2.12.3 Communication Interface
2.13 Functional Requirements
2.14 Performance Requirements
2.15 Design Contraints
2.16 System Attributes

2.17 Other Requirements
2.17.1 Safety Requirments
2.17.2 Security Requirments

3. System Design

CHAPTER-3

3.1 Introduction
3.2 Context flow Design
3.3 Data flow Diagram
3.4 Rules Regarding DFD Constraints
3.5 DFD Symbols
3.6 DFD for Admin
3.6.1 DFD for Admin Approval
3.6.2 DFD for Job applications
3.6.3 DFD for Upload data
3.6.4 DFD for HR Manager
3.6.5 DFD for Login
3.6.6 DFD for Login
3.6.7 DFD for Register
3.6.8 DFD for Upload Data
3.6.9 DFD for Data Analyst
3.6.10 DFD for Login
3.6.11 DFD for Register
3.6.12 DFD for Upload Data
3.6.13 DFD for Employee
3.6.14 DFD for Login
3.6.15 DFD for Register
3.6.16 DFD for Upload Data
3.6.17 DFD for Job Vacancies
3.6.18 DFD for Users
3.6.19 DFD for Login
3.6.20 DFD for Register
3.6.21 DFD for Job Vacancies
3.7 Entity relationship Daigram
3.7.1 Data Objects
3.7.2 Attributes
3.7.3 Relationships
3.7.4 Cardinality
3.8 ER Daigram

4. Database Design

CHAPTER-4

4.1 Introduction
4.2 Database Tables
4.3 Database Structure
4.3.1 users Table
4.3.2 roles Table
4.3.3 Job_postings Table
4.3.4 Job_ applications Table
4.3.5 attrition_analytics Table

5. Detailed Design

CHAPTER-5

5.1 Introduction
5.2 Application Documents
5.3 structure of the software package the components
5.4 modular decomposition of components
5.4.1 Admin component
5.4.2 Structure chart for Admin
5.4.3 HR Manager components
5.4.3.1 Structure chart for HR Manager
5.4.4 Data Analyst components
5.4.4.1 Structure chart for Data Analyst
5.4.5 Employee component
5.4.5.1 Structure chart for Employee
5.4.5.2 Users Component
5.4.5.3 Structure chart for Users
5.4.6 Procedural details of Admin
5.4.6.1 Login
5.4.6.2 Dashboard
5.4.6.3 Upload Data
5.4.6.4 Predictor
5.4.6.5 Admin Approval
5.4.6.6 Job Applications
5.5 Procedural details of HR Manager
5.5.1 Registration
5.5.2 login
5.5.3 Upload Data
5.5.4 Predictor
5.6 Procedural details of Data Analyst
5.6.1 Registration
5.6.2 Login
5.6.3 Upload Data
5.6.4 Predictor

6. Coding

CHAPTER-6
6.1 coding

7. TESTING

CHAPTER – 7
7.1 Introduction
7.2 Testing objectives
7.3 Testing steps
7.3.1 Unit testing
7.3.2 Integration testing
7.3.3 Validation
7.3.4 Output testing
7.3.5 User acceptance testing
7.4 Test cases

8. USER INTERFACE

CHAPTER – 8
8.1 Screenshots

9. USER MANUAL

CHAPTER –9
9.1 Introduction
9.2 Hardware requirements
9.3 Software requirements

10. CONCLUSION

CHAPTER –10
10.1 Conclusion

11. BIBLIOGRAPHY

CHAPTER – 11
11.1 Books and Website reference

12. PLAGIARISM REPORT

CHAPTER – 12
12.1 Plagiarism

SYNOPSIS

.1 Title of the project:

Employee Attrition Predictor

.2 Introduction of project:

- The "Employee Attrition Predictor" is a web application designed to estimate the probability of employee departure from a company by using machine learning techniques and data analysis. Turnover, or the loss of employees, can result in higher recruitment expenses and decreased efficiency. The goal of this predictor is to equip organizations with valuable insights into retaining talent by examining critical elements like job satisfaction, performance levels, workplace conditions, and various HR indicators.. This application supports proactive strategies to lower turnover rates and enhance organizational resilience.

.3 Objective of project:

- To determine the key factors of employee attrition and forecast which employees are at risk of leaving.
- To empower organizations to implement proactive measures to retain valuable talent and reduce the adverse effects of employee turnover.
- To offer a feature that highlights job vacancies , enabling organizations to recruit replacements early and mitigate the challenges of attrition.
- To deliver actionable insights and detailed reports to organizations while allowing them to update datasets for ongoing analysis.

.4 Category:

Web Application

.5 Language to be used:

- **F**ront end HTML,CSS, Java script. (React
- Back end Python (Flask /Django)
- Database: MySQL

1.6 Hardware interface:

- RAM: 8GB or Higher
- Processor : Intel i5 or higher
- Storage : 250 GB SSD or Higher

1.7 Software interface:

- Browser: Google Chrome
- Operating system :Windows10, Linux or MacOS
- Dependencies: Python 3.9+, Node.js

1.8 Description

- The Employee Attrition Predictor empowers HR teams to anticipate and address workforce turnover, fostering greater employee retention and workforce stability.
- It features a variety of tools, including options to predict turnover, upload data, and explore job openings.
- Users can browse and apply for available positions.
- These functionalities enable HR to track departure trends and assess risk levels, allowing them to strategize on employee retention or recruitment.
- Whether employees are retiring, resigning, or pursuing new opportunities, unchecked attrition can disrupt a company's productivity, morale, and financial performance.

1.9 Module description:

a) Admin :

- **Login:** Admin will enter the website using user name and password.
- **Dashboard:** In this module , admin can see Hr reports,charts and statistics .
- **Data Upload:** In this module, admin can upload datasets .
- **Predictor:** In this module admin can predict individual employee attrition risk percentage and risk level.
- **Admin Approval:** admin can approve\reject user. Only approved user can login .
- **Job Applications:** In this module the admin can view all the job applications of the users and employees.

b) **HR Manager:**

- **Login:** HR Managers will enter the website using user name and password.
- **Register**: HR Managers can register with HR Manager role.
- **Dashboard:** HR Manager can view HR Reports.
- **Upload Data:** In this module, HR Managers can upload datasets.
- **Predictor:** In this module HR Managers can predict individual employee attrition risk percentage and risk level.

c) **Data Analyst:**

- **Login**: Data Analysts can login using his username and password.
- **Register**: Data Analysts can register using Data Analyst role.
- **Dashboard:** Data Analyst can view attrition trends,department-wise attrition as charts and graphs.
- **Upload Data:** In this module, Data Analyst can upload datasets.
- **Predictor:** In this module Data Analyst can predict individual employee attrition risk percentage and risk level

d) **Employee:**

- **Register:** Employees can register using Employee role
- **Login:** Employees can login using his/her user name and password only after admin approval.
- **Job vacancies:** Employees can view and apply for jobs.
- **Apply now:** Employees can upload resumes and apply fo a job
- **Upload data:** Employees can upload files

- **Predictor:** In this module Employees can predict individual employee attrition risk percentage and risk level.

- **My Applications**: in this module, Employees can view the job applications submitted and track status.

e) Users:

- **Register:** Users can register using Users role
- **Login:** Users can login using his/her user name and password only after admin approval.
- **Job vacancies:** Users can view and apply for jobs.
- **Apply now:** Users can upload resumes and apply for a job
- **My Applications**: in this module, Users can view the job applications submitted and track status

1.11 Future Scope:

- Wide-ranging,focousing on proactive HR Strategies ,improved workforce stability and enhanced talent retention
- By accurately predicting employee turnover ,organizations can implement targeted interventions to reduce attrition rates and retain valuble employees

1.12 Team members:

YADHUNANDH.C(03SU22CC246)
ABHAYDEV A. K(03SU22CC025)

2. SOFTWARE REQUIREMENT SPECIFICATION

2.1 Introduction:

The Employee Attrition Predictor is a web-based tool crafted to forecast employee turnover within an organization. Its primary goal is to equip organizations with valuable insights into employee retention by evaluating critical factors like job satisfaction, performance, workplace conditions, and various HR metrics.

2.2 Purpose:

The Employee Attrition Predictor aims to assist organizations in forecasting employee turnover through machine learning algorithms. By evaluating essential HR data, the tool delivers actionable insights to HR teams, empowering them to implement proactive strategies for employee retention and minimizing recruitment expenses.

2.3 Scope

- Collect employee-related data (job satisfaction, salary, experience, etc.).
- Predict the probability of an employee leaving the organization.
- Provide visualization and insights to HR teams.
- Process and analyze data using machine learning algorithms.

2.4 Definition, Acronyms, Abbreviation

Attrition	Employee turnover or departure from the organization.
API	Application programming interface
ML	Machine Learning
HR	Human Resources

2.1 Overview:

The "Employee Attrition Predictor" is a web-based platform designed to forecast employee turnover rates. It uses machine learning models to help organizations estimate the probability of employee attrition. By evaluating critical HR data, the system delivers actionable insights to HR teams, empowering them to implement proactive strategies for employee retention and minimize recruitment expenses. This Software Requirements Specification (SRS) outlines the essential requirements for developing the Employee Attrition Predictor using Machine Learning technology.

2.2 Overall description:

This section provides a comprehensive overview of the entire system, detailing its interactions with external systems and outlining its core functionalities. It will also specify the types of users who will engage with the system and the distinct features available to each user group. Finally, the system's constraints and assumptions will be highlighted

2.3 Productive perspective:

The Employee Attrition Predictor is an independent web-based application that connects with existing HR platforms through APIs. The system categorizes its users into Admin, HR Manager, Data Analyst, Employee, and Users, with each group accessing the platform online. It offers HR teams a straightforward method to pinpoint employees who are more likely to leave the organization. The system features an intuitive and user-friendly graphical user interface (GUI), designed to be accessible for both new and expirienced users.

2.4 Product Function :

- Data Collection: Import employee data from CSV/Excel/API
- Data Processing: Clean and preprocess data for machine learning models.
- Prediction Model: Train and test ML models for attrition prediction.
- Visualization: Display attrition trends using charts and graphs.
- Admin Dashboard: Manage employee records and track predictions.

2.5 User classes and characteristics:

The system has five user levels.

a) **Admin:**

- **Login:** Admin will enter the website using user name and password.
- **Dashboard:** admin can see Hr reports,charts , graphs and statistics .

- **Data Upload:** admin can upload datasets .
- **Predictor:** admin can predict individual employee attrition risk percentage and risk level.
- **Admin Approval:**admin can approve\reject user. Only approved user can login .
- **Job Applications:** admin can view all the job applications of the users and employees.

b) **HR Manager:**

- **Login:** HR Managers will enter the website using user name and password.
- **Register**: HR Managers can register with HR Manager role.
- **Dashboard:** HR Manager can view HR Reports.
- **Upload Data:** HR Managers can upload datasets.
- **Predictor:** HR Managers can predict individual employee attrition risk percentage and risk level

c) **Data Analyst:**

- **Login**: Data Analysts can login using his username and password.
- **Register**: Data Analysts can register using Data Analyst role.
- **Dashboard:** Data Analyst can view attrition trends,department-wise attrition as charts and graphs.
- **Upload Data:** Data Analyst can upload datasets.
- **Predictor:** Data Analyst can predict individual employee attrition risk percentage and risk level

d) Employee:

- **Register:** Employees can register using Employee role
- **Login:** Employees can login using his/her user name and password only after admin approval.
- **Job vacancies:** Employees can view and apply for jobs.
- **Apply now:** Employees can upload resumes and apply fo a job
- **Upload data:** Employees can upload files
- **Predictor:** In this module Employees can predict individual employee attrition risk percentage and risk level.
- **My Applications**: in this module, Employees can view the job applications submitted and track status.

e) Users:

- **Register:** Users can register using Users role
- **Login:** Users can login using his/her user name and password only after admin approval.
- **Job vacancies:** Users can view and apply for jobs.
- **Apply now:** Users can upload resumes and apply for a job
- **My Applications**: in this module, Users can view the job applications submitted and track status

2.6 General Constraints:

General constraints include the following:

- Customers must register and get confirmation from the admin to access the website for the first time.
- Must comply with GDPR and data privacy regulations.
- Requires real-time processing capabilities.
- Supports only structured employee datasets.

2.7 Assumptions and Dependencies:

- All the data entered will be correct and up to date.
- Users have basic technical knowledge.
- The organization provides historical employee data for model training.

2.12 Specific Requirements:

External Interface Requirements:

User Interface:

- Login Page: Secure authentication for users.
- Dashboard: Display attrition trends and key metrics.
- Data Upload Page: Allows CSV/Excel uploads.
- Predict page : allows users to predict individual employee risk percentage and risk level
- Job vacancies page : allow employees and users to view and apply for jobs
- Apply page: Allow employees and users to upload resume and apply for job

2.12.1 Software Requirements:

- Operating System: Windows
- Programming Languages: Python (Flask/Django), JavaScript (React/Angular)., HTML,CSS
- Database: MySQL
- Machine Learning Library: Scikit-learn, TensorFlow.
- Visualization Tools: Matplotlib, Seaborn, Plotly
- Browser: Google Chrome or any other browser

2.12.2 Hardware Requirements:

- Processor: Intel i5 or higher.
- RAM: 8 GB or higher.
- Storage: 250 GB SSD or higher.

2.12.3 Communication Interface:

The communications function required by this product is HTTP protocol, and internet communication is through TCP/IP protocol.

2.13 Functional Requirements:

a) Admin:

- **Login:** Admin will enter the website using user name and password.
- **Dashboard:** In this module , admin can see Hr reports,charts and statistics .
- **Data Upload:** In this module, admin can upload datasets .
- **Predictor:** In this module admin can predict individual employee attrition risk percentage and risk level.
- **Admin Approval:** admin can approve\reject user. Only approved user can login .
- **Job Applications:** In this module the admin can view all the job applications of the users and employees.

b) **HR Manager:**

- **Login:** HR Managers will enter the website using user name and password.
- **Register**: HR Managers can register with HR Manager role.
- **Dashboard:** HR Manager can view HR Reports.
- **Upload Data:** In this module, HR Managers can upload datasets.
- **Predictor:** In this module HR Managers can predict individual employee attrition risk percentage and risk level

c) **Data Analyst:**

- **Login**: Data Analysts can login using his username and password.
- **Register**: Data Analysts can register using Data Analyst role.
- **Dashboard:** Data Analyst can view attrition trends,department-wise attrition as charts and graphs.
- **Upload Data:** In this module, Data Analyst can upload datasets.
- **Predictor:** In this module Data Analyst can predict individual employee attrition risk percentage and risk level

d) **Employee:**

- **Register:** Employees can register using Employee role
- **Login:** Employees can login using his/her user name and password only after admin approval.
- **Job vacancies:** Employees can view and apply for jobs.
- **Apply now:** Employees can upload resumes and apply fo a job
- **Upload data:** Employees can upload files
- **Predictor:** In this module Employees can predict individual employee attrition risk percentage and risk level.
- **My Applications**: in this module, Employees can view the job applications submitted and track status

e) Users:

- **Register:** Users can register using Users role
- **Login:** Users can login using his/her user name and password only after admin approval.
- **Job vacancies:** Users can view and apply for jobs.
- **Apply now:** Users can upload resumes and apply for a job
- **My Applications**: in this module, Users can view the job applications submitted and track status

2.14 Performance Requirements:.

- get uploaded in 60 sec.
- System should process large datasets efficiently.
- System should process predictions within 2 seconds.
- Should be error-free
- 1MB file should

2.15 Design Constraints:.

- All mandatory fields should be filled by the user while registering
- role-based access.each role is redirected to their respective pages
- Responsive and user-friendly UI.

2.16 System Attributes:

- **Performance**: It defines how well software system accomplishes certain functions under specific conditions.
- **Reliability:** The system that performs correctly during a specific time.

- **User-friendly Interfaces:** An interface that is created for specific group of people or without a clear target audience but aimed at maximum user interface.

- **Maintainability:** The ease with which you can repair, improve, and understand software code.

- **Portability:** This system shall be portable, and we can switch the server very easily.
- **Flexibility:** The ability of a system to respond to potential internal or external changes effecting its value delivery, in a timely and cost affecting manner.
- **Security**: Data encryption and role-based access is implemented

2.17 Other Requirements:

2.17.1 Safety Requirements:

- There are five user levels , Access to the various subsystems will be protected by a user log-in screen that requires a username and password. This gives different views and accessible functions of user levels through the system

- Login is using json token so only authenticated users can login.

- Passwords are protected replacing it with hash in database

2.17.2 Security Requirements:

- Data encryption (AES-256) for sensitive HR data.
- Regular security audits and compliance checks.
- Data encryption and role-based access.

3 SYSTEM DESIGN

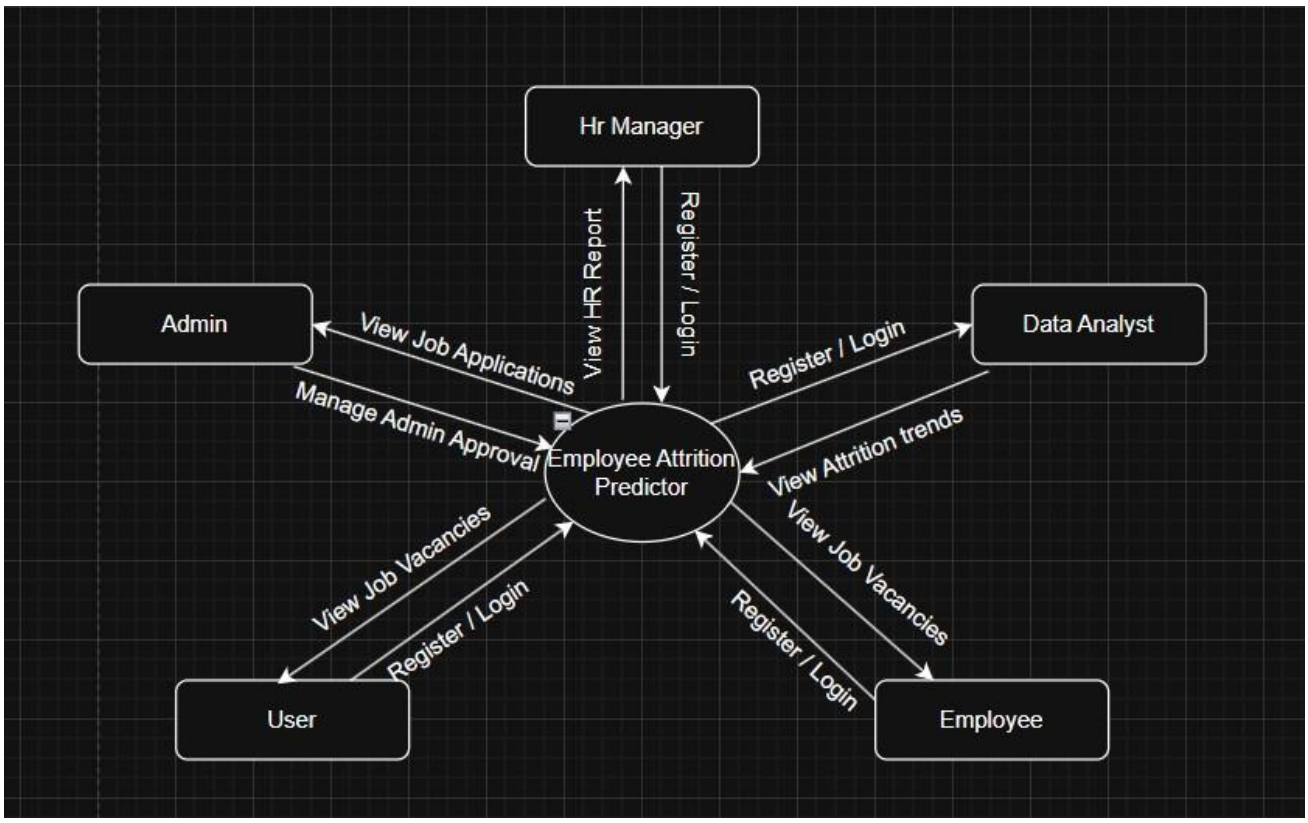
3.12 Introduction:

The objective of the design process is to create a model or blueprint of a system that serves as a foundation for its development. System design involves outlining the architecture, components, interfaces, and data structures to meet predefined requirements. The primary focus is on identifying necessary components, specifying their functionalities, and determining how they will interact within the system.

System design can be viewed as the practical application of systems theory to the creation of products. It follows a methodical approach, employing either a top-down or bottom-up strategy, while systematically addressing all relevant aspects of the system to be developed. The design process shapes the system's core structural properties, significantly influencing its testability and adaptability. The result is a detailed architectural plan for constructing the software system.

3.13 Context Flow Diagram:

Context flow diagram is a top-level data flow diagram. It only contains one process node that generalizes the function of the entire system in relationship to external entities. In context diagram the entire system is treated as a single process and all its inputs, outputs, sinks and sources are identified and shown.



3.14 Data Flow diagram:

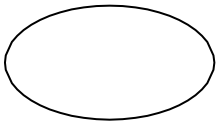
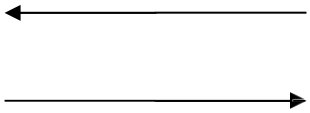
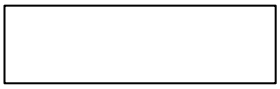
Data Flow Diagram is a graphical representation of a system or a portion of the system. It consists of dataflows, process, sources and sink and stores all the description through the use of easily understandable symbols.

DFD is one of the most important modelling tools. It is used to model the system, components that interact with thesystem, uses the data and information flows in the system.

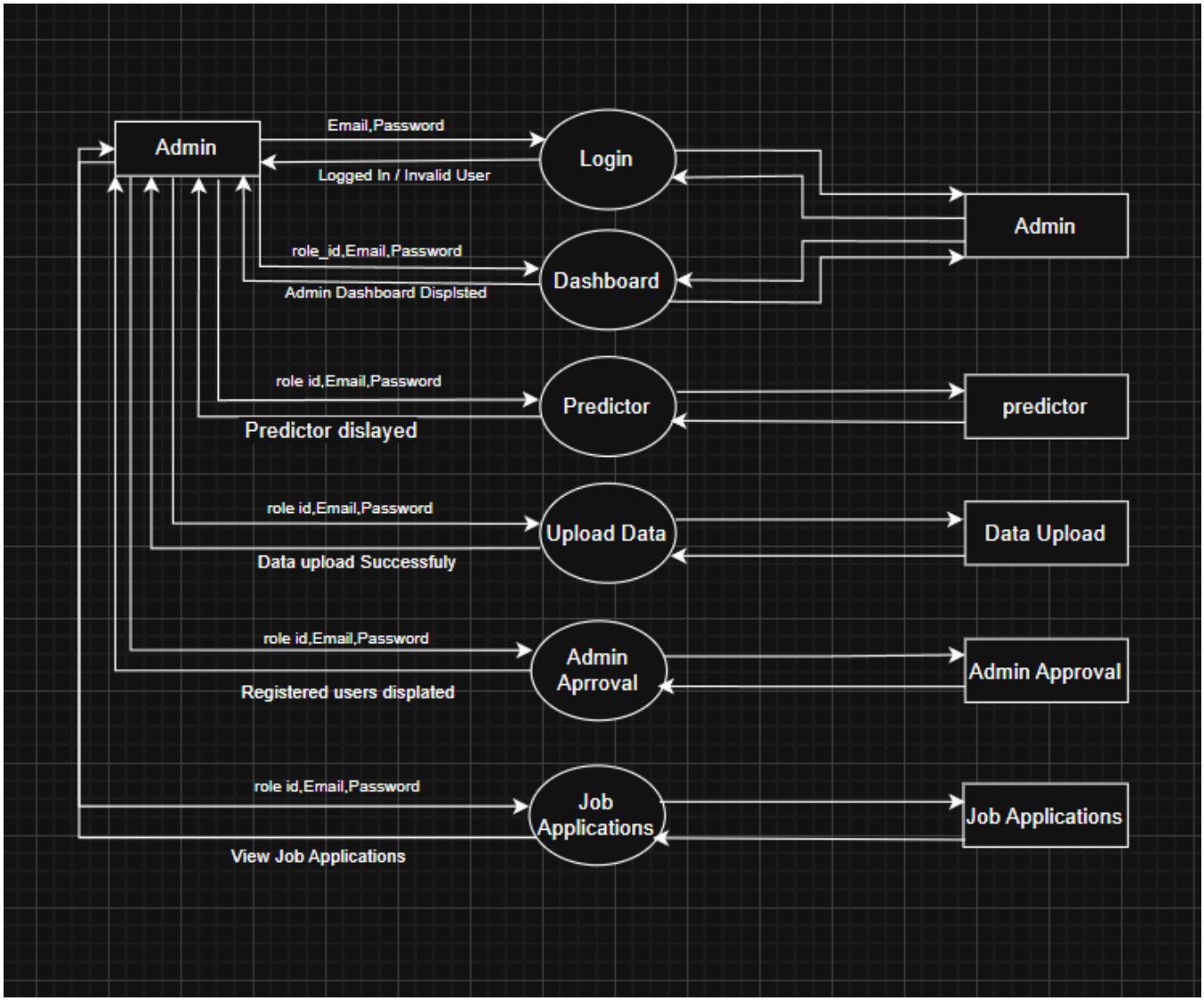
DFD shows the information moves through the and how it is modified by a series of transformations. It is a graphicaltechnique that depicts information moves from input or output.

DFD is also knows as bubble chart or Data Flow Graphs. DFD may be used to represent the system at any level ofabstraction.

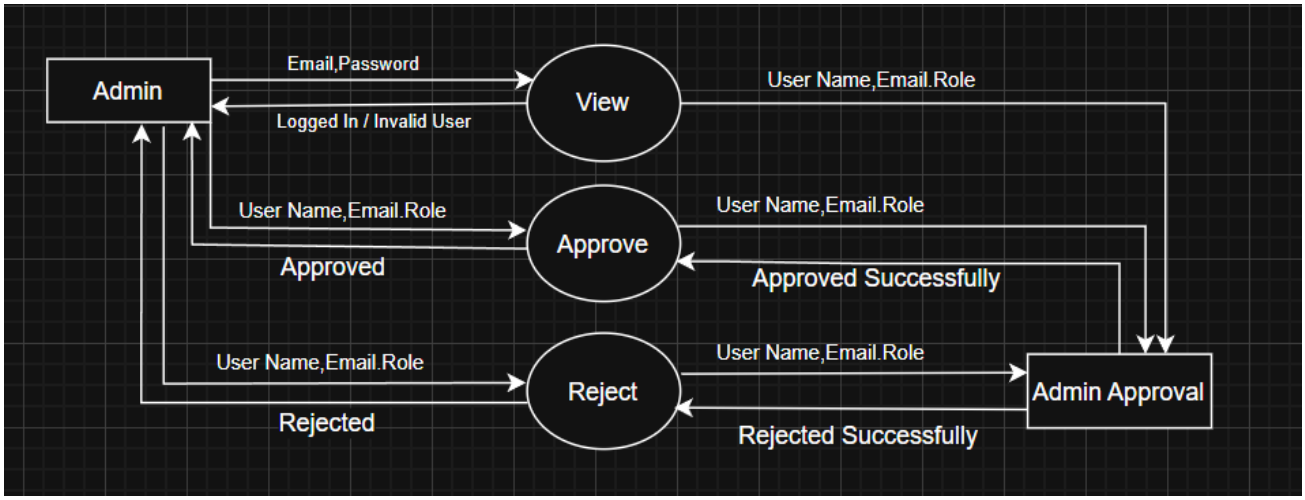
3.15 DFD Symbols:

Name	Notation	Description
Process		A process transforms incoming data flow into outgoing data flow. The processes are shown bynamed circles.
Datastore		Data stores are repositories of data in the system.They are sometimes also referred to as files.
Dataflows		Data flows are pipelines through which packets of information flow. Label the arrows with the name ofthe data that moves through it.
External Entity		External entities are objects outside the system with which the system communicates. External Entities are sources and destinations of the system‟s inputs and outputs

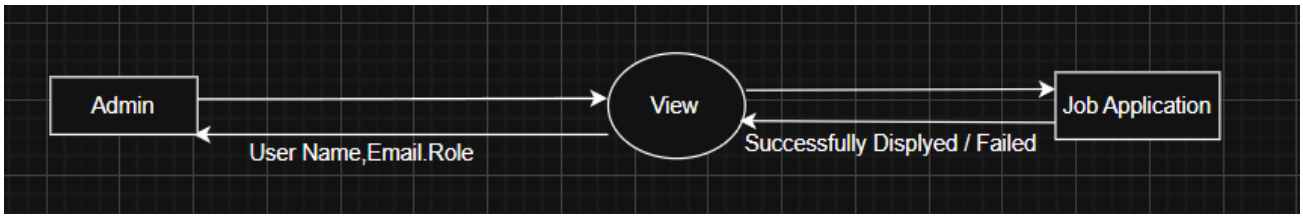
2.12 DFD Level 1(Admin):



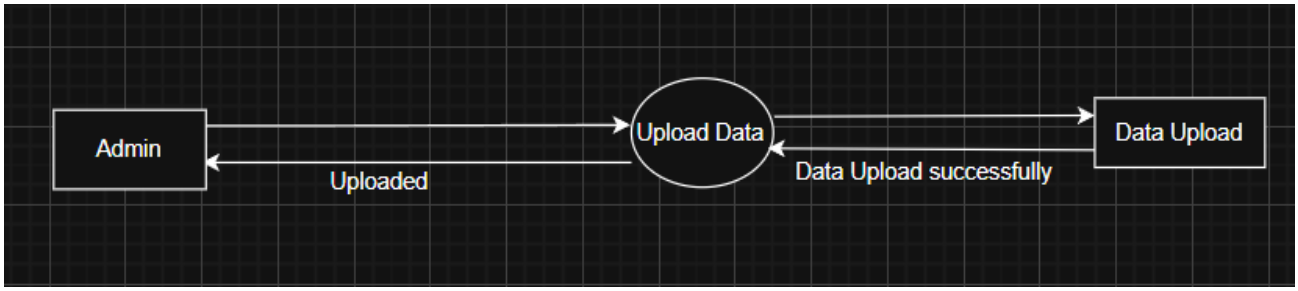
3.17.1 DFD Level 2 (Admin Approval):



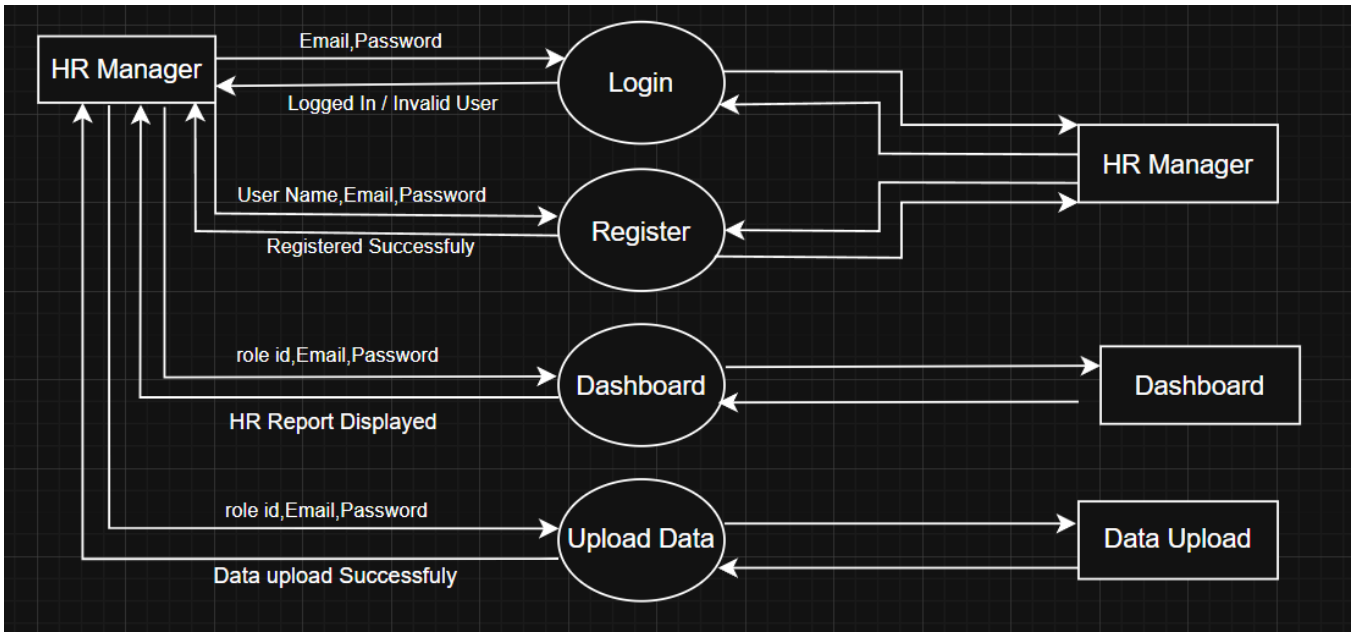
2.12.1 DFD level 2(Job Applications):



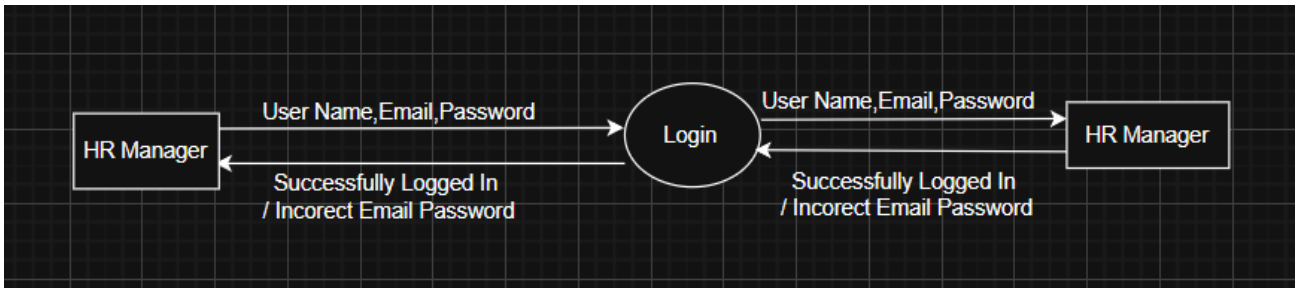
2.12.1 DFD level 2(Upload Data):



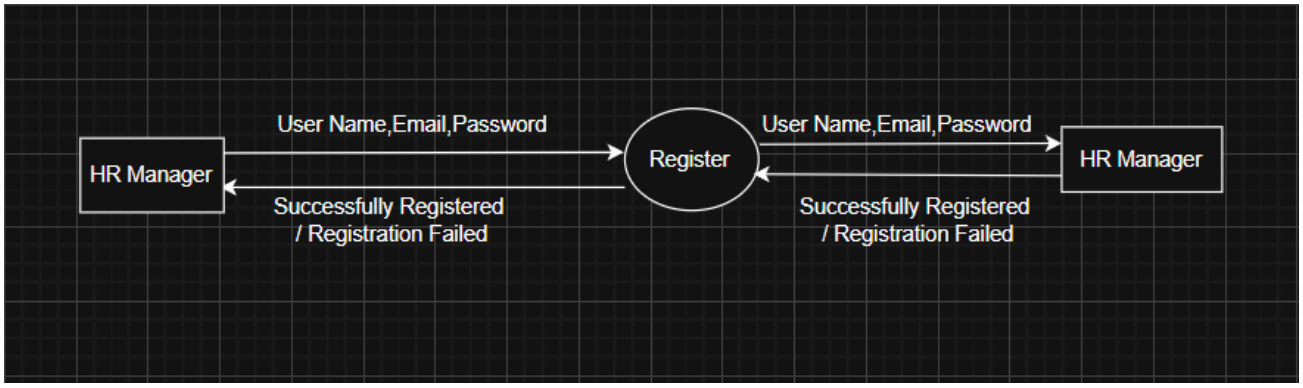
DFD level 1 (HR Manager):



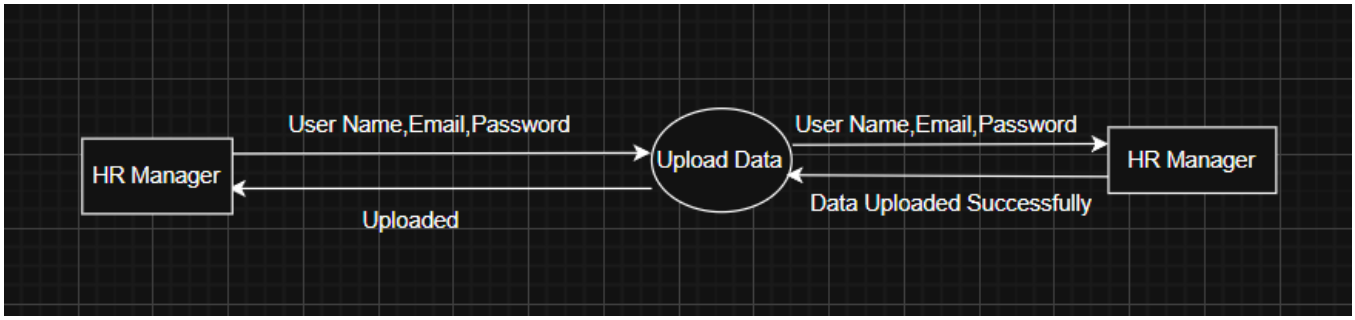
DFD level 2(Login):



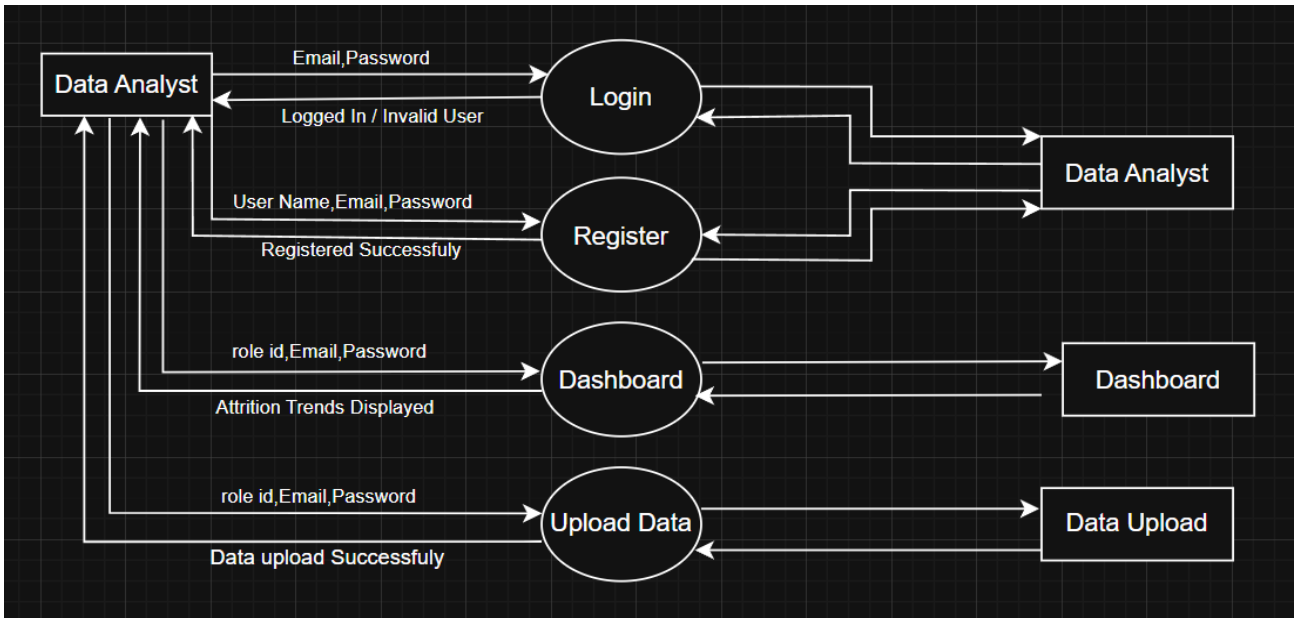
3.6.7 DFD level 2(Register):



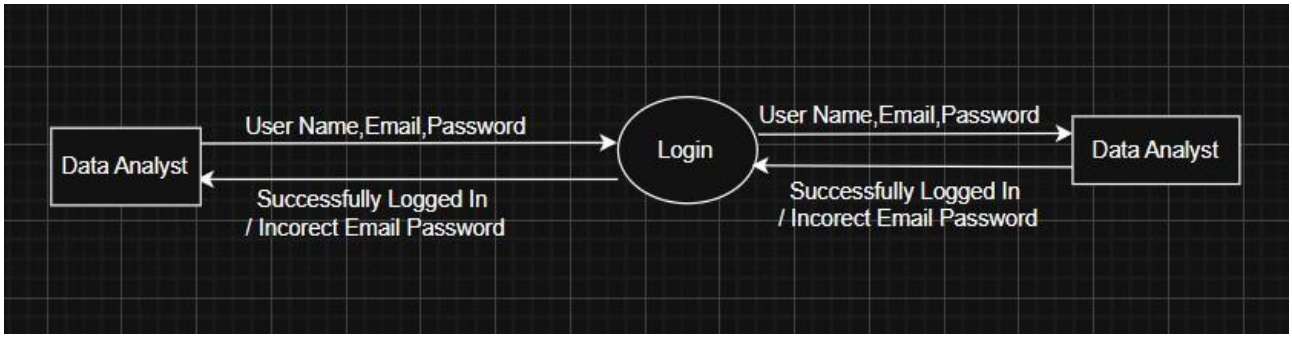
3.6.7 DFD level 2(Upload Data):



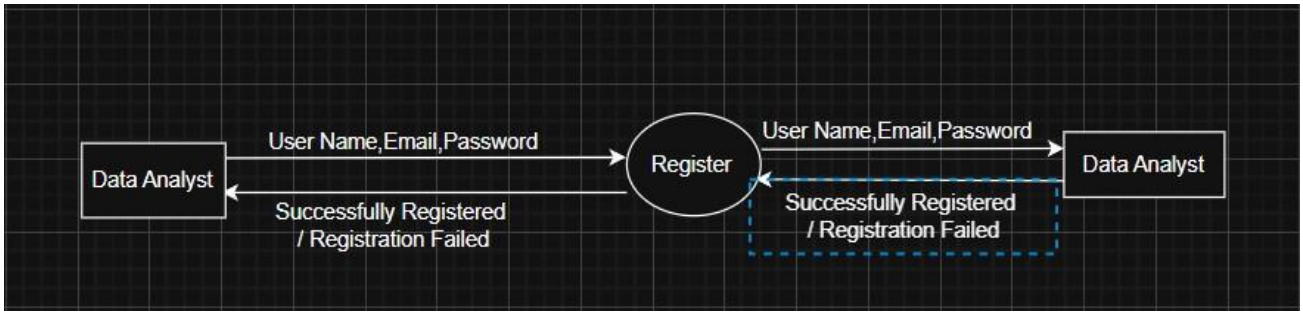
3.6.7 DFD level 1(Data Analyst):



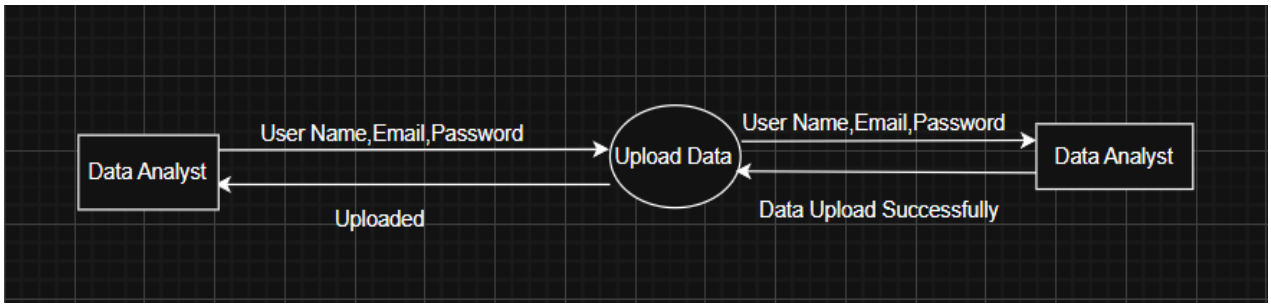
3.6.7 DFD level 2(Login):



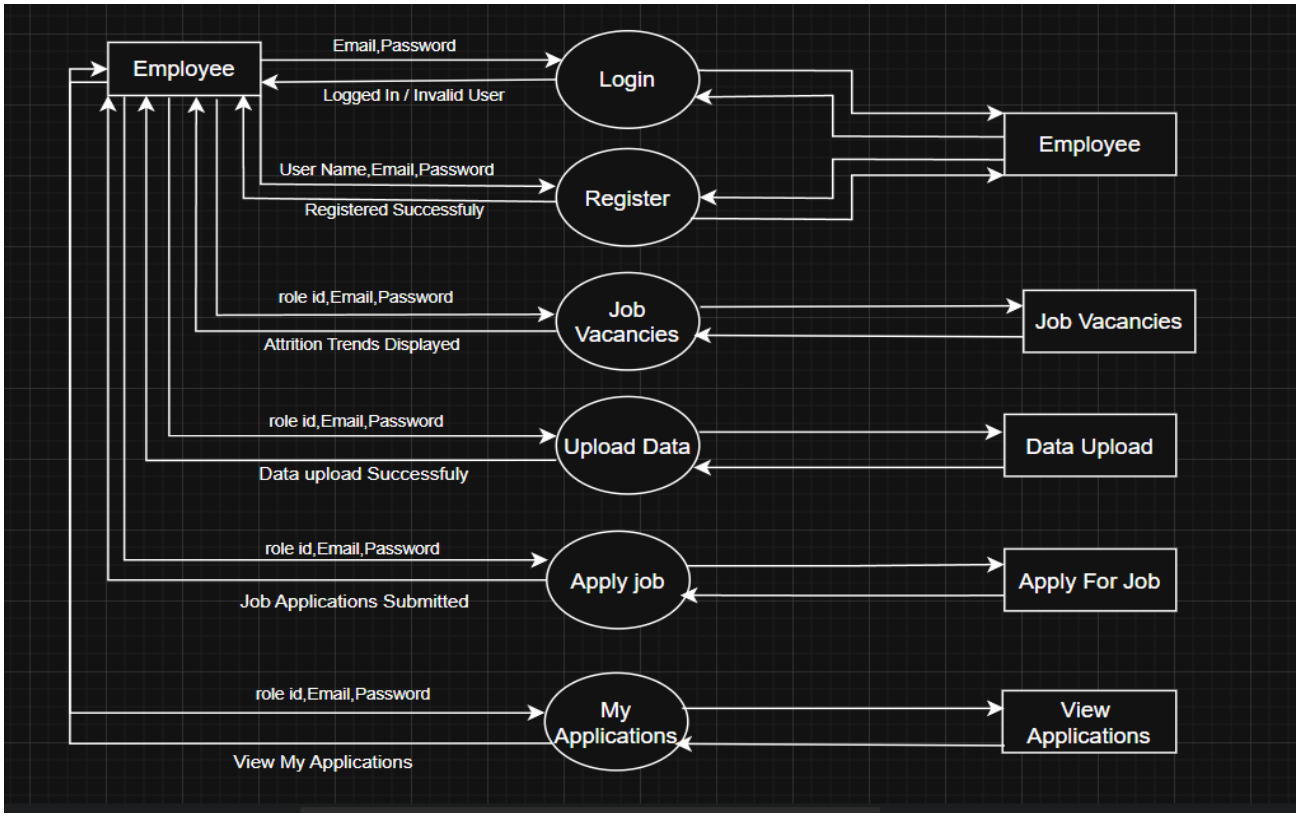
3.6.7 DFD level 2(Register):



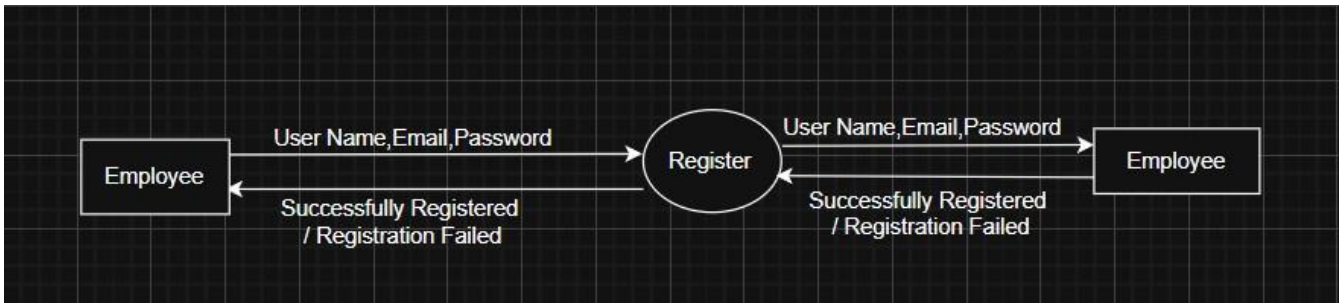
DFD level 2(Upload Data):



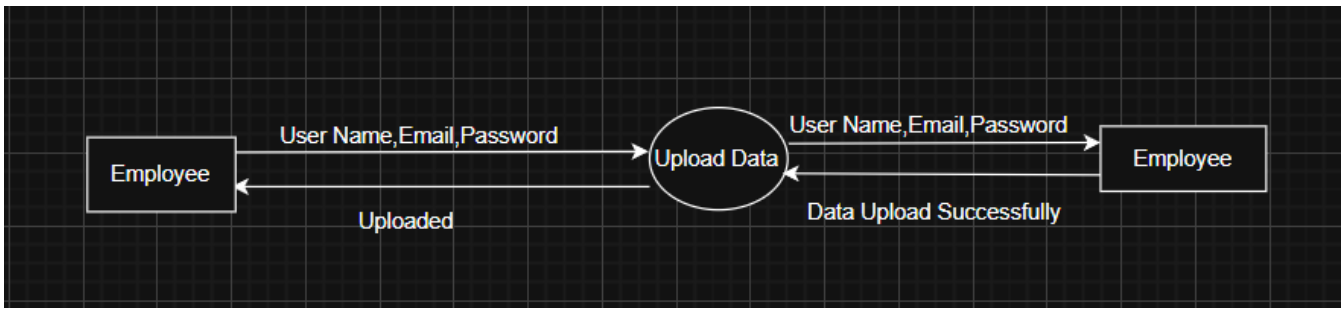
DFD level 1(Employee):



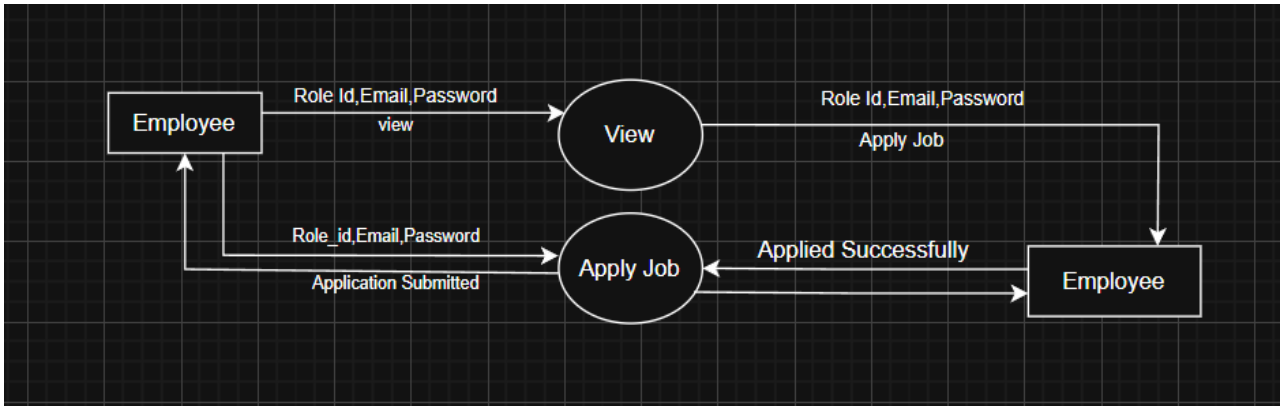
3.6. 23 DFD level 2(Register):



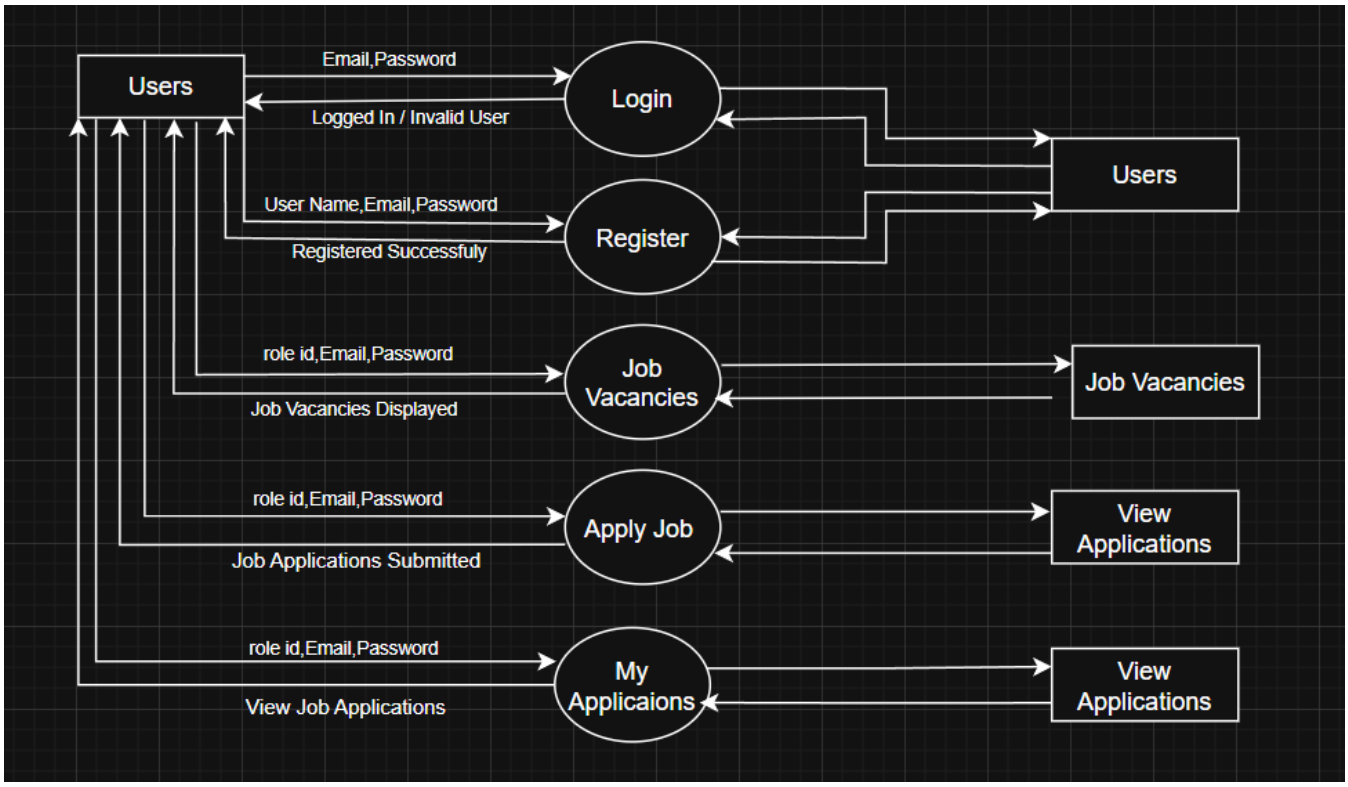
3.6.24 DFD level 2(Upload Data):



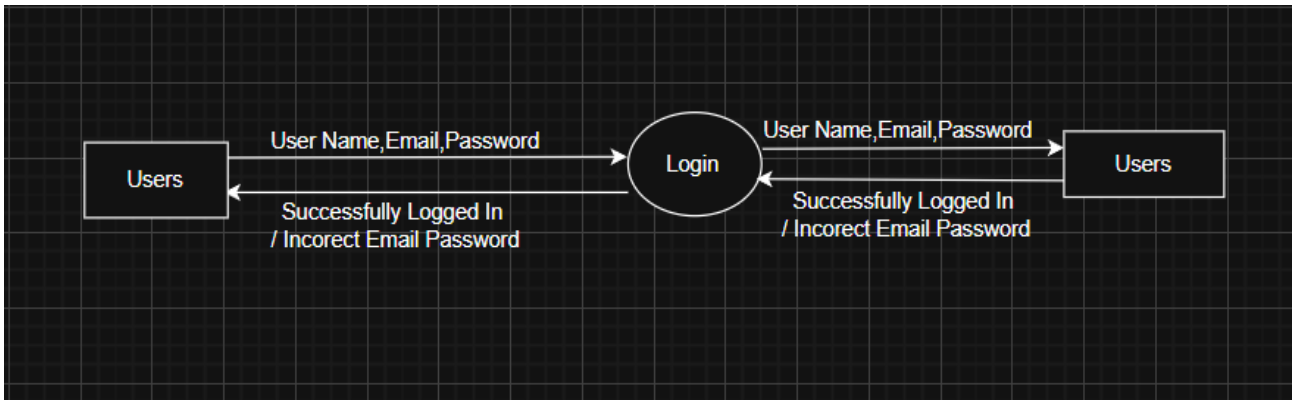
3.6 25 DFD level 2(Job Vacancies):



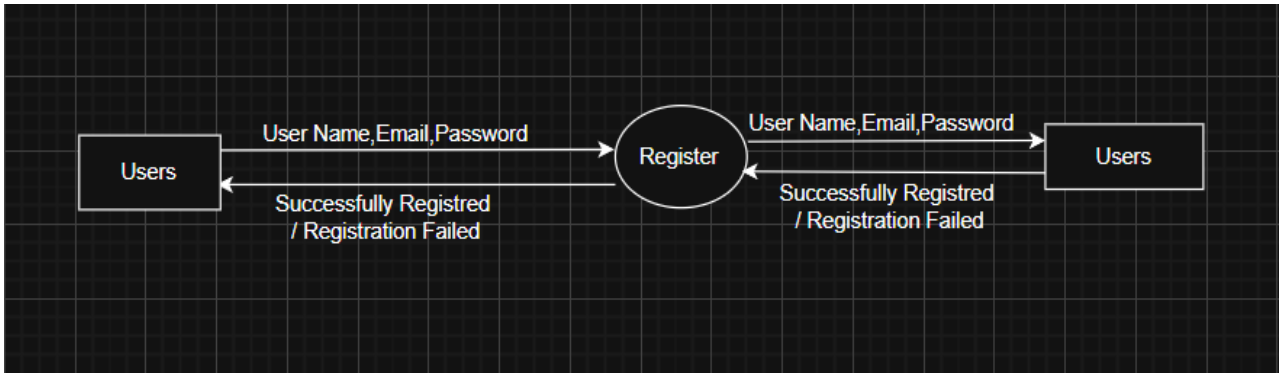
3.6 28 DFD level 1 (Users):



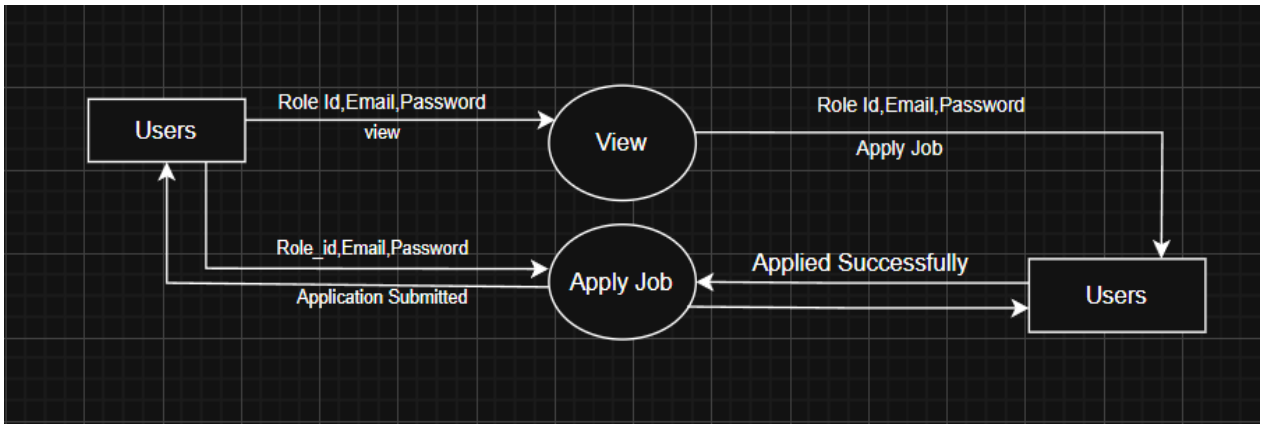
DFD level 2(Login):



DFD level 2(Register):



DFD level 2(Job Vacancies):



3.7 Entity Relationship Diagram

The ER Diagram comprises key elements, including data objects, attributes, relationships, and different type indicators. Its main function is to illustrate the data objects and the connections between them.

3.7.1 Data Objects

A data object is a representation of almost any composite information that must be understood by software. Composite information refers to something that has a number of different properties or attributes.

3.7.2 Attributes

Attributes defines the properties of data object and take on one of three different characteristics. They can be used to (1) name an instance of the data object (2) describe the instance or (3) make reference to another instance in another table. In addition, one or more of the attributes must be defined as an identifier that is the identifier attribute becomes a “key” when we want to find an instance of the data object.

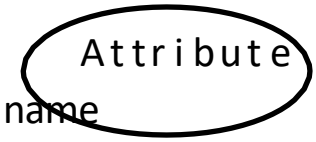
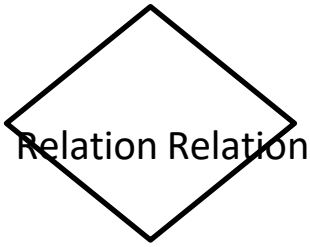
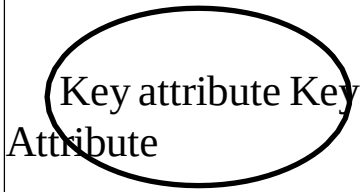
3.7.3 Relationships

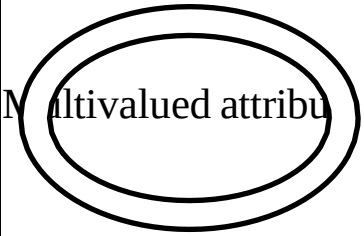
Indicate the manner in which data objects are “connected” to one another

3.7.4 Cardinality

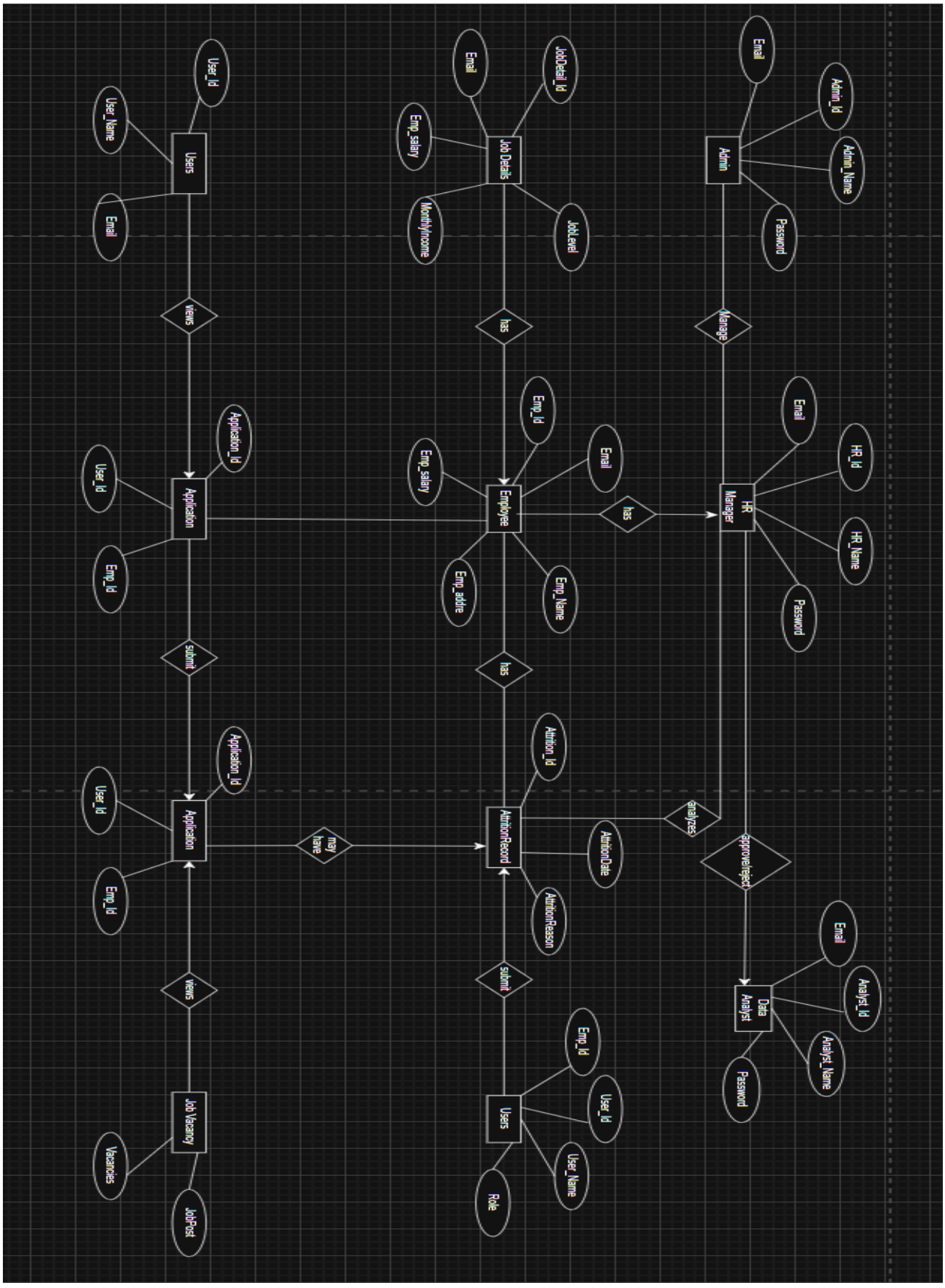
The data model must be capable of representing the number of occurrences objects in a given relationships. Tillman defines the cardinality of an object/relationships pair in the following manner: “Cardinality is the specification of the number of occurrences of one [object] that can be related to the number of occurrences of another [object]. Cardinality is usually expressed as simply „one“ or „many““. Taking into consideration all combinations of „one“ and „many“, two [object] can be as:

The Symbols are shown in below table:

Name	Notation	Description
Entity		It may be an object with the physical existence or conceptual existence. It is represented by a Rectangle.
Attribute		The properties of the entity can be aattribute. It is represented by a Ellipse.
Relationship		Whenever an attribute of one entityrefers to another entity, some relationship exists. It is representedby a Diamond.
Link		Lines link attributes to entity sets andentity sets to relation.
Derived attribute		Dashed ellipse denote derivedattributes.
Key Attribute		An entity type usually has an attribute whose values are distinct for each individual entry in the entity set. It is represented by aUnderlined word in ellipse.

Multivalu ed Attribute		Attributes that have different numbers ofvalues for a particular attribute.It is represented by a Doubleellipse representsmulti-valued attributes.
Cardinality Ratio	1) 1:1 2) 1:M 3) M:1 4) M:M	It specifies the maximum number of relationships instances that an entity can participate in. There are four cardinality ratios.

3.8 ER diagram:



4 .DATABASE DESIGN

4.1 Introduction:

- **Database:** A Database is collection of related data, which can be of any size and complexity. By using the concept of Database, we can easily store and retrieve the data. The major purpose of a database is to provide the information, which utilizes it with the information‟s that the system needs according to its own requirements.

- **Database Design:** Database design is done before building it to meet needs of endusers within a given information-system that the database is intended to support. The database design defines the needed data and data structures that such a database comprises The database is physically implemented using MySQL.

4.2 The database for “Employee Attrition Predictor ”is organized into 5 tables:

- users
- roles
- job_postings
- job_applications
- attrition_analytics

4.3 Database structure

4.3.1 Structure of Table ”users”:

Name	Type	Collation	Attribute	Null	Default	Comment	Extra
id	int(11)			No	None		AUTO_INCREMENT
username	Varchar(50)	utf8mb4_general_ci		No	None		
email	Varchar(120)	utf8mb4_general_ci		No	None		
Password_hash	Varchar(200)	utf8mb4_general_ci		No	None		
role_id	int(11)			No	None		
status	Varchar(20)	utf8mb4_general_ci		No	pending		
Attrition_risk	float			Yes	NULL		

4.1.1 Structure of Table “roles”:

Name	Type	Collation	Attributes	Null	Default	Comments	Extra
id	int(11)			NO	None		AUTO_INCREMENT
name	varchar(50)	utf8mb4_general_ci		NO	None		

4.3.3 Structure of Table “job_postings”:

Name	Type	Collation	Attributes	Null	Default	Comments	Extra
id	int(11)			**NO**	**None**		AUTO_ INCREMENT
title	Varchar (100)	utf8mb4_general_ci		**NO**	**None**		
job_type	Varchar (50)	utf8mb4_general_ci		**NO**	**None**		
department	Varchar (100)	utf8mb4_general_ci					
salary	float						

experience_required	Varchar (100)	utf8mb4_general_ci		**NO**	**None**		
Skills_required	text	utf8mb4_general_ci		**NO**	**None**		
Job_description	text	utf8mb4_general_ci		**NO**	**None**		
Posted_date	datetime			**NO**	**None**		
application_deadline	datetime			**NO**	**None**		

4.3.4 structure of Table "job_applications":

Name	Type	Collation	Attribute	Null	Default	Comment	Extra
id	int(11)			No	None		AUTO_INCREMENT
job_id	int(11)			No	None		
applicant_name	Varchar (100)	utf8mb4_general_ci		No	None		
email	Varchar (120)	utf8mb4_general_ci		No	None		
resume_link	Varchar (255)	utf8mb4_general_ci		Yes	NULL		
resume_file	Varchar (255)	utf8mb4_general_ci		Yes	NULL		
application_date	datetime			Yes	NULL		
job_role	Varchar (100)	utf8mb4_general_ci		No	None		
department	Varchar (100)	utf8mb4_general_ci		No	None		
cover_letter	text	utf8mb4_general_ci		Yes	NULL		
user_id	int(11)			No	None		
status	Varchar (50)	utf8mb4_general_ci		Yes	NULL		

4.3.5 Structure of Table “attrition_analytics”:

Name	Type	Collation	Attribute	Null	Default	Comment	Extra
id	int(11)			No	None		AUTO_INCREMENT
department	Varchar (100)	utf8mb4_general_ci		No	None		
Attrition_ rate	float			No	None		
last_ updated	datetime			Yes	NULL		

5 . DETAILED DESIGN

5.1. Introduction

The purpose of preparing this document is to explain complete design details of Employee Attrition Predictor. This detailed design report will mainly contain the general definition and features of the project, design constraints, the overall system architecture and data architecture. Additionally, a brief explanation about our current progress and schedule of the project will be provided in related sections. Design of the system and subsystems/modules will be explained both verbally and visually by means of diagrams in order to help the programmer to understand all information stated in this document correctly and easily,

5.2. Applicable documents

The documents used during detailed design are :

- System Requirements Document
- System Design
- Database Design

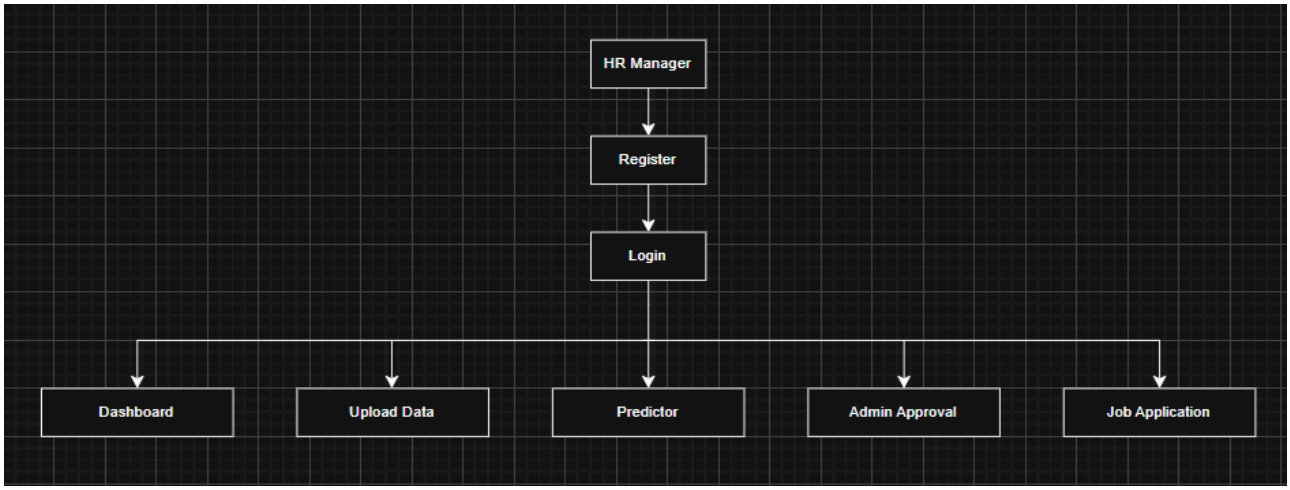
5.3 Structure of the software package The Components are

- Registration component for Users, Employee, Hr Manager, Data Analyst.
- Login component for Users, Employee, Hr Manager, Data Analyst, Admin.
- dashboard component Users, Employee, Hr Manager, Data Analyst, Admin
- predictor component for Employee, Hr Manager, Data Analyst, Admin.
- File upload components Employee, Hr Manager, Data Analyst, Admin.
- Job vacancies component for Users, Employee.

5.4 Modular decomposition of components

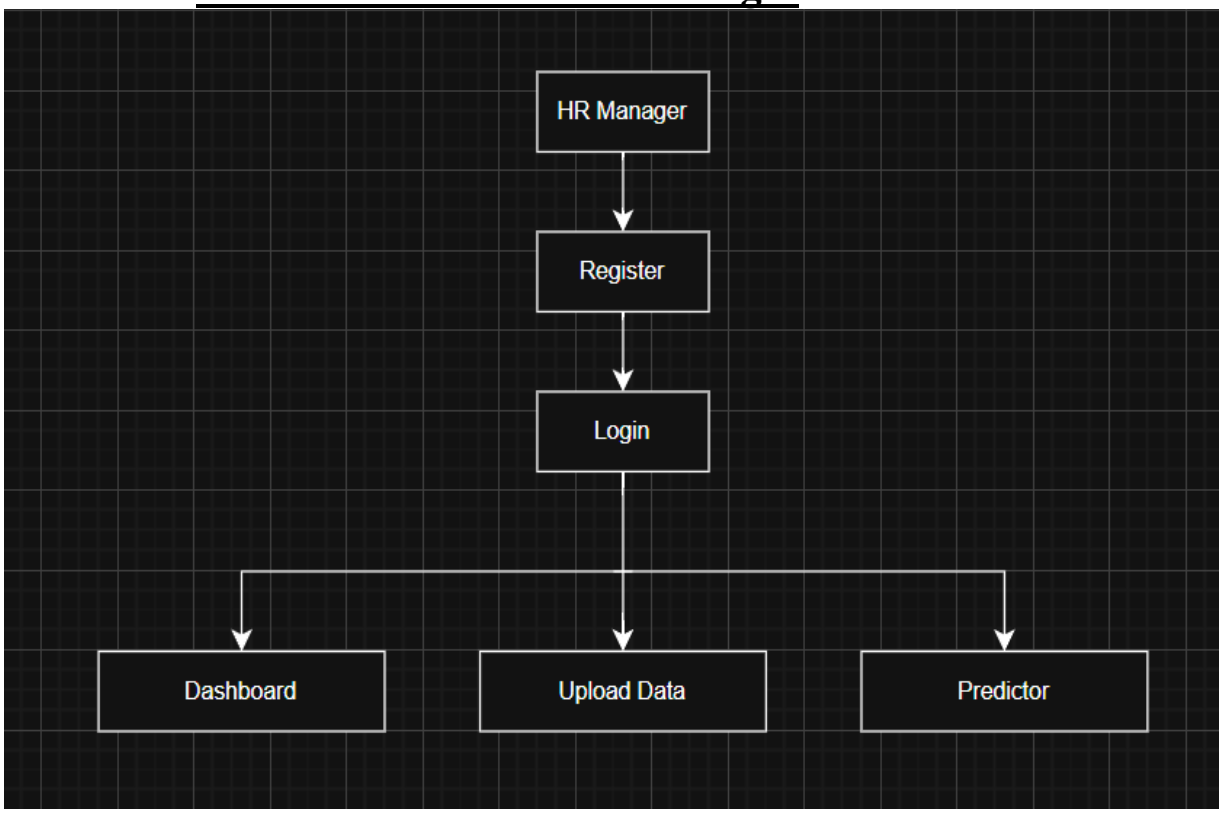
5.4.1 Admin component

5.4.1.1 Structure chart for Admin



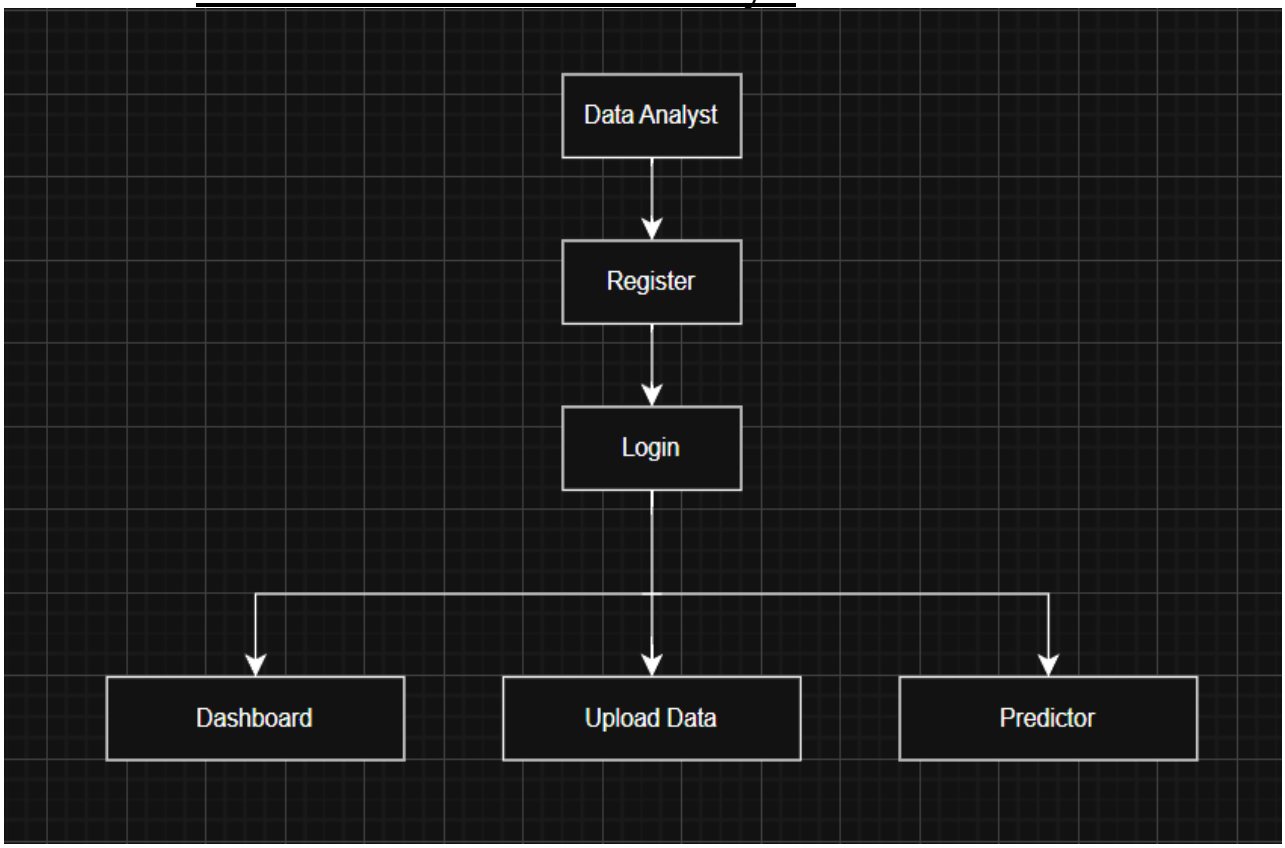
5.3.1 HR Manager component

5.3.1.1 Structure chart for HR Manager



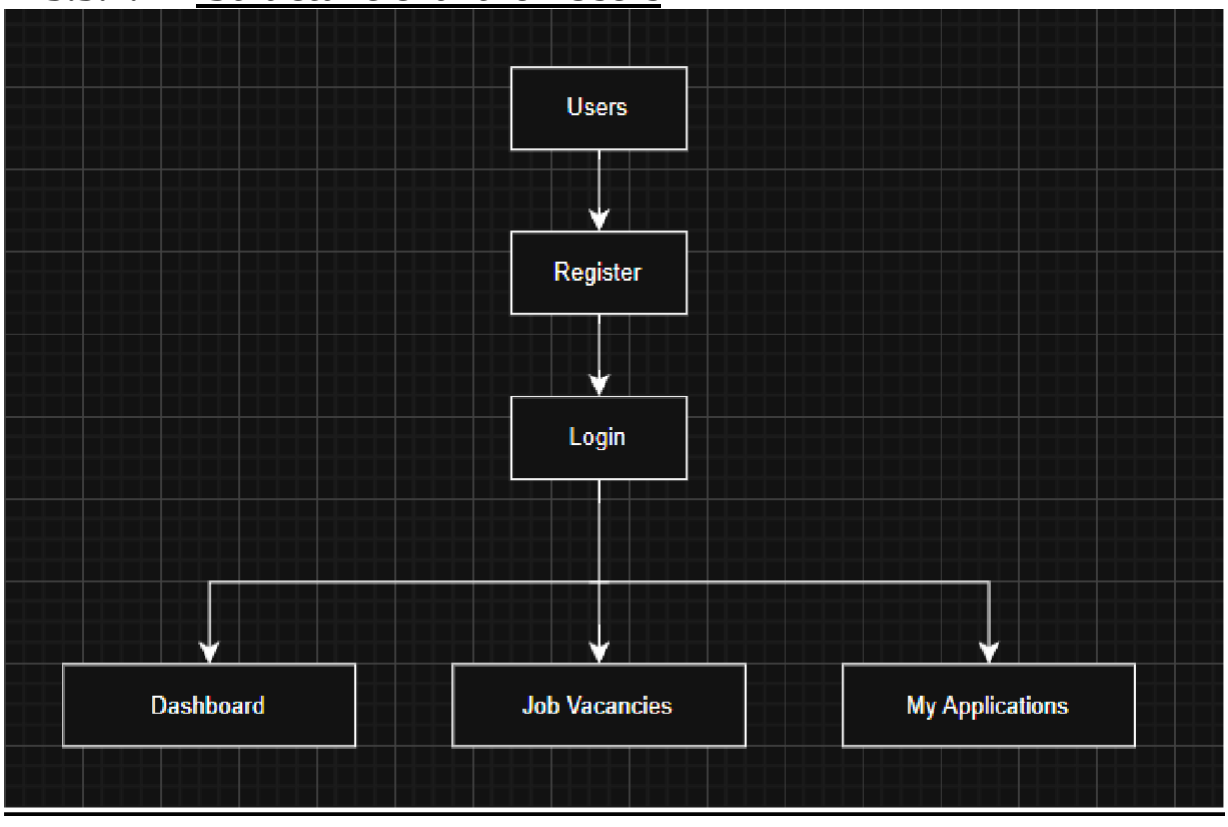
5.3.2 Data Analyst component

5.3.2.1 Structure chart for Data Analyst



5.3.1 Users component

5.3.1.1 Structure chart for Users

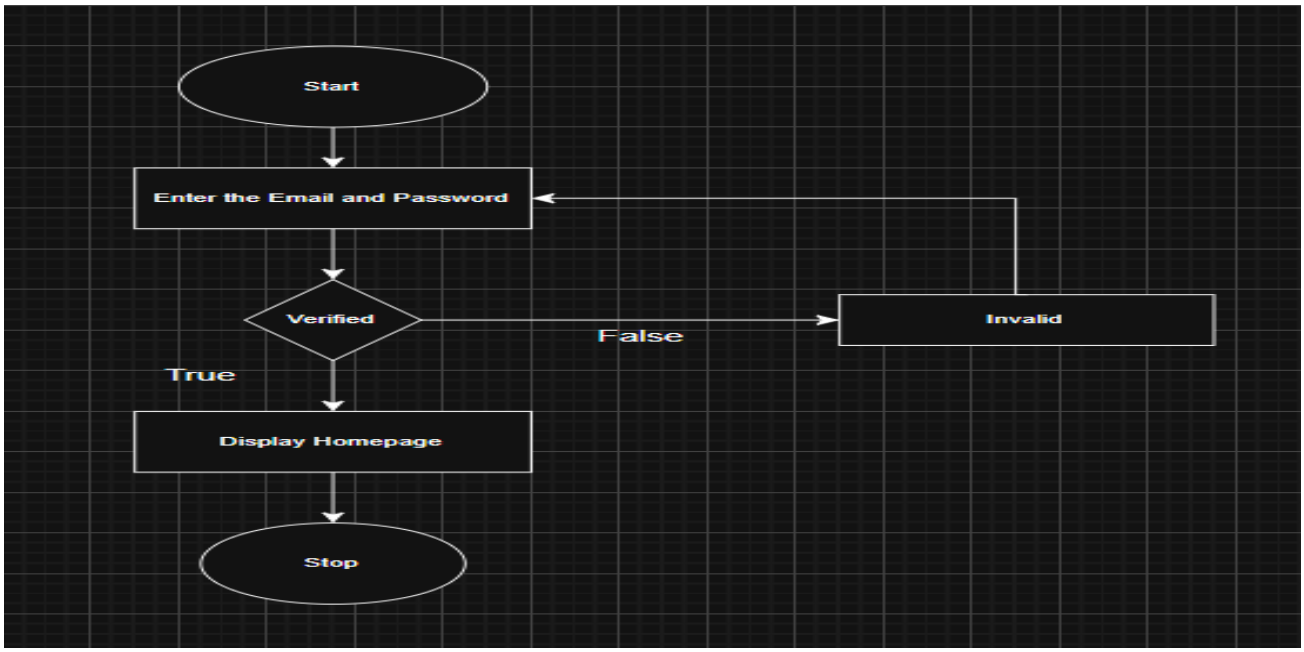


5.3.1 Procedural details of Admin

5.3.1.1 Login:

Input: Email_id,pass

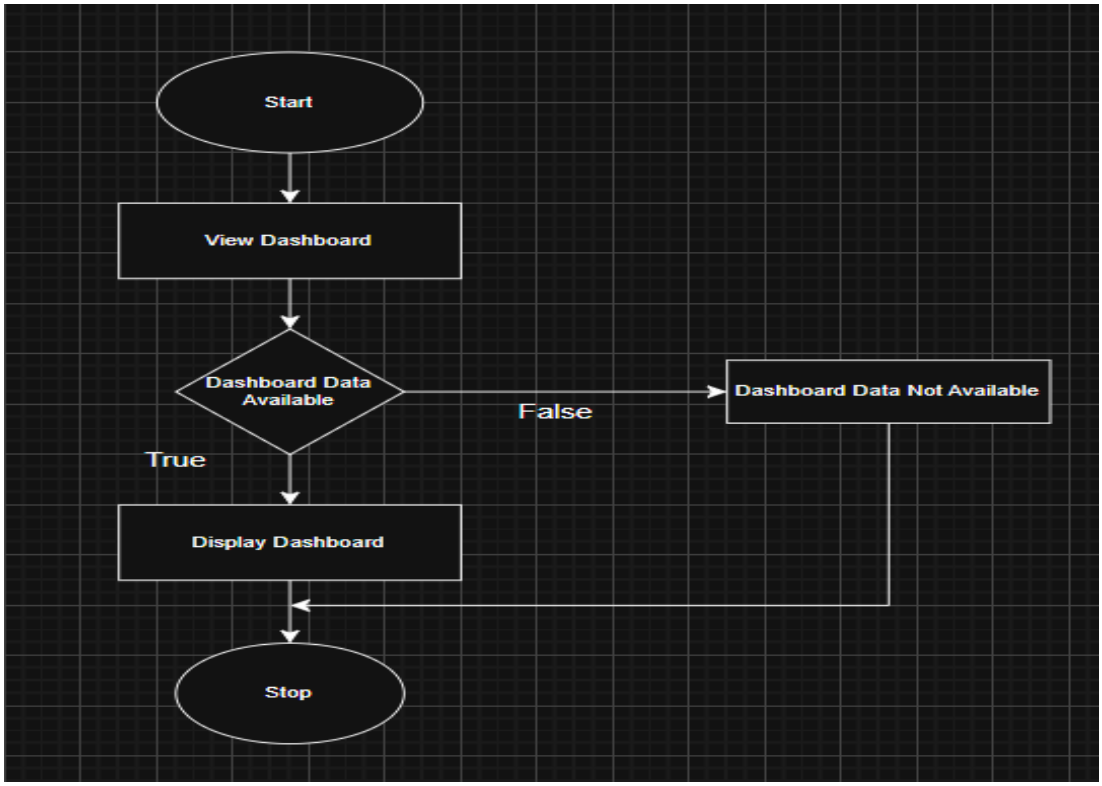
Procedural details:



5.4.4.2 Dashboard:

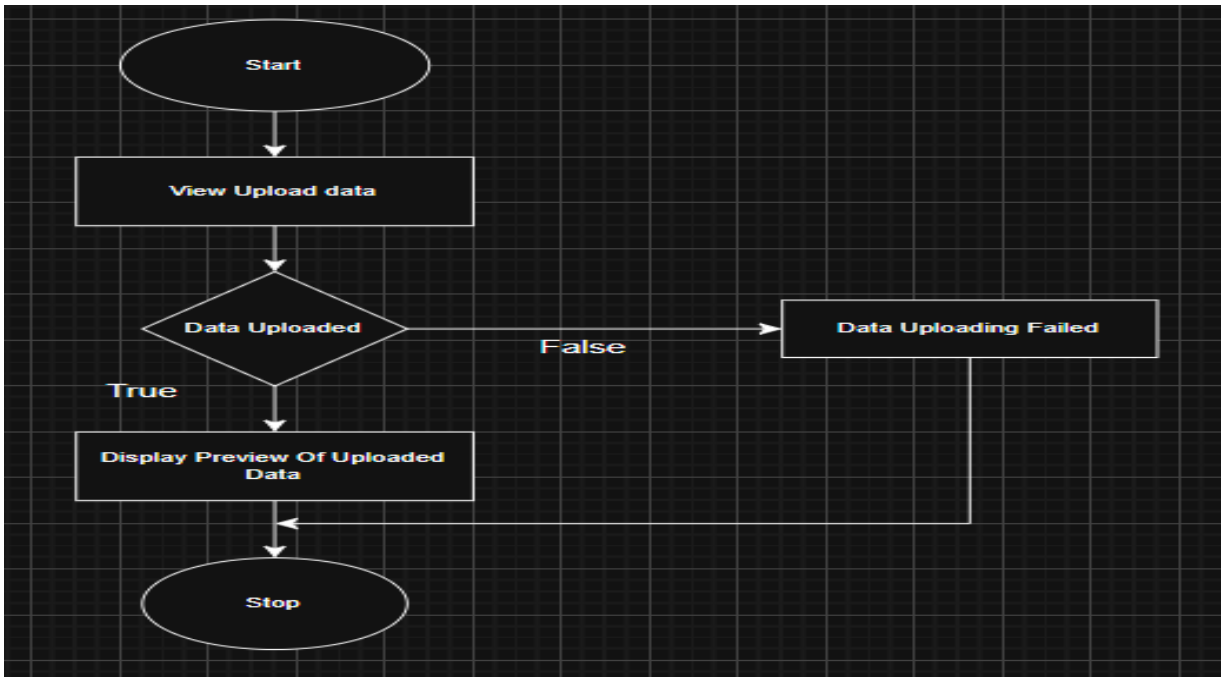
Input:Email,Pass

Procedural details:



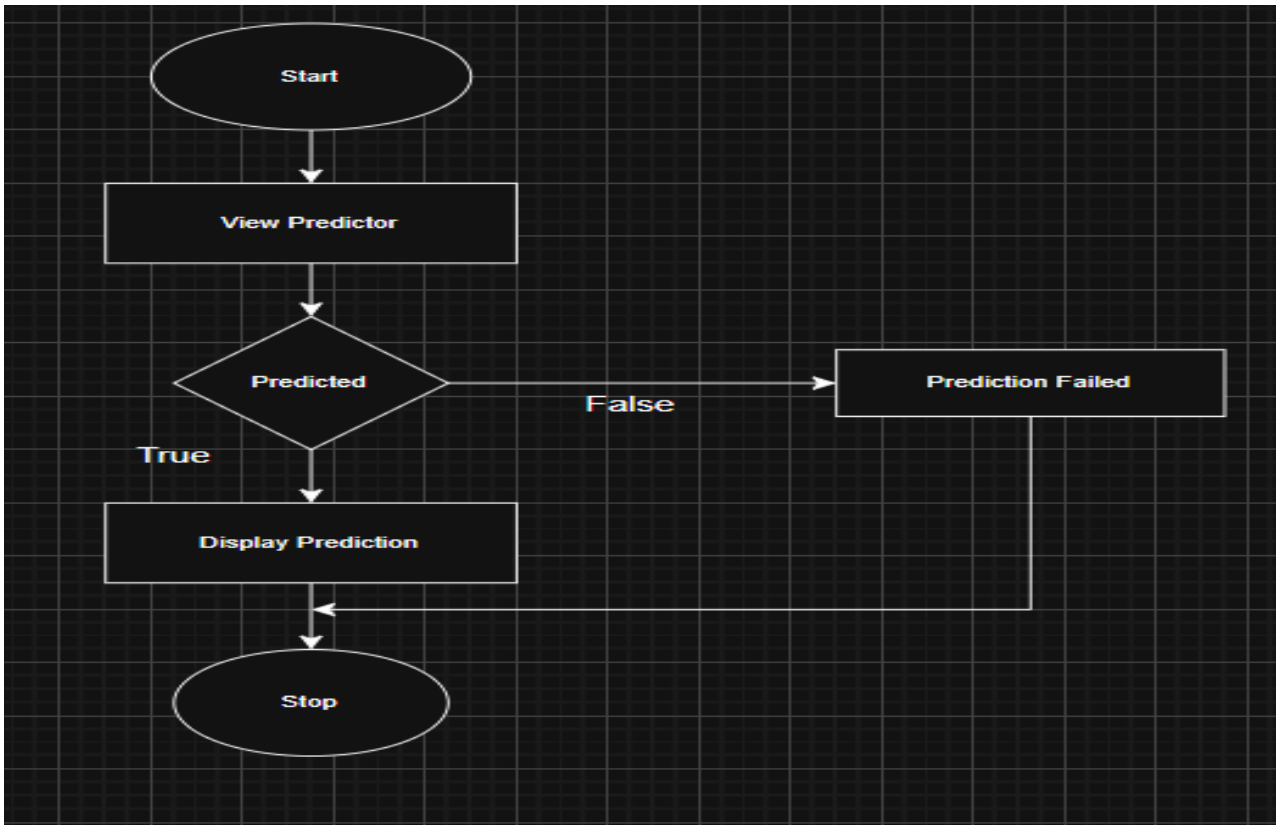
5.4.4.2 Upload Data:

Input:Email,Pass
Procedural details:



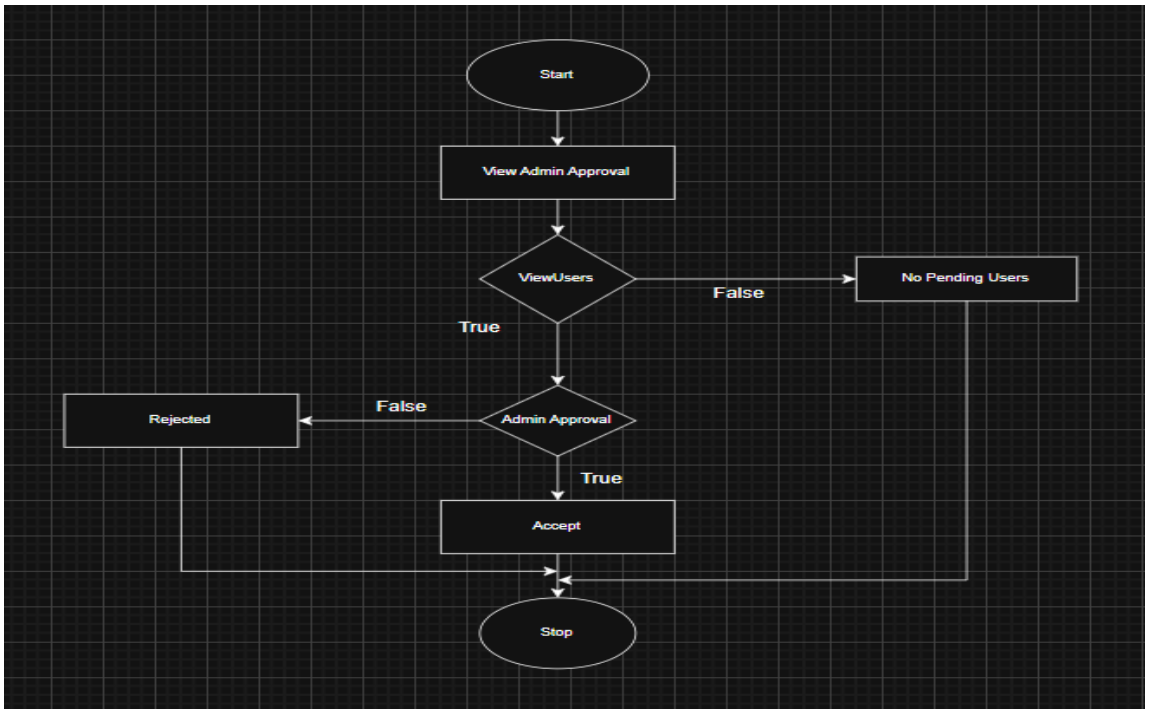
5.4.4.2 Predictor:

Input:Email,Pass
Procedural details:



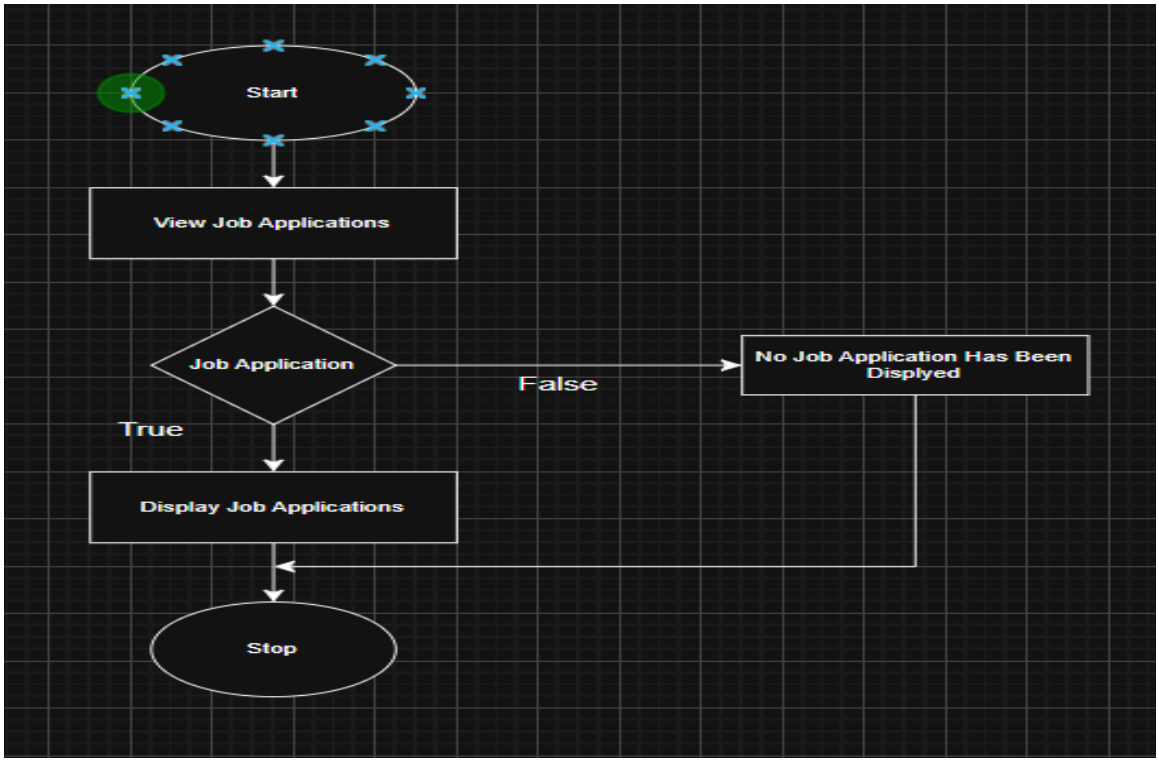
5.4.4Admin-Approval

Input:Email,Pass
Procedural details:



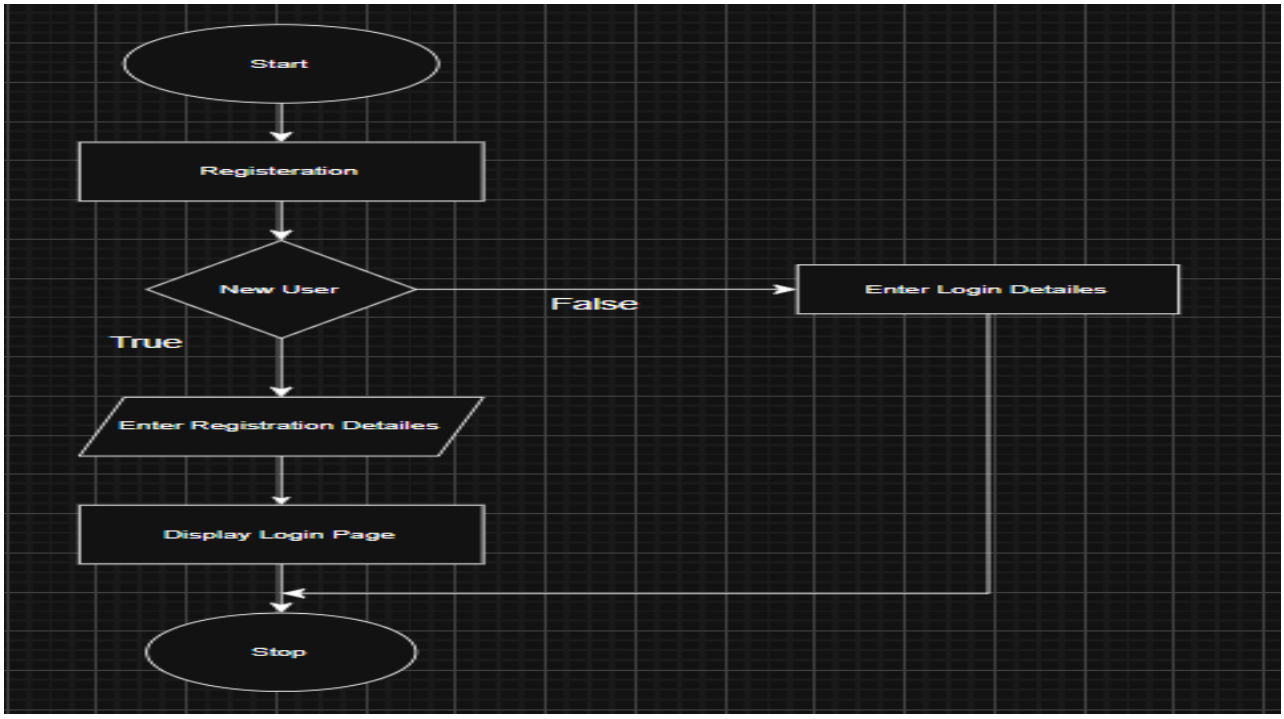
5.4.5.4 Job Application:

Input:Email,Pass
Procedural details:

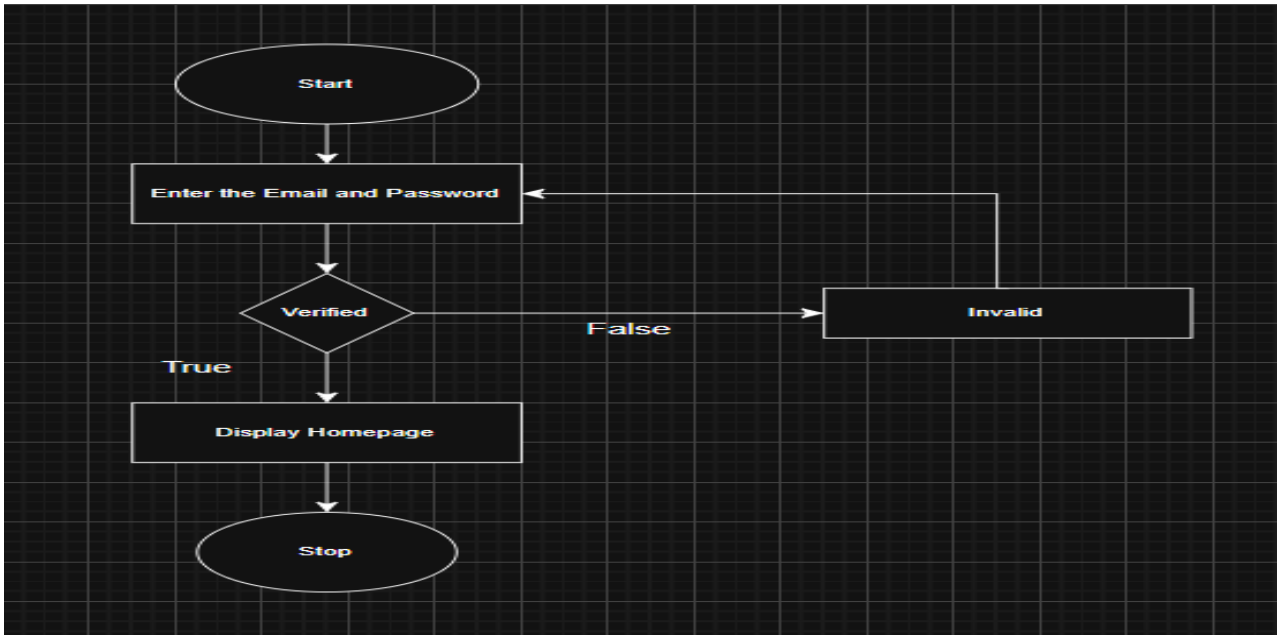


5.4 Procedural details of HR Manager:

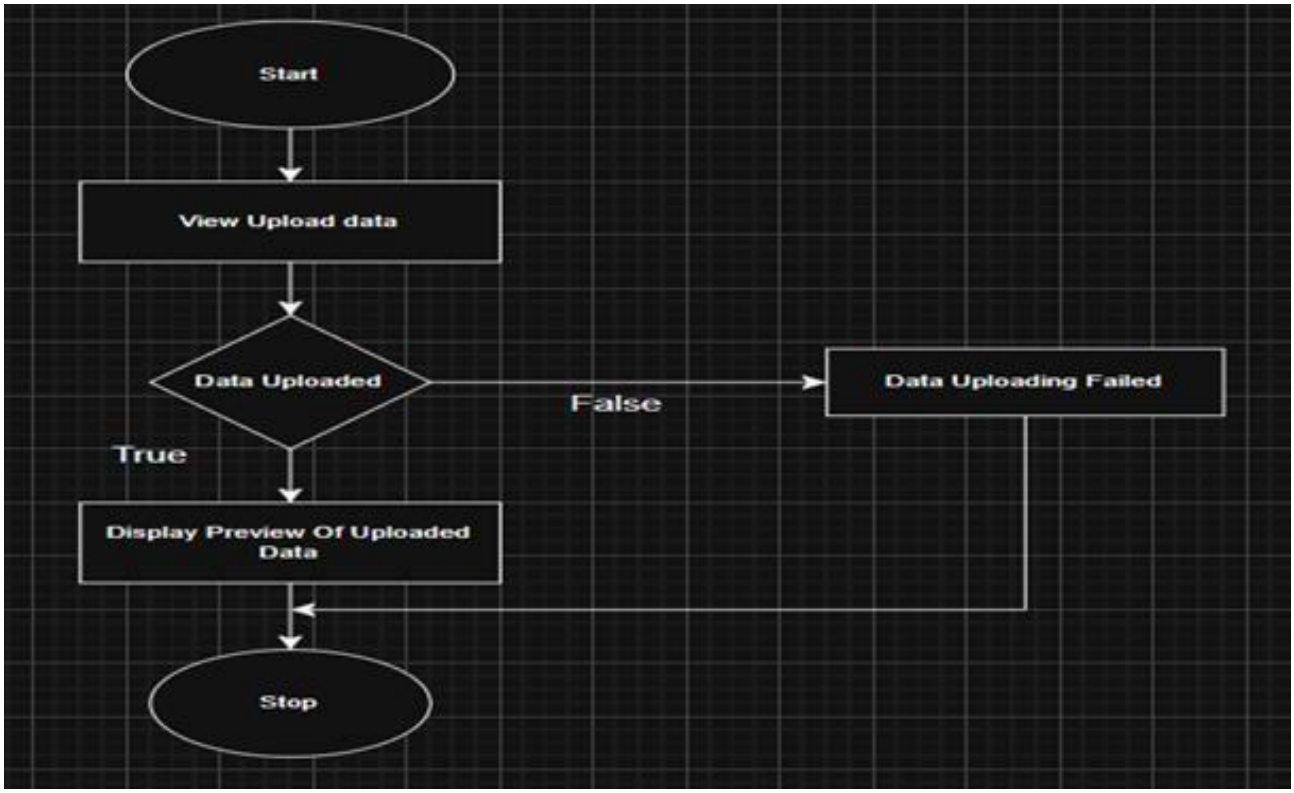
5.5 Registration:



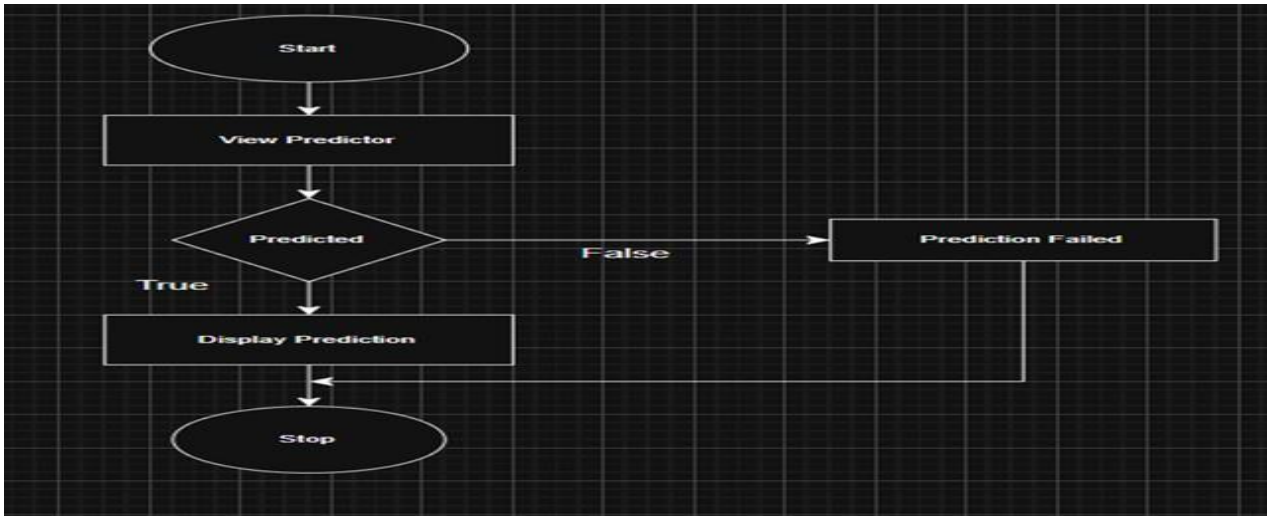
5.5.2 Login:



5.5.2 Upload Data

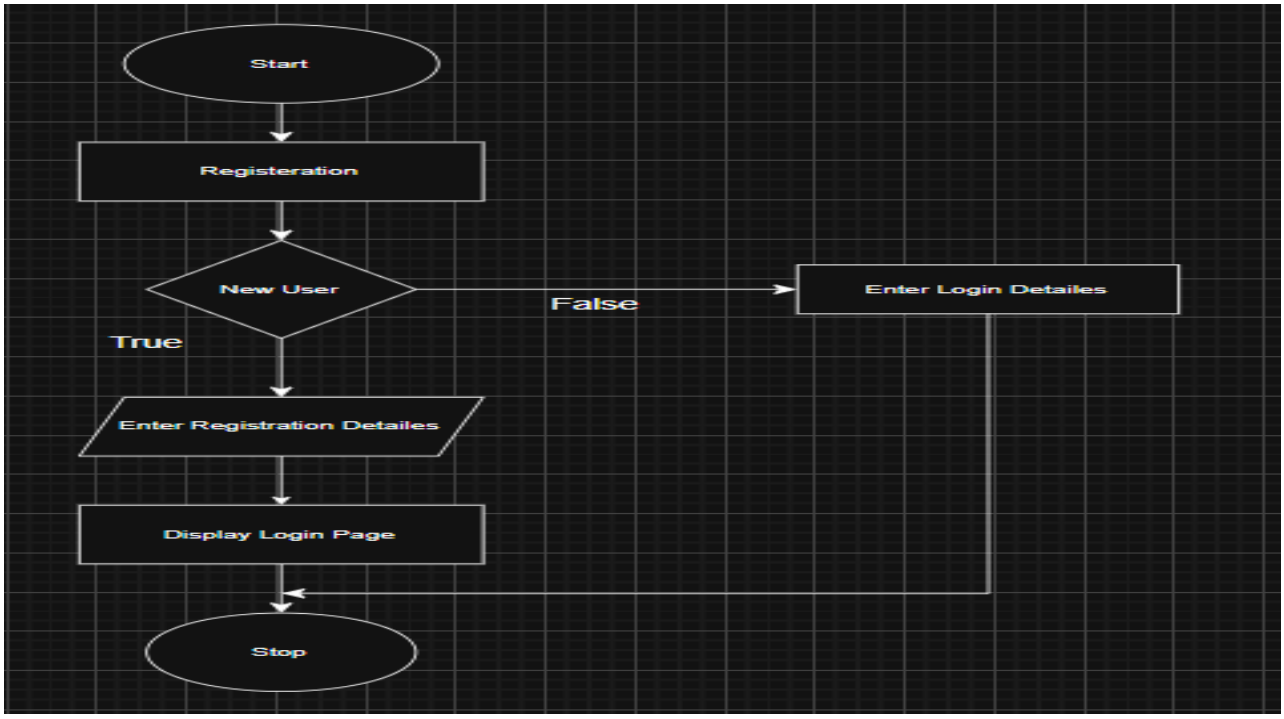


5.5.2 Predictor

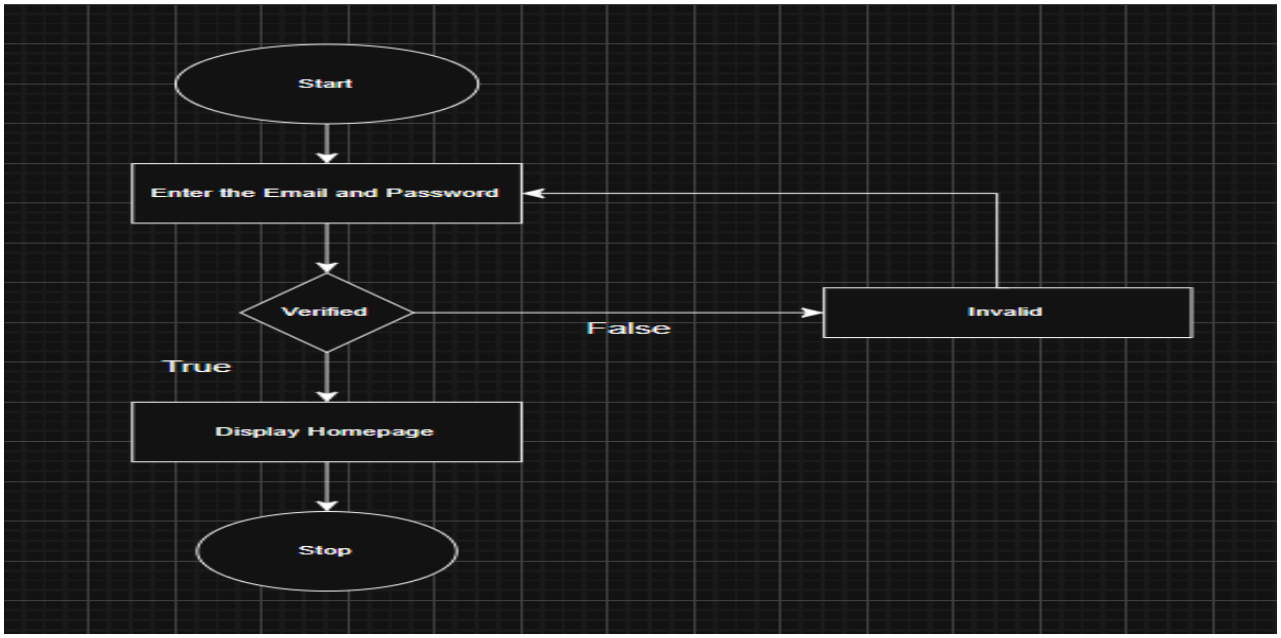


5.3 Procedural Detailes Of Data Analyst:

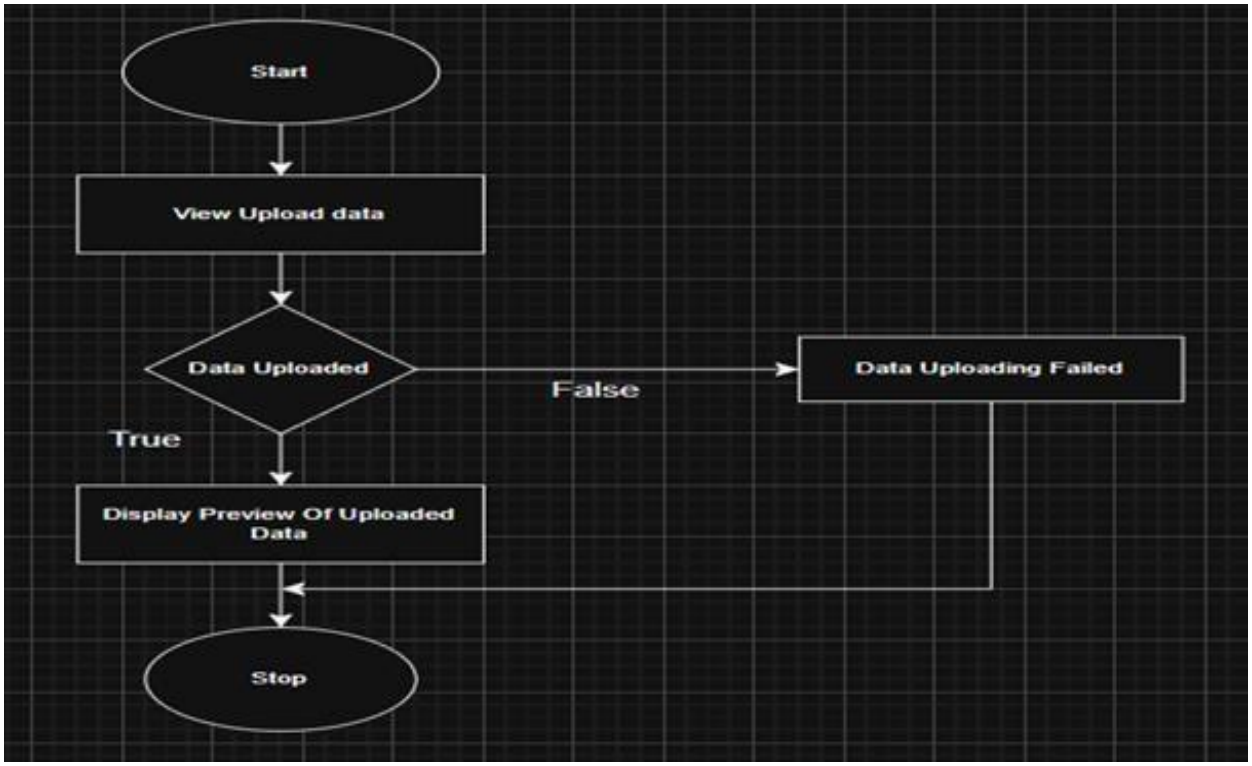
5.4 Registration:



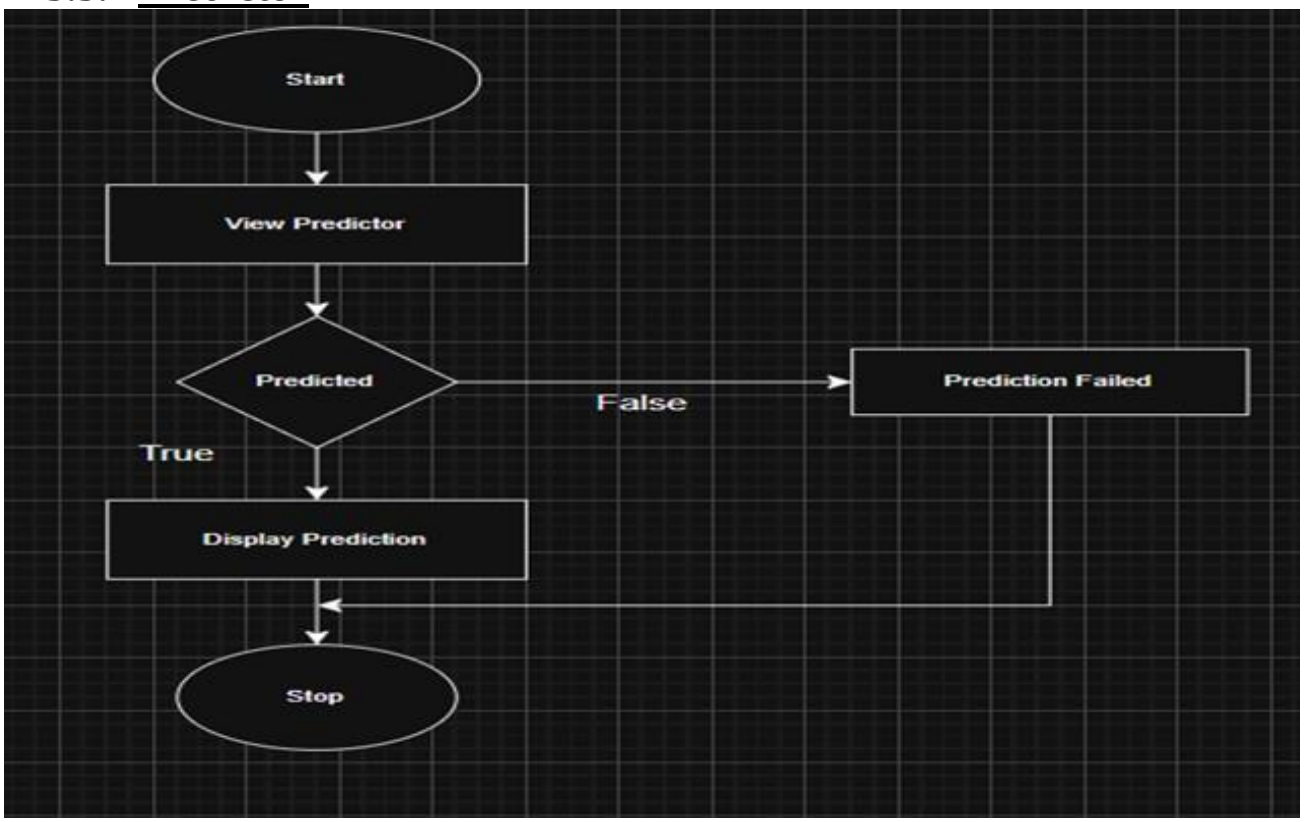
Login:



5.5.2 Upload Data

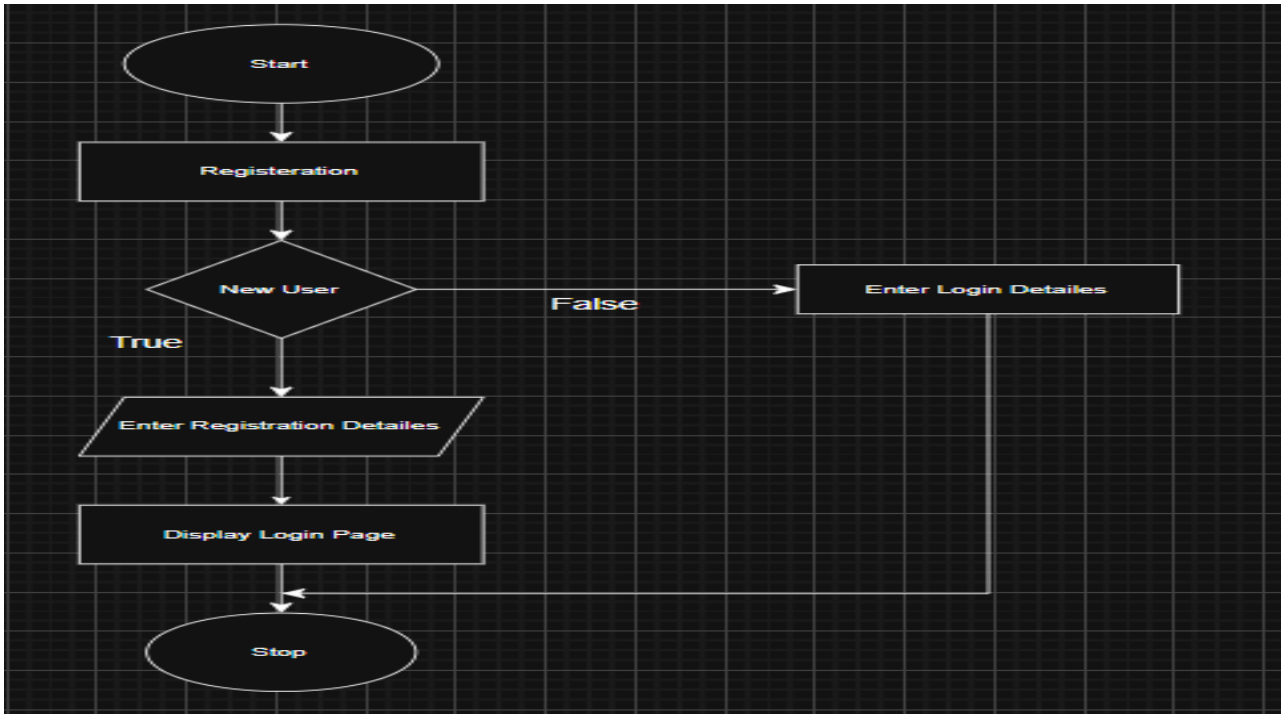


5.5.2 Predictor

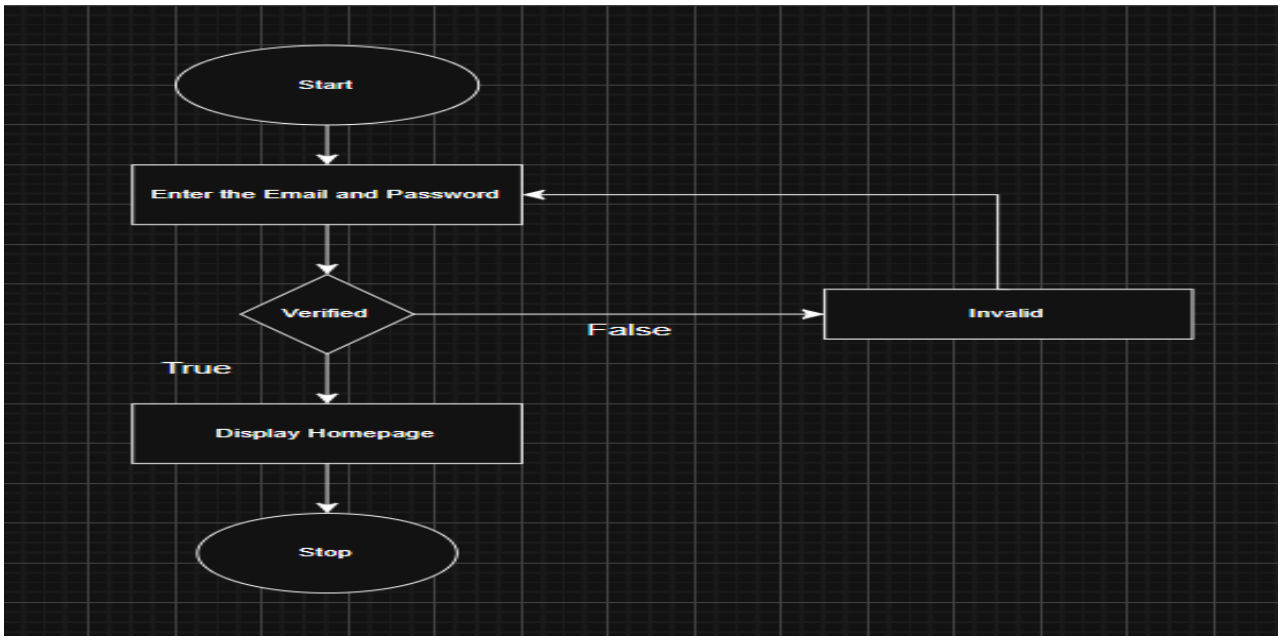


Procedural Detailes Of Employee:

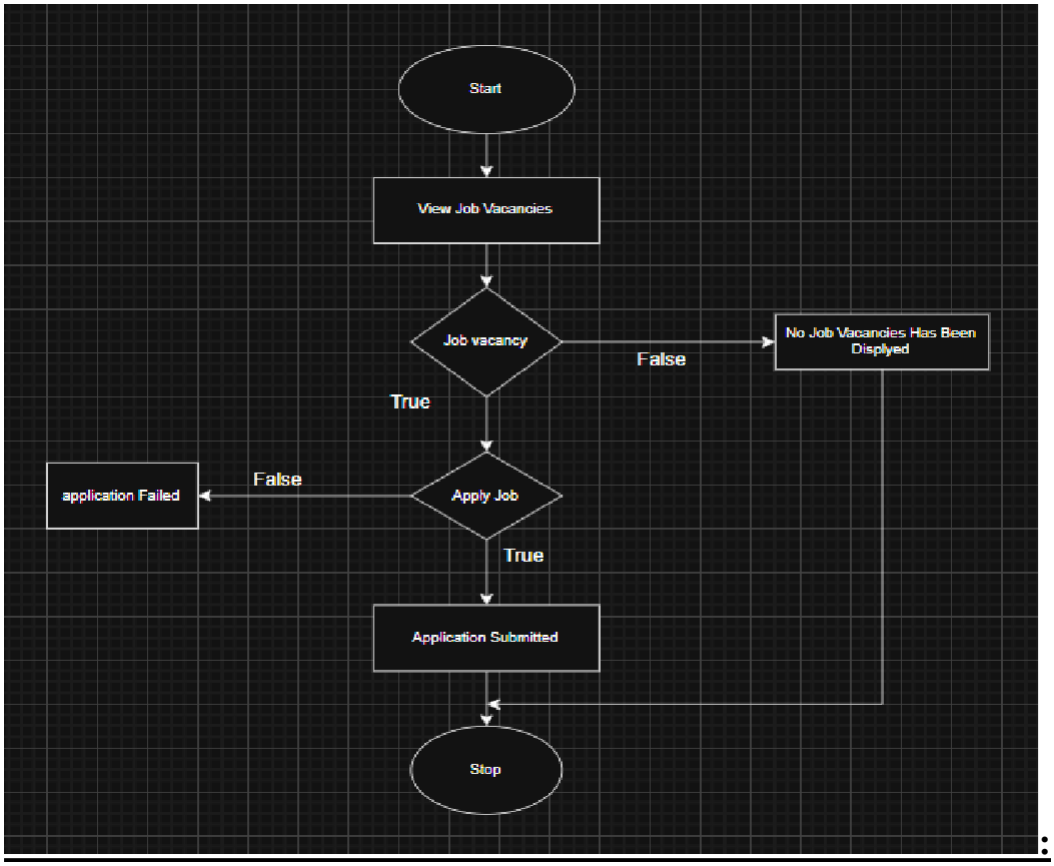
5.3 Registration:



Login:

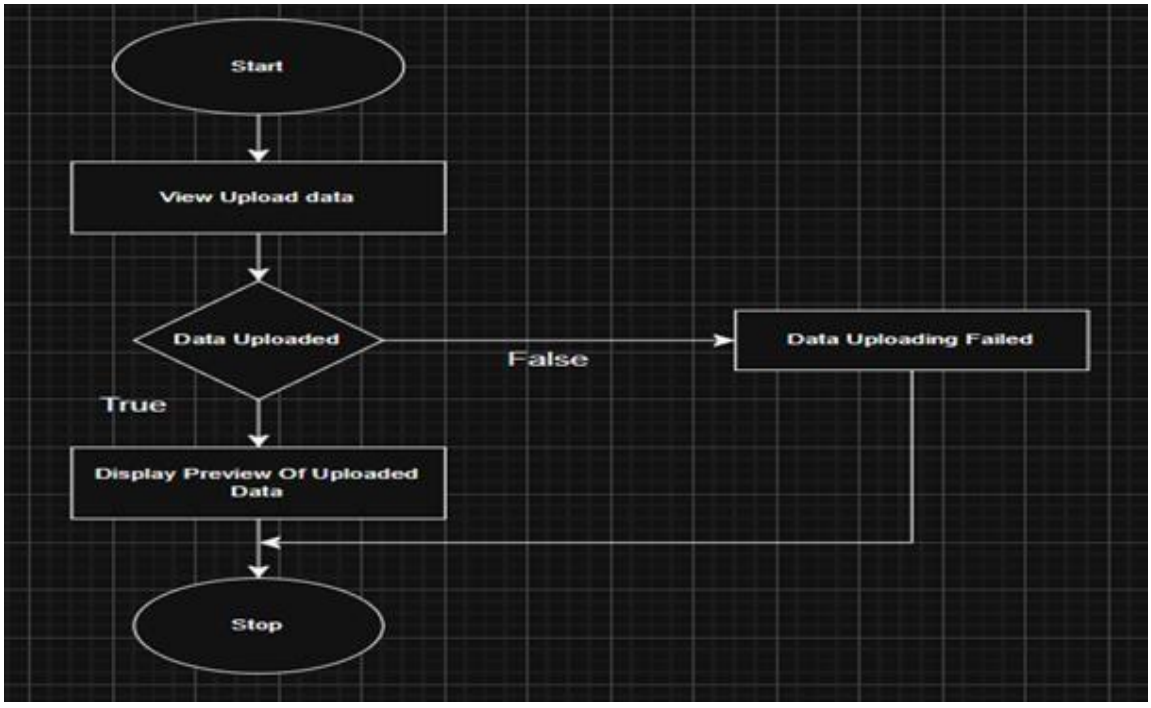


Job Vacancies:

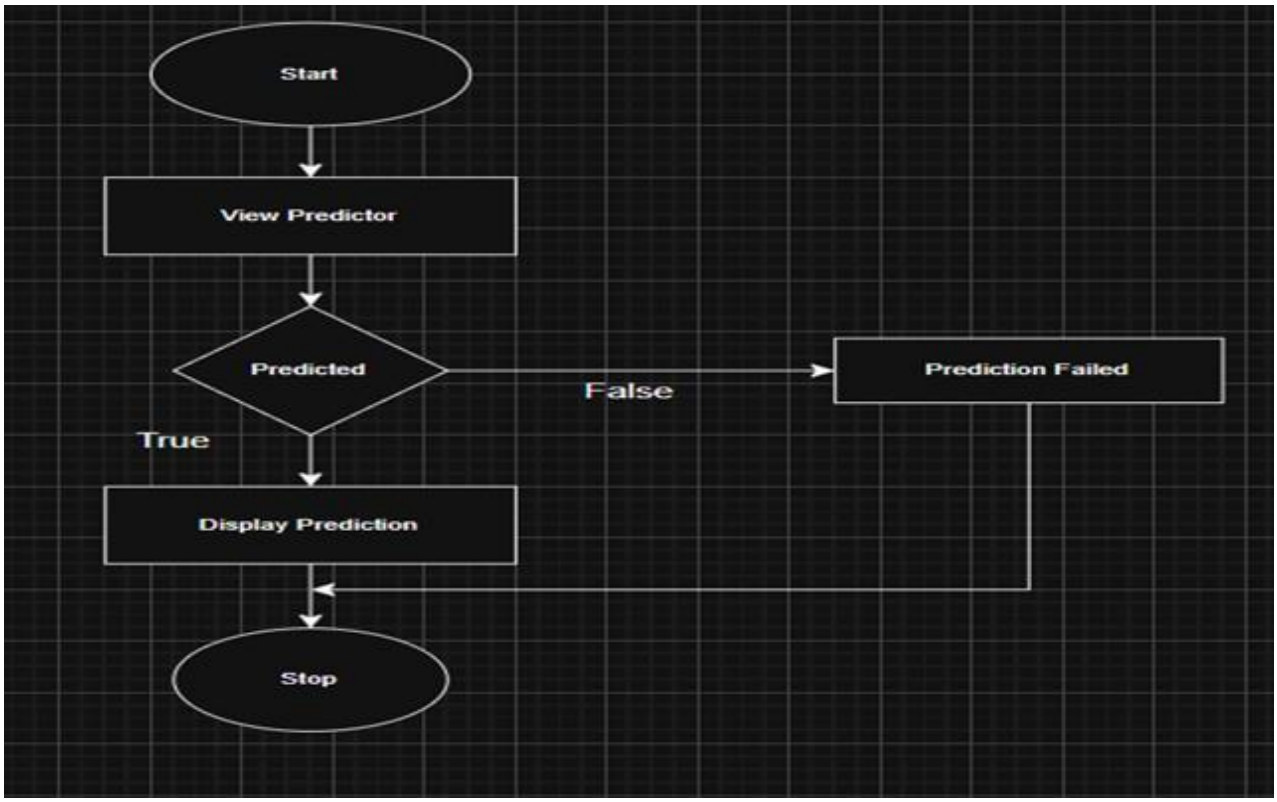


:

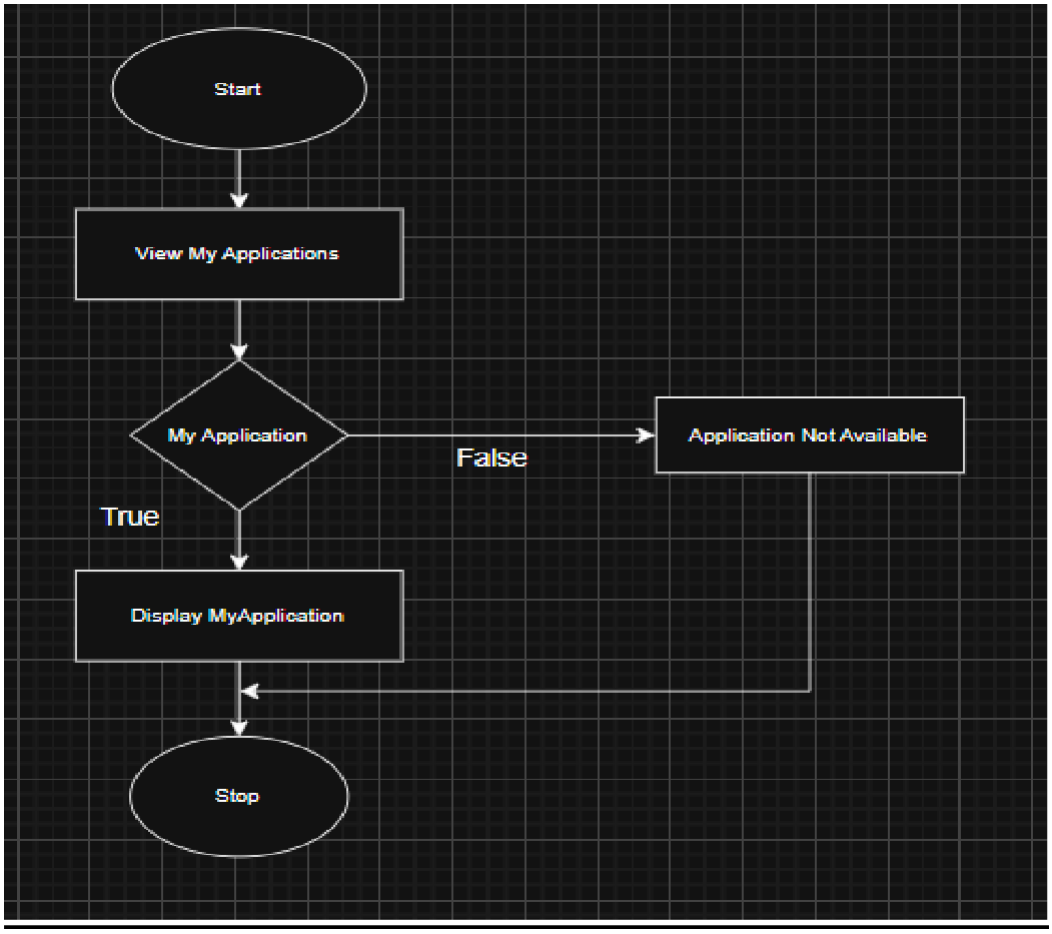
5.5.2 Upload Data



5.5.2 Predictor

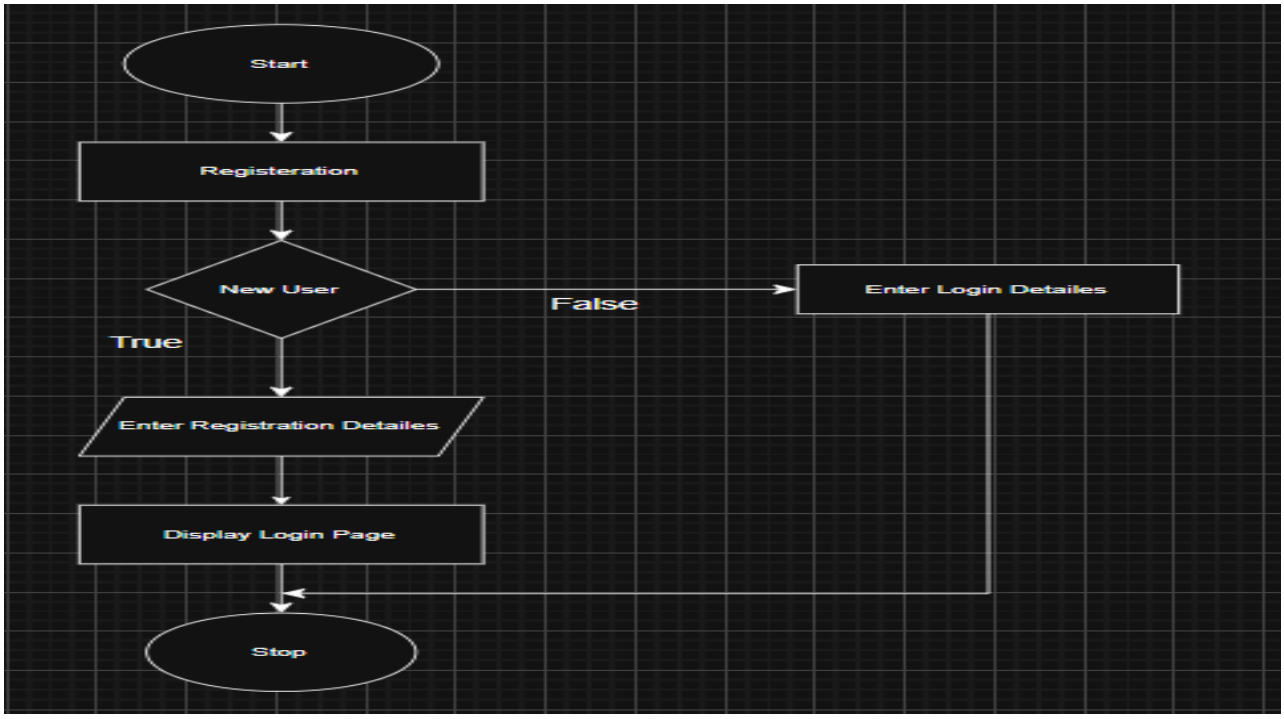


My Applications:

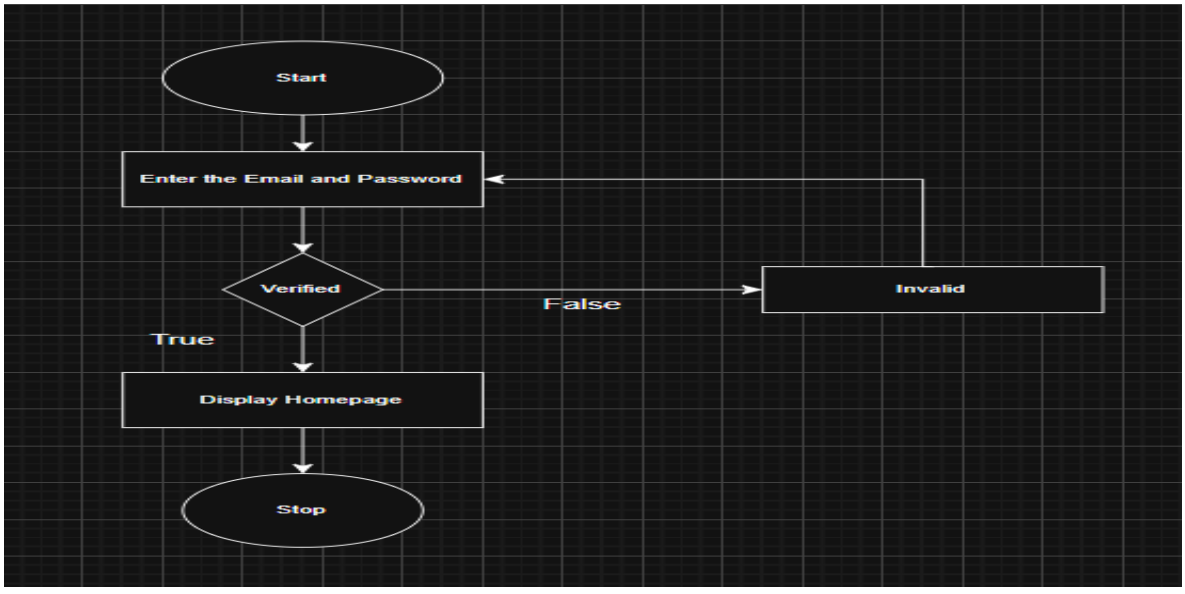


Procedural Detailes Of Users:

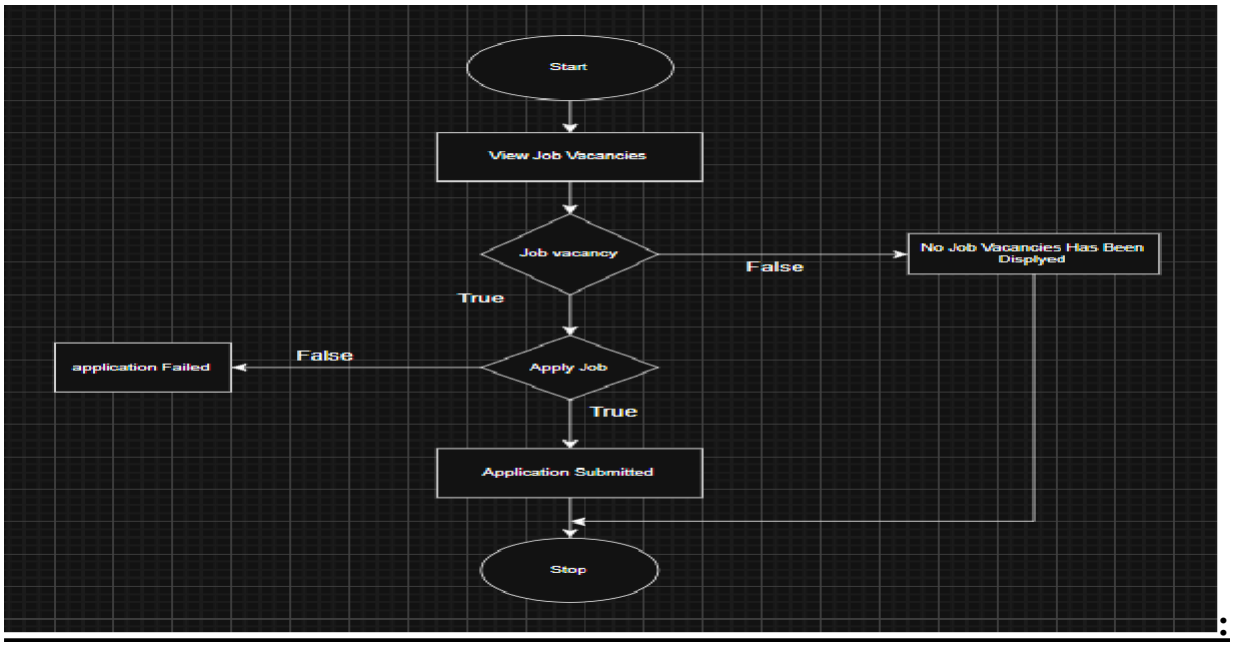
5.4 Registration:



Login:

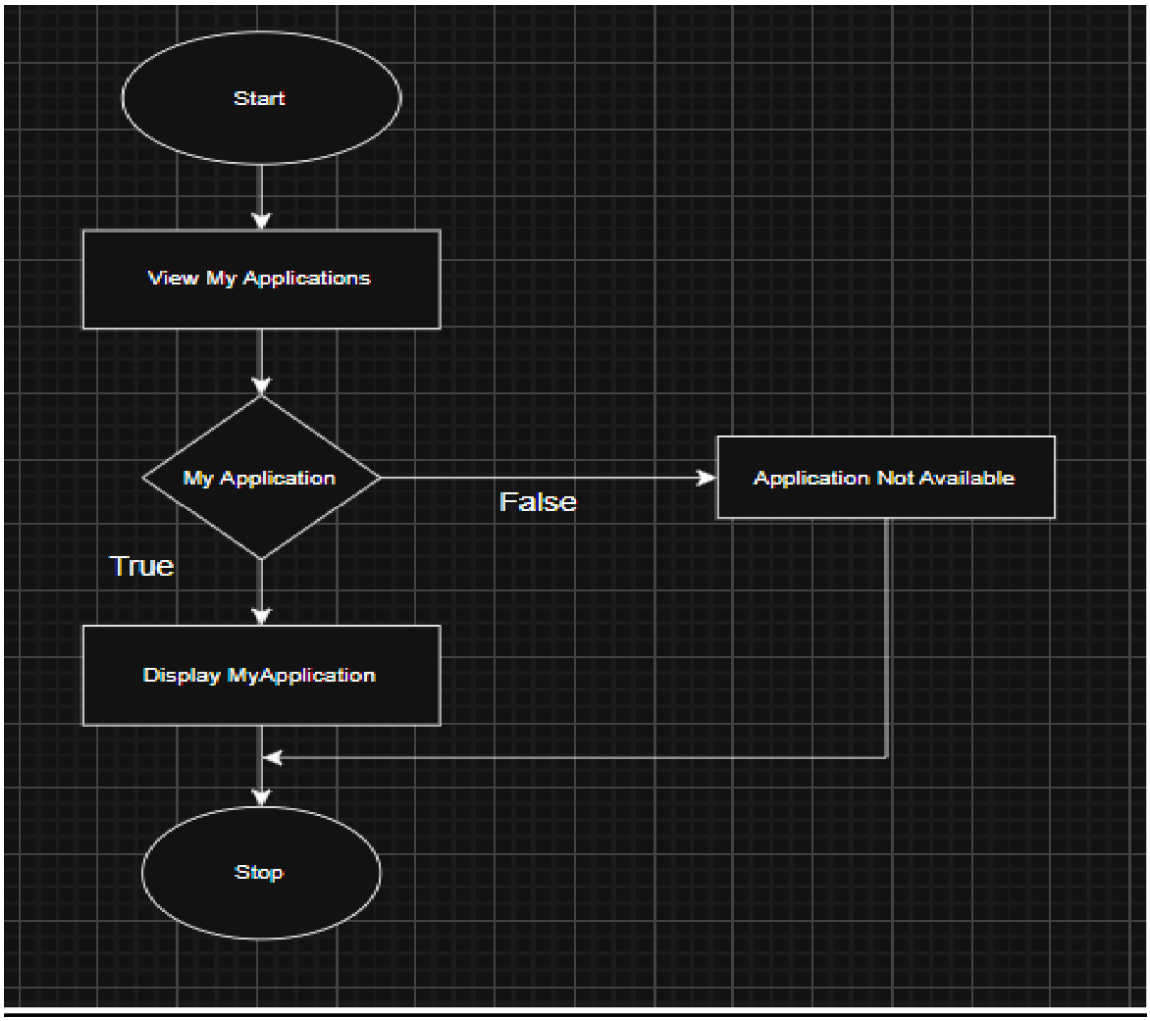


Job Vacancies:



:

My Applications:



6. CODING

Login .js:

```
import React, { useState, useContext } from "react";
import { Link, useNavigate } from "react-router-dom";
import AuthContext from "../context/AuthContext";
import "./Login.css";


 const Login = () => {
 const [formData, setFormData] = useState({ email: "", password: "" });
 const [error, setError] = useState("");
 const [loading, setLoading] = useState(false);
 const authContext = useContext(AuthContext);
 const navigate = useNavigate();

 if (!authContext) {
  console.error("❌AuthContext is missing! Ensure AuthProvider wraps your app.");
  return <p>□ Authentication system is unavailable.</p>;
 }

 const { login } = authContext; // Ensure login function exists

 const handleChange = (e) => {
  setFormData({ ...formData, [e.target.name]: e.target.value.trim() });


 };
```

```
const handleLogin = async (e) => {
  e.preventDefault();
  setError("");

   setLoading(true);

  try {
    const response = await fetch("http://127.0.0.1:5000/auth/login", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify(formData),
    });

    if (!response.ok) {
      throw new Error("❌Invalid email or password!");
    }

    const data = await response.json();
    console.log("□ Login Response:", data);

    const token = data.access_token; // ✅Fix: Use `access_token`
    const role = data.role;



    if (!token || !role) {
      throw new Error("❌Server response missing token or role.");
    }
```

```
    login(token, role);
    navigate("/dashboard");
   } catch (err) {
    console.error("□ Login Error:", err.message);
    setError(err.message || "✕Server error. Please try again.");
   } finally {

    setLoading(false);

   }
 };

 return (
  <div className="login-container">
   <div className="login-box">
    <h2>Login</h2>
    {error && <p className="error-message">{error}</p>}

    <form onSubmit={handleLogin}>
     <input
      type="email"
      name="email"
      placeholder="Enter your email"
      value={formData.email}
      onChange={handleChange}
      required
      autoComplete="email"
```

```
            aria-label="Email"
          />
          <input
            type="password"
            name="password"
            placeholder="Enter your password"
            value={formData.password}
            onChange={handleChange}
            required
            autoComplete="current-password"

            aria-label="Password"
          />

          <button type="submit" disabled={loading}>
            {loading ? "□ Logging in..." : "Login"}
          </button>
        </form>

        <p>Don't have an account? <Link to="/register">Register</Link></p>

      </div>
    </div>
  );
};
```

App.js

```
import React, { useContext, useState, useEffect } from "react";
import { Routes, Route, useLocation, Navigate } from "react-router-dom";
import AuthContext from "./context/AuthContext";
import ProtectedRoutes from "./routes/ProtectedRoutes";
import Login from "./pages/Login";
import Register from "./pages/Register";
import Dashboard from "./pages/Dashboard";
import Predict from "./pages/Predict";
import DataUpload from "./pages/DataUpload";
import AdminApproval from "./pages/AdminApproval";
import JobVacancies from "./pages/JobVacancies";
import ApplyJob from "./pages/ApplyJob";
import UserDashboard from "./pages/UserDashboard";
import MyApplications from "./pages/MyApplications";
import AdminApplications from "./pages/AdminApplications";
import Navbar from "./components/Navbar";
import Sidebar from "./components/Sidebar";

import "./App.css";

function App() {
  const { isAuthenticated, user } = useContext(AuthContext) || {};
  const location = useLocation();

  const [darkMode, setDarkMode] = useState(() =>
    JSON.parse(localStorage.getItem("darkMode") || "false")
  );

  useEffect(() => {
    document.body.classList.toggle("dark-mode", darkMode);


    localStorage.setItem("darkMode", JSON.stringify(darkMode));
  }, [darkMode]);

  const toggleDarkMode = () => setDarkMode((prev) => !prev);

  const authPages = ["/", "/login", "/register"];
  const showLayout = isAuthenticated && !authPages.includes(location.pathname);

  const getDefaultRedirect = () => {
    if (!isAuthenticated) return "/login";
    if (user?.role === "users") return "/user-dashboard";
```

```
  return "/dashboard";
};

return (
  <div className={`app-container ${darkMode ? "dark-mode" : ""}`}>
    {showLayout && <Sidebar />}

    <div className={`main-content ${showLayout ? "shifted" : ""}`}>
      {showLayout && (
        <Navbar
          darkMode={darkMode}
          toggleDarkMode={toggleDarkMode}
        />
      )}

      <div className="page-container">
        <Routes>
          {/* Public Routes */}
          <Route path="/" element={<Navigate to={getDefaultRedirect()} />} />
          <Route path="/login" element={<Login />} />
          <Route path="/register" element={<Register />} />

          {/* Protected Routes (All Authenticated Users) */}
          <Route element={<ProtectedRoutes />}>
            <Route path="/dashboard" element={<Dashboard />} />
            <Route path="/predict" element={<Predict />} />
            <Route path="/upload" element={<DataUpload />} />
          </Route>

          {/* Admin Only Routes */}
          <Route element={<ProtectedRoutes requiredRoles={["admin"]} />}>
            <Route path="/admin-approval" element={<AdminApproval />} />
            <Route path="/admin-applications" element={<AdminApplications />} />
          </Route>

          {/* Employee & Users */}
          <Route element={<ProtectedRoutes requiredRoles={["employee", "users"]} />}>
            <Route path="/job-vacancies" element={<JobVacancies />} />
            <Route path="/apply/:jobRole" element={<ApplyJob />} />
          </Route>

          {/* Users Only */}
          <Route element={<ProtectedRoutes requiredRoles={["users"]} />}>
            <Route path="/user-dashboard" element={<UserDashboard />} />
            <Route path="/my-applications" element={<MyApplications />} />
          </Route>
```

```
          {/* Fallback */}
          <Route path="*" element={<Navigate to={getDefaultRedirect()} />} />
        </Routes>
      </div>
    </div>
  </div>
 );
}

export default App;
```

Register.js

```
import React, { useState } from "react";
import { Link, useNavigate } from "react-router-dom";
import "./Register.css";

const Register = () => {
 const navigate = useNavigate();

 const [formData, setFormData] = useState({
  username: "",
  email: "",
  password: "",
  confirmPassword: "",
  role: "Users",
 });

 const [error, setError] = useState("");
 const [success, setSuccess] = useState("");
 const [loading, setLoading] = useState(false);

 const handleChange = (e) => {
  setFormData((prev) => ({
   ...prev,
   [e.target.name]: e.target.value.trim(),
  }));
 };

 const handleRegister = async (e) => {
  e.preventDefault();
  setError("");
  setSuccess("");
  setLoading(true);
```

```
const { username, email, password, confirmPassword, role } = formData;


if (!username || !email || !password || !confirmPassword) {
  setError("❌All fields are required.");
  setLoading(false);
  return;
}

if (password !== confirmPassword) {
  setError("❌Passwords do not match!");
  setLoading(false);
  return;
}

try {
  const response = await fetch("http://127.0.0.1:5000/auth/register", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({
      username,
      email,
      password,
      role,
    }),
  });

  const data = await response.json();
  console.log("□ Register Response:", data);

  if (!response.ok) {
    throw new Error(data?.error || "❌Registration failed!");
  }

  setSuccess("✅Registration successful! Redirecting to login...");

  setTimeout(() => {
    navigate("/login");
  }, 2000);

} catch (err) {
  console.error("❌Registration Error:", err.message);
  setError(err.message || "❌Server error. Please try again.");
} finally {
```

```
      setLoading(false);


    }
  };

  return (
    <div className="register-container">
      <div className="register-box">
        <div className="register-left">
          <img src="https://cdn-icons-png.flaticon.com/512/3135/3135715.png"
alt="Register" />
          <h2>Join Us Today!</h2>
          <p>Register now to unlock the Employee Attrition Predictor.</p>

        </div>
        <div className="register-right">
          <h2>Register</h2>
          {error && <p className="error-message">{error}</p>}
          {success && <p className="success-message">{success}</p>}

          <form onSubmit={handleRegister}>
            <input
              type="text"
              name="username"
              placeholder="Username"
              value={formData.username}
              onChange={handleChange}
              required
            />
            <input
              type="email"
              name="email"
              placeholder="Email"
              value={formData.email}
              onChange={handleChange}
              required
            />
            <input
              type="password"
              name="password"
              placeholder="Password"
              value={formData.password}
              onChange={handleChange}
              required
```

```
          />
          <input
            type="password"
            name="confirmPassword"
            placeholder="Confirm Password"
            value={formData.confirmPassword}
            onChange={handleChange}
            required
          />

          <select name="role" value={formData.role} onChange={handleChange}>
            <option value="Users">Users</option>
            <option value="HR Manager">HR Manager</option>
            <option value="Data Analyst">Data Analyst</option>
            <option value="Employee">Employee</option>
          </select>

          <button type="submit" disabled={loading}>
            {loading ? "□ Registering..." : "Register"}
          </button>
        </form>

 <p>Already have an account? <Link to="/login">Login</Link></p>


      </div>
    </div>
  </div>
 );
};

export default Register;
```

Predict.js

```
import React, { useState } from "react";
import "./Predict.css";

const Predict = () => {
   const [formData, setFormData] = useState({
      Age: 25,
      BusinessTravel: "Non-Travel",
      DailyRate: 500,
      DistanceFromHome: 5,


      Education: 3,
      EducationField: "Human Resources",
      EnvironmentSatisfaction: 3,
      Gender: "Male",
      HourlyRate: 50,
      JobInvolvement: 3,
      JobLevel: 2,
      JobRole: "Sales Executive",
      JobSatisfaction: 3,
      MaritalStatus: "Single",
      MonthlyIncome: 5000,
      MonthlyRate: 20000,
      NumCompaniesWorked: 2,
      OverTime: "No",
      PercentSalaryHike: 10,
      PerformanceRating: 3,
      RelationshipSatisfaction: 3,
      StockOptionLevel: 1,
      TotalWorkingYears: 5,
      TrainingTimesLastYear: 2,
      WorkLifeBalance: 3,
      YearsAtCompany: 3,
      YearsInCurrentRole: 2,
      YearsSinceLastPromotion: 1,
      YearsWithCurrManager: 2,

   });

   const [result, setResult] = useState(null);
   const [loading, setLoading] = useState(false);
   const [error, setError] = useState(null);
   const handleChange = (e) => {
```

```
    const { name, value, type } = e.target;
    setFormData((prevData) => ({

  ...prevData,
        [name]: type === "number" ? (value ? parseFloat(value) : 0) : value,
    }));
};

const getRiskLevel = (percentage) => {
    if (percentage >= 70) return "High";
    if (percentage >= 40) return "Medium";
    return "Low";
};


const handleSubmit = async (e) => {
    e.preventDefault();
    setLoading(true);
    setError(null);
    setResult(null);

    try {
        const response = await fetch("http://127.0.0.1:5000/predict", {
            method: "POST",
            headers: { "Content-Type": "application/json" },
            body: JSON.stringify([formData]),
        });

        const data = await response.json();
        console.log("API Response:", data);

        if (response.ok && data.results && data.results.length > 0) {
            let rawPercentage = parseFloat(data.results[0].risk_percentage);
            if (isNaN(rawPercentage)) rawPercentage = 0;

            const scaledPercentage = rawPercentage * 100;
            const roundedPercentage = scaledPercentage.toFixed(1);

            setResult({
                prediction: data.results[0].prediction,
                riskPercentage: `${roundedPercentage}%`,
                riskLevel: getRiskLevel(scaledPercentage),
            });
```

```
        } else {
            setError("Unexpected response from the server.");
        }


    } catch (err) {
        console.error(err);


        setError("Failed to connect to the backend.");
    }

    setLoading(false);
  };


  return (
    <div className="predict-container">
      <h2>Employee Attrition Predictor</h2>
      <form onSubmit={handleSubmit} className="predict-form">
        {Object.keys(formData).map((key) => (
          <div className="form-group" key={key}>
            <label>{key.replace(/([A-Z])/g, " $1").trim()}:</label>
            {["Gender", "BusinessTravel", "EducationField", "JobRole",
"MaritalStatus", "OverTime"].includes(key) ? (
              <select name={key} onChange={handleChange}
value={formData[key]}>
                {key === "Gender" && (
                  <>
                    <option value="Male">Male</option>
                    <option value="Female">Female</option>
                  </>
                )}
                {key === "BusinessTravel" && (
                  <>
                    <option value="Non-Travel">Non-Travel</option>
                    <option value="Travel_Rarely">Travel Rarely</option>
                    <option value="Travel_Frequently">Travel
Frequently</option>
                  </>
                )}
                {key === "EducationField" && (
                  <>
                    <option value="Human Resources">Human
Resources</option>
```

```
                        <option value="Life Sciences">Life Sciences</option>
                        <option value="Marketing">Marketing</option>
                        <option value="Medical">Medical</option>
                        <option value="Technical Degree">Technical Degree</option>
                        <option value="Other">Other</option>
                    </>
                )}
                {key === "JobRole" && (
                    <>
                        <option value="Sales Executive">Sales Executive</option>
                        <option value="Research Scientist">Research
Scientist</option>
                        <option value="Laboratory Technician">Laboratory
Technician</option>



                        <option value="Manufacturing Director">Manufacturing
Director</option>
                        <option value="Healthcare Representative">Healthcare
Representative</option>
                        <option value="Manager">Manager</option>
                        <option value="Sales Representative">Sales
Representative</option>


                        <option value="Research Director">Research Director</option>
                        <option value="Human Resources">Human
Resources</option>
                    </>
                )}


                {key === "MaritalStatus" && (
                    <>


                        <option value="Single">Single</option>
                        <option value="Married">Married</option>
                        <option value="Divorced">Divorced</option>
                    </>
                )}
                {key === "OverTime" && (
                    <>
```

```
                                <option value="No">No</option>
                                <option value="Yes">Yes</option>
                            </>
                        )}
                    </select>
                ) : (
                    <input
                        type="number"
                        name={key}
                        value={formData[key]}
                        onChange={handleChange}
                        required
                    />
                )}
            </div>
        ))}



        <button type="submit" disabled={loading}>
            {loading ? <div className="spinner"></div> : "Predict"}
        </button>
    </form>

    {loading && <div className="loading">Predicting...</div>}

    {error && <p className="error">{error}</p>}

    {result && (
        <div className={`result ${result.riskLevel.toLowerCase()}`}>
            <p>Prediction: <strong>{result.prediction}</strong></p>
            <p>Risk Percentage: <strong>{result.riskPercentage}</strong></p>
            <p>Risk Level: <strong>{result.riskLevel}</strong></p>
        </div>
    )}
</div>
);
};

export default Predict;
```

DataUpload.js:

```
import React, { useState, useEffect, useRef } from "react";

const DataUpload = () => {
  const [file, setFile] = useState(null);
  const [message, setMessage] = useState("");
  const [loading, setLoading] = useState(false);
  const [previewData, setPreviewData] = useState([]);
  const fileInputRef = useRef(null);

  const allowedExtensions = ["csv", "xls", "xlsx"];

  const handleFileChange = (e) => {
    const selectedFile = e.target.files[0];
    if (!selectedFile) {
      setMessage("✕No file selected.");
      return;
    }

    const fileExtension = selectedFile.name.split(".").pop().toLowerCase();
    if (!allowedExtensions.includes(fileExtension)) {
      setMessage("✕Invalid file type. Only CSV, XLS, and XLSX are allowed.");
      setFile(null);
      if (fileInputRef.current) fileInputRef.current.value = "";
      return;
    }

    setFile(selectedFile);
    setMessage("");
  };

  const handleUpload = async () => {
    if (!file) {
      setMessage("✕Please select a file to upload.");
      return;
    }

    setLoading(true);
    setMessage("");
    setPreviewData([]);

    const formData = new FormData();
    formData.append("file", file);
```

```
 const token = localStorage.getItem("token");
 if (!token) {
   setMessage("❌Unauthorized! Please log in.");
   setLoading(false);

 return;
 }
 try {
   const response = await fetch("http://127.0.0.1:5000/upload", {
     method: "POST",
     body: formData,
     headers: {
       Authorization: `Bearer ${token}`,
     },
   });

   const data = await response.json();


   if (response.ok) {
     setMessage("✅File uploaded successfully!");
     setPreviewData(Array.isArray(data.preview) ? data.preview : []);
     setFile(null);
     if (fileInputRef.current) fileInputRef.current.value = "";
   } else {
     setMessage(`❌${data.error || "Upload failed."}`);
   }
 } catch (err) {
   console.error("❌Upload error:", err);
   setMessage("❌Server error. Please try again.");
 } finally {
   setLoading(false);
 }
};

useEffect(() => {
 if (message.includes("✅")) {
   const timer = setTimeout(() => setMessage(""), 5000);
   return () => clearTimeout(timer);
 }
}, [message]);

return (
 <div style={{ textAlign: "center", marginTop: "50px", padding: "20px" }}>
   <h2>Upload Employee Data</h2>
```

```
{message && (
  <p
    style={{
      color: message.startsWith("✅") ? "green" : "red",
      fontWeight: "bold",
    }}
  >
    {message}
  </p>
)}

<input
  ref={fileInputRef}

  type="file"
  onChange={handleFileChange}


  disabled={loading}
  accept=".csv, .xls, .xlsx"
  style={{
    padding: "10px",
    border: "1px solid #ccc",
    borderRadius: "5px",
    marginRight: "10px",
  }}
/>

<button
  onClick={handleUpload}
  disabled={loading || !file}
  style={{
    padding: "10px 15px",
    backgroundColor: loading ? "#ccc" : "#007bff",
    color: "#fff",
    border: "none",
    borderRadius: "5px",
    cursor: loading ? "not-allowed" : "pointer",
  }}
>
  {loading ? "Uploading..." : "Upload"}
</button>

{previewData.length > 0 && (
  <div style={{ marginTop: "30px" }}>
```

```
<h3>File Preview</h3>
<div
  style={{
    overflowX: "auto",
    maxWidth: "95%",
    margin: "auto",
    border: "1px solid #ddd",
    borderRadius: "5px",
    boxShadow: "0 2px 5px rgba(0,0,0,0.1)",
  }}
>
  <table
    border="1"
    style={{
      width: "100%",
      borderCollapse: "collapse",
      textAlign: "left",

    }}
  >

    <thead>
      <tr>

        {Object.keys(previewData[0]).map((col, idx) => (
          <th
            key={idx}
            style={{ padding: "10px", background: "#f8f8f8", fontWeight: "bold" }}
          >

            {col}
          </th>
        ))}
      </tr>
    </thead>
    <tbody>
      {previewData.map((row, rowIndex) => (
        <tr key={rowIndex}>
          {Object.values(row).map((val, colIndex) => (
            <td key={colIndex} style={{ padding: "8px" }}>
              {val}
            </td>
          ))}
        </tr>
      ))}
```

```
            </tbody>
          </table>
        </div>
      </div>
    )}
  </div>
 );
};

export default DataUpload;
```

Dashboard.js

```
import React, { useState, useEffect, useContext, useCallback } from "react";
import { useNavigate } from "react-router-dom";
import { jwtDecode } from "jwt-decode";
import { FaUsers, FaChartLine, FaUserShield, FaFileAlt } from "react-icons/fa";
import AuthContext from "../context/AuthContext";
import JobVacancies from "./JobVacancies";
import AttritionTrendChart from "../components/AttritionTrendChart";
import DepartmentAttritionChart from "../components/DepartmentAttritionChart";
import RetentionPieChart from "../components/RetentionPieChart";
import HRReport from "../components/HRReport";
import './Dashboard.css';

const Dashboard = () => {
  const navigate = useNavigate();
  const { userRole } = useContext(AuthContext);
  const [dashboardData, setDashboardData] = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState("");

  const role = userRole?.toLowerCase();
  useEffect(() => {

    if (role === "users") {
      navigate("/job-vacancies", { replace: true });
    }
  }, [role, navigate]);

  const isTokenValid = (token) => {
    if (!token) return false;
    try {
      const decoded = jwtDecode(token);
```

```
    return decoded?.exp * 1000 > Date.now();
  } catch (err) {
    console.error("□ Token decoding error:", err);
    return false;
  }
};

const fetchDashboardData = useCallback(async () => {
  const token = localStorage.getItem("token");



  if (!token || !isTokenValid(token)) {
    console.warn("✖Invalid or expired token. Redirecting to login...");
    localStorage.removeItem("token");
    localStorage.removeItem("role");
    navigate("/login");
    return;
  }

  try {
    setLoading(true);
    setError("");

    const response = await fetch("http://localhost:5000/dashboard", {
      headers: { Authorization: `Bearer ${token}` },
    });

    if (!response.ok) {
      throw new Error(`Server Error: ${await response.text()}`);
    }

    const data = await response.json();
    console.log("□ Dashboard Data Received:", data);

    if (!data || Object.keys(data).length === 0) {
      throw new Error("Empty response from server.");
    }

    setDashboardData(data);
  } catch (err) {
    console.error("✖Dashboard Fetch Error:", err.message);
    setError("Unable to load dashboard. Please try again.");
  } finally {
```

```
      setLoading(false);
    }

  }, [navigate]);

  useEffect(() => {
    if (role !== "users") {
      fetchDashboardData();
      const interval = setInterval(fetchDashboardData, 10000);
      return () => clearInterval(interval);
    }
  }, [fetchDashboardData, role]);



  if (loading) return <p>Loading dashboard...</p>;
  if (error) return <p className="error-message">{error}</p>;

  const jobs = dashboardData?.jobs || [];
  const stats = dashboardData?.stats || {};
  const reports = dashboardData?.reports || {};

  return (
    <div className="dashboard-container">
      <h1 className="dashboard-title">□ Employee Attrition Dashboard</h1>

      <h2 className="welcome-message">
        □ Welcome, {role?.charAt(0).toUpperCase() + role.slice(1)}!
      </h2>

      {role === "users" && jobs.length > 0 && <JobVacancies jobs={jobs} />}

      {role === "employee" && stats.total_employees && (
        <div className="dashboard-cards">
          <DashboardCard icon={<FaUsers />} title="Total Employees"
value={stats.total_employees} />
          <DashboardCard icon={<FaChartLine />} title="Attrition Rate"
value={`${stats.attrition_rate}%`} />
          <DashboardCard icon={<FaUserShield />} title="Retention Rate"
value={`${stats.retention_rate}%`} />
        </div>
      )}

      {role === "data analyst" && (
        <div className="charts-section">
```

```
        <AttritionTrendChart />
        <DepartmentAttritionChart />
        <RetentionPieChart />

      </div>
    )}

    {role === "hr manager" && reports && (
      <div className="hr-report-section">
        <h2><FaFileAlt /> HR Report</h2>
        <HRReport




          data={reports.department_attrition}
          attritionRate={reports.attrition_rate}
          totalEmployees={reports.total_employees}
        />
      </div>
    )}

    {role === "admin" && (
      <>
        <div className="dashboard-cards">
          <DashboardCard icon={<FaUsers />} title="Total Employees"
value={stats.total_employees} />
          <DashboardCard icon={<FaChartLine />} title="Attrition Rate"
value={`${stats.attrition_rate}%`} />
          <DashboardCard icon={<FaUserShield />} title="Retention Rate"
value={`${stats.retention_rate}%`} />
        </div>

        <div className="charts-section">
          <AttritionTrendChart />
          <DepartmentAttritionChart />
          <RetentionPieChart />
        </div>

        <div className="hr-report-section">
          <h2><FaFileAlt /> HR Report</h2>
          <HRReport
            data={reports.department_attrition}
            attritionRate={reports.attrition_rate}
            totalEmployees={reports.total_employees}
```

```
          />
        </div>
      </>
    )}

    {role !== "users" && (
      <div className="predict-section">
        <button className="predict-button" onClick={() => navigate("/predict")}>
          Predict Employee Attrition
        </button>
      </div>
)}
   </div>



 );

};

const DashboardCard = ({ icon, title, value }) => (
  <div className="dashboard-card">
    <div className="dashboard-icon">{icon}</div>
    <div className="dashboard-card-content">
      <h3>{title}</h3>
      <p>{value?.toLocaleString?.() || value}</p>
    </div>
  </div>
);

export default Dashboard;
```

AdminDashboard.js:

```
import React, { useEffect, useState } from "react";

const AdminDashboard = () => {
  const [pendingUsers, setPendingUsers] = useState([]);
  const [message, setMessage] = useState("");
  const [loading, setLoading] = useState(false);
  const [actionLoading, setActionLoading] = useState(null);

  useEffect(() => {
    fetchPendingUsers();
  }, []);


  useEffect(() => {
    if (message) {

      const timer = setTimeout(() => setMessage(""), 3000);
      return () => clearTimeout(timer);
    }
  }, [message]);

  const fetchPendingUsers = async () => {
    const token = localStorage.getItem("token");
    if (!token) {
      setMessage("✕Unauthorized! Please log in.");
      return;

    }

    setLoading(true);
    try {
      const response = await fetch("http://127.0.0.1:5000/admin/users/pending", {
        headers: { Authorization: `Bearer ${token}` },
      });

      const data = await response.json();
      if (response.ok) {
        setPendingUsers(data.pending_users || []);
      } else {
        setMessage(data.error || "✕Error fetching pending users.");
      }
```

```
  } catch (error) {
    setMessage("✗Network error. Try again later.");

  } finally {
    setLoading(false);
  }
};

const handleUserAction = async (userId, status) => {
  const token = localStorage.getItem("token");
  if (!token) {
    setMessage("✗Unauthorized! Please log in.");
    return;
  }

  setActionLoading(userId);

  try {
    const response = await fetch(`http://127.0.0.1:5000/admin/users/${userId}/status`, {
      method: "PUT",
      headers: {
        "Content-Type": "application/json",
        Authorization: `Bearer ${token}`,
      },
      body: JSON.stringify({ status }),
    });

    const data = await response.json();
    if (response.ok) {
      setPendingUsers((prev) => prev.filter((user) => user.id !== userId));
      setMessage(`✓User ${status} successfully!`);


    } else {
      setMessage(data.error || `✗Failed to ${status} user.`);
    }
  } catch (error) {
    setMessage(`✗Network error while trying to ${status} user.`);
  } finally {
    setActionLoading(null);
  }

};
```

```
  return (

    <div style={containerStyle}>
      <h2>Admin Dashboard - Approve or Reject Users</h2>

      {message && (
        <div style={{ color: message.includes("✅") ? "green" : "red", marginTop: "10px"
}}>
          {message}
        </div>
      )}

      {loading ? (
        <p>⏳ Loading pending users...</p>
      ) : (
        <table style={tableStyle}>
          <thead>
            <tr style={headerRowStyle}>

              <th style={tableHeaderStyle}>Email</th>
              <th style={tableHeaderStyle}>Role</th>
              <th style={tableHeaderStyle}>Actions</th>
            </tr>
          </thead>
          <tbody>
            {pendingUsers.length > 0 ? (
              pendingUsers.map((user) => (
                <tr key={user.id} style={rowStyle}>
                  <td style={tableCellStyle}>{user.email}</td>
                  <td style={tableCellStyle}>{capitalize(user.role)}</td>
                  <td style={tableCellStyle}>
                    <button
                      onClick={() => handleUserAction(user.id, "approved")}
                      style={{
                        ...approveButtonStyle,
                        opacity: actionLoading === user.id ? 0.7 : 1,


                      }}
                      disabled={actionLoading === user.id}
                    >

                      {actionLoading === user.id ? "Processing..." : "Approve"}
                    </button>
```

```
                <button
                  onClick={() => handleUserAction(user.id, "rejected")}
                  style={{
                    ...rejectButtonStyle,
                    opacity: actionLoading === user.id ? 0.7 : 1,
                  }}
                  disabled={actionLoading === user.id}
                >
                  {actionLoading === user.id ? "Processing..." : "Reject"}
                </button>
              </td>
            </tr>

          ))
        ) : (
          <tr>
            <td colSpan="3" style={{ ...tableCellStyle, textAlign: "center" }}>
              No pending users.
            </td>
          </tr>
        )}
      </tbody>
    </table>
  )}
 </div>
 );
};

const capitalize = (str) => str?.charAt(0).toUpperCase() + str?.slice(1);

const containerStyle = {

  padding: "30px",
  maxWidth: "900px",
  margin: "auto",
  backgroundColor: "#ffffff",
  borderRadius: "10px",
  boxShadow: "0 4px 12px rgba(0, 0, 0, 0.1)",
  fontFamily: "Arial, sans-serif",
};

const tableStyle = {
  width: "100%",
  borderCollapse: "collapse",
```

```
  marginTop: "20px",
};

const headerRowStyle = {
  backgroundColor: "#f2f2f2",
};

const tableHeaderStyle = {
  padding: "12px",
  textAlign: "left",
  borderBottom: "2px solid #ddd",
};

const rowStyle = {
  borderBottom: "1px solid #eee",
};

const tableCellStyle = {
  padding: "12px",
};



const approveButtonStyle = {
  marginRight: "10px",
  padding: "8px 14px",
  backgroundColor: "#28a745",
  color: "#fff",
  border: "none",
  borderRadius: "5px",
  cursor: "pointer",
};

const rejectButtonStyle = {
  padding: "8px 14px",
  backgroundColor: "#dc3545",
  color: "#fff",
  border: "none",
  borderRadius: "5px",
  cursor: "pointer",
};
export default AdminDashboard;
```

UserDashboard.js:

```
import React, { useEffect, useState, useContext } from "react";
import AuthContext from "../context/AuthContext";
import "./UserDashboard.css"; //

const UserDashboard = () => {
  const { user } = useContext(AuthContext);
  const [jobs, setJobs] = useState([]);
  const [appliedJobs, setAppliedJobs] = useState([]);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    const fetchJobsAndApplications = async () => {
      try {
        const [jobRes, appliedRes] = await Promise.all([
          fetch("/api/jobs"),
          fetch("/api/job-applications/me")
        ]);

        if (!jobRes.ok || !appliedRes.ok) {
          throw new Error("Failed to fetch data");
        }

        const jobsData = await jobRes.json();
        const appliedData = await appliedRes.json();

        setJobs(jobsData);
        setAppliedJobs(appliedData.map(app => app.job_id));
      } catch (err) {
        console.warn("Error loading dashboard data:", err);
      } finally {
        setLoading(false);
      }
    };

    fetchJobsAndApplications();
  }, []);

  const handleApply = async (jobId) => {
    try {
      const res = await fetch("/api/job-applications", {
        method: "POST",
```

```
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ job_id: jobId }),
    });

    if (res.ok) {
      setAppliedJobs(prev => [...prev, jobId]);
    } else {
      const error = await res.json();
      console.warn("Application failed:", error.message || "Unknown error");

    }
  } catch (err) {
    console.warn("Error applying for job:", err);
  }
};

return (
  <div className="user-dashboard">
    <h2>□ Welcome, {user?.username || "User"}!</h2>
    <p>Explore job opportunities tailored for you:</p>

    {loading ? (
      <p>Loading job vacancies...</p>
    ) : jobs.length === 0 ? (
      <p></p>
    ) : (
      <div className="job-list">
        {jobs.map((job) => (
          <div key={job.id} className="job-card">
            <h3>{job.title}</h3>
            <p><strong>Department:</strong> {job.department}</p>
            <p><strong>Description:</strong> {job.description}</p>
            <p><strong>Location:</strong> {job.location}</p>
            <button
              className={`apply-btn ${appliedJobs.includes(job.id) ? "applied" : ""}`}
              onClick={() => handleApply(job.id)}
              disabled={appliedJobs.includes(job.id)}
            >
              {appliedJobs.includes(job.id) ? "✓Applied" : "Apply"}
            </button>
          </div>
        ))}
      </div>
    )}
  </div>
```

```
  );
};

export default UserDashboard;
```

AdminApproval.js:

```
import React, { useEffect, useState } from "react";
import axios from "axios";
import "./AdminApproval.css";

const AdminApproval = () => {
    const [pendingUsers, setPendingUsers] = useState([]);
    const [message, setMessage] = useState("");
    const [loading, setLoading] = useState(false);
    const [actionLoading, setActionLoading] = useState(null);

    useEffect(() => {

        fetchPendingUsers();
    }, []);

    const fetchPendingUsers = async () => {
        const token = localStorage.getItem("token");
        if (!token) {
            setMessage("❌Unauthorized! Please log in.");
            return;
        }

        setLoading(true);
        try {
            const { data } = await axios.get("http://localhost:5000/admin/users/pending", {
                headers: { Authorization: `Bearer ${token}` },
            });

            setPendingUsers(data.pending_users || []);
            setMessage("");
        } catch (error) {
            console.error("Error fetching pending users:", error);
            setMessage("❌Error fetching pending users.");
        } finally {
            setLoading(false);
        }
    };
```

```
  const handleUserAction = async (userId, status) => {
    const token = localStorage.getItem("token");
    setActionLoading(userId);
    setMessage("");

    try {
      const res = await axios.put(
        `http://localhost:5000/admin/users/${userId}/status`,
        { status },
        {

          headers: {
            Authorization: `Bearer ${token}`,
            "Content-Type": "application/json",
          },
        }
      );

      if (res.status === 200) {
        setPendingUsers((prev) => prev.filter((u) => u.id !== userId));
        setMessage(`✅User ${status} successfully!`);
      } else {
        setMessage("❌Failed to update user status.");
      }
    } catch (err) {
      console.error(`Error updating user ${userId}:`, err);
      setMessage("❌Error updating user.");
    } finally {
      setActionLoading(null);
    }
  };

  return (
    <div className="admin-approval-container">
      <h2>□ Admin Approval - Pending Users</h2>

      {message && (
        <p className={message.includes("✅") ? "success-message" : "error-
message"}>
          {message}
        </p>
      )}
```

```
{loading ? (
  <p className="loading-text">⏳ Loading pending users...</p>
) : (
  <table className="approval-table">
    <thead>



      <tr>
        <th>Name</th>
        <th>Email</th>
        <th>Role</th>
        <th>Actions</th>
      </tr>
    </thead>
    <tbody>
      {pendingUsers.length > 0 ? (
        pendingUsers.map((user) => (
          <tr key={user.id}>
            <td>{user.username}</td>
            <td>{user.email}</td>
            <td>{user.role}</td>
            <td>


<div className="button-group">
                  <button
                    onClick={() => handleUserAction(user.id, "approved")}
                    className="approve-btn"
                    disabled={actionLoading === user.id}
                  >
                    {actionLoading === user.id ? "Processing..." : "Approve"}
                  </button>
                  <button
                    onClick={() => handleUserAction(user.id, "rejected")}
                    className="reject-btn"
                    disabled={actionLoading === user.id}
                  >
                    {actionLoading === user.id ? "Processing..." : "Reject"}
                  </button>
                </div>
              </td>
            </tr>
```

```
                    ))
                ) : (
                    <tr>
                        <td colSpan="4" className="no-users">
                            No pending users.
                        </td>
                    </tr>
                )}
            </tbody>


        </table>
      )}
    </div>
  );
};

export default AdminApproval;
```

AdminApplication.js:

```
import React, { useEffect, useState, useContext } from "react";
import AuthContext from "../context/AuthContext"; //
import "./AdminApplications.css";

const AdminApplications = () => {
  const [applications, setApplications] = useState([]);
  const [loading, setLoading] = useState(true);
  const { token } = useContext(AuthContext);

  useEffect(() => {
    const fetchApplications = async () => {
      try {
        const res = await fetch("http://127.0.0.1:5000/applications", {
          headers: {
            Authorization: `Bearer ${token}`,
          },
        });

        if (!res.ok) throw new Error("Failed to fetch");

        const data = await res.json();
        setApplications(data);
```

```
      } catch (err) {
        console.error(err);
      } finally {
        setLoading(false);
      }
    };

    fetchApplications();
  }, [token]);



  return (
    <div className="admin-applications">
      <h2>Job Applications</h2>
      {loading ? (
        <p>Loading...</p>
      ) : (
        <table>
          <thead>
            <tr>
              <th>Applicant</th>
              <th>Job Title</th>
              <th>Applied On</th>
              <th>Status</th>
            </tr>
          </thead>
          <tbody>
            {applications.map((app) => (
              <tr key={app.id}>


                <td>{app.applicant_name}</td>
                <td>{app.job_title}</td>
                <td>{app.application_date}</td>
                <td>{app.status}</td>
              </tr>
            ))}
          </tbody>
        </table>
      )}
    </div>
```

```
    );
};

export default AdminApplications;
```

JobVacancies.js:

```
import React, { useEffect, useState, useContext, useCallback } from "react";
import { useNavigate } from "react-router-dom";
import { ToastContainer, toast } from "react-toastify";
import AuthContext from "../context/AuthContext";
import "react-toastify/dist/ReactToastify.css";
import "./JobVacancies.css";



const formatDate = (dateStr) => {
  const date = new Date(dateStr);
  return isNaN(date) ? "N/A" : date.toLocaleDateString();
};

const JobVacancies = () => {
  const { isAuthenticated } = useContext(AuthContext);
  const navigate = useNavigate();
  const token = localStorage.getItem("token");

  const [jobs, setJobs] = useState([]);
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState("");
  const [selectedJob, setSelectedJob] = useState(null);

  const fetchJobs = useCallback(async () => {
    if (!token) return;

    setLoading(true);
    setError("");

    try {
      const res = await fetch("http://localhost:5000/jobs/list", {
        headers: {
          "Content-Type": "application/json",
          Authorization: `Bearer ${token}`,
        },
      });
```

```
    const data = await res.json();

    if (!res.ok) {
      if (res.status === 401) {
        toast.error("Session expired. Please log in again.");
      }
      throw new Error(data?.error || "Failed to fetch job vacancies");
    }

    setJobs(data?.vacancies || []);
  } catch (err) {
    console.error("Fetch Jobs Error:", err);
    setError(err.message || "Something went wrong.");


  } finally {
    setLoading(false);
  }
}, [token]);

useEffect(() => {
  if (isAuthenticated && token) {
    fetchJobs();
  }
}, [isAuthenticated, token, fetchJobs]);

useEffect(() => {
  const handleEscape = (e) => {
    if (e.key === "Escape") setSelectedJob(null);
  };
  window.addEventListener("keydown", handleEscape);
  return () => window.removeEventListener("keydown", handleEscape);
}, []);

const handleApply = (job) => {
  navigate(`/apply/${job.job_id}`, { state: { job } });
  setTimeout(() => setSelectedJob(null), 200);
};

const renderJobCard = (job, index) => {
  const jobId = job?.job_id ?? index;

  return (
    <div key={`job-${jobId}`} className="job-card">
      <button className="job-title-clickable" onClick={() => setSelectedJob(job)}>
```

```
        {job.job_role || "Untitled Role"}
      </button>
      <p><strong>Department:</strong> {job.department || "N/A"}</p>
      <button
        className="apply-btn"
        onClick={() => handleApply(job)}
      >
        Apply Now
      </button>
    </div>
  );
};


const renderJobModal = () => {
  if (!selectedJob) return null;

  const {
    job_id,
    job_role,
    department,
    job_type,
    salary,
    experience_required,


    skills_required,
    posted_date,
    application_deadline,
    job_description,
  } = selectedJob;

  return (
    <div className="modal-overlay" onClick={() => setSelectedJob(null)}>
      <div
        className="modal-content"
        onClick={(e) => e.stopPropagation()}
        role="dialog"
        aria-modal="true"
      >
        <h3>{job_role || "Untitled"}</h3>
        <p><strong>Department:</strong> {department || "N/A"}</p>
        <p><strong>Job Type:</strong> {job_type || "N/A"}</p>
        <p><strong>Salary:</strong> ₹{salary?.toLocaleString() || "N/A"}</p>
        <p><strong>Experience Required:</strong> {experience_required || "N/A"}</p>
```

```
        <p><strong>Skills Required:</strong> {skills_required || "N/A"}</p>
        <p><strong>Posted On:</strong> {formatDate(posted_date)}</p>
        <p><strong>Application Deadline:</strong> {formatDate(application_deadline)}</p>
        <p><strong>Description:</strong> {job_description || "N/A"}</p>

        <div className="modal-actions">
          <button
            className="apply-btn"
            onClick={() => handleApply(selectedJob)}
          >
            Apply Now
          </button>
          <button className="close-btn" onClick={() => setSelectedJob(null)}>
            Close
          </button>
        </div>
      </div>
    </div>
  );
};

if (!isAuthenticated || !token) {
  return (

    <div className="job-vacancies-container">
      <h2>Job Opportunities</h2>
      <p className="info-message">Please log in to view and apply for jobs.</p>
    </div>
  );
}

return (
  <div className="job-vacancies-container">
    <h2>Job Opportunities</h2>



    {loading && <div className="loading-spinner">Loading job vacancies...</div>}
    {!loading && error && <p className="error-message">{error}</p>}
    {!loading && !error && jobs.length > 0 && (
      <div className="job-list">{jobs.map(renderJobCard)}</div>
    )}
    {!loading && !error && jobs.length === 0 && (
      <p className="no-data">No job vacancies found.</p>
```

```
      )}

      {renderJobModal()}
      <ToastContainer position="top-center" autoClose={2500} hideProgressBar />
    </div>
  );
};

export default JobVacancies;
```

ApplyJob.js

```
import React, { useContext, useState, useEffect } from "react";
import { useLocation, useNavigate } from "react-router-dom";
import AuthContext from "../context/AuthContext";

import "./ApplyJob.css";
import { ToastContainer, toast } from "react-toastify";
import "react-toastify/dist/ReactToastify.css";

const ApplyJob = () => {
  const { isAuthenticated } = useContext(AuthContext);
  const location = useLocation();
  const navigate = useNavigate();
  const token = localStorage.getItem("token");

  const [job] = useState(() => {
    const jobFromLocation = location.state?.job;
    if (jobFromLocation) {
      sessionStorage.setItem("selectedJob", JSON.stringify(jobFromLocation));
      return jobFromLocation;
    }
    const storedJob = sessionStorage.getItem("selectedJob");
    return storedJob ? JSON.parse(storedJob) : null;
  });

  const [formData, setFormData] = useState({
    fullName: "",
    email: "",
    resume: null,
    resumeLink: "",
    coverLetter: "",
  });

  const [loading, setLoading] = useState(false);
```

```
useEffect(() => {
  if (!isAuthenticated) navigate("/login");
}, [isAuthenticated, navigate]);

useEffect(() => {
  if (!job) {
    toast.warn("No job selected. Redirecting...");
    setTimeout(() => navigate("/job-vacancies"), 2000);
  }
}, [job, navigate]);

const handleChange = (e) => {
  const { name, value, files } = e.target;

  if (name === "resume" && files.length > 0) {
    setFormData((prev) => ({


      ...prev,
      resume: files[0],
      resumeLink: "",
    }));
  } else if (name === "resumeLink") {
    setFormData((prev) => ({
      ...prev,
      resumeLink: value,
      resume: null,
    }));
  } else {
    setFormData((prev) => ({
      ...prev,
      [name]: value,
    }));
  }
};

const handleSubmit = async (e) => {
  e.preventDefault();

  const { fullName, email, resume, resumeLink, coverLetter } = formData;

  if (!fullName || !email || (!resume && !resumeLink)) {
    toast.error("Please fill all required fields.");
    return;
  }
```

```
const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
if (!emailRegex.test(email)) {
  toast.error("Please enter a valid email address.");
  return;
}

if (resume) {
  const allowedTypes = [
    "application/pdf",
    "application/msword",
    "application/vnd.openxmlformats-officedocument.wordprocessingml.document",
  ];
  if (!allowedTypes.includes(resume.type)) {
    toast.error("Resume must be a PDF or Word document.");
    return;
  }
  if (resume.size > 2 * 1024 * 1024) {
    toast.error("Resume file size must be under 2MB.");
    return;
  }
}

setLoading(true);


try {
  const form = new FormData();
  form.append("fullName", fullName);
  form.append("email", email);
  form.append("coverLetter", coverLetter);
  form.append("jobRole", job?.JobRole || job?.job_role || "Unknown");
  form.append("department", job?.Department || job?.department || "Unknown");
  form.append("job_id", job?.id || job?.job_id || "");

  if (resume) {
    form.append("resumeFile", resume);
  } else {
    form.append("resumeLink", resumeLink);
  }

  const response = await fetch("http://localhost:5000/jobs/apply", {
    method: "POST",
    headers: {
      Authorization: `Bearer ${token}`,
    },
    body: form,
```

```
    });

    const data = await response.json();
    console.log("Server response:", data);

    if (!response.ok) {
      throw new Error(data?.error || "Failed to apply.");
    }

    toast.success("Application submitted successfully!");
    setFormData({
      fullName: "",
      email: "",
      resume: null,
      resumeLink: "",
      coverLetter: "",
    });

    setTimeout(() => navigate("/job-vacancies"), 2000);
  } catch (err) {
    console.error("Application error:", err);
    toast.error(err.message || "Something went wrong.");
  } finally {
    setLoading(false);
  }
};

return (
  <div className="apply-job-container">
    <h2>


      Apply for <strong>{job?.JobRole || job?.job_role || "this Job"}</strong>
      {job?.Department || job?.department ? (
        <span style={{ fontWeight: "normal", color: "#777" }}>
          {" "}in {job?.Department || job?.department}
        </span>
      ) : null}
    </h2>

    <form className="apply-form" onSubmit={handleSubmit}>
      <label htmlFor="fullName">Full Name</label>
      <input
        type="text"
        name="fullName"
        id="fullName"
```

```
  value={formData.fullName}
  onChange={handleChange}
  required
/>

<label htmlFor="email">Email Address</label>
<input
  type="email"
  name="email"
  id="email"
  value={formData.email}
  onChange={handleChange}
  required
/>

<label htmlFor="resume">Upload Resume (PDF/DOC)</label>
<input
  type="file"
  name="resume"
  id="resume"
  accept=".pdf,.doc,.docx"
  onChange={handleChange}
  disabled={!!formData.resumeLink}
  required={!formData.resumeLink}
/>

<label htmlFor="resumeLink">OR Paste Resume Link</label>
<input
  type="url"
  name="resumeLink"
  id="resumeLink"
  placeholder="https://your-resume-link.com"
  value={formData.resumeLink}
  onChange={handleChange}
  disabled={!!formData.resume}
  required={!formData.resume}
/>

<label htmlFor="coverLetter">Cover Letter (optional)</label>
<textarea
  name="coverLetter"
  id="coverLetter"
  rows={4}
  placeholder="Cover Letter"
  value={formData.coverLetter}
  onChange={handleChange}
```

```
        />
        <button className="apply-btn" type="submit" disabled={loading}>
          {loading ? <span className="spinner" /> : "Submit Application"}
        </button>
      </form>

      <ToastContainer position="top-center" autoClose={2500} />
    </div>
  );
};

export default ApplyJob;
```

MyApplicatons.js

```
import React, { useEffect, useState } from "react";
import axios from "axios";
import "./MyApplications.css";

const MyApplications = () => {
  const [applications, setApplications] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    const fetchApplications = async () => {
      try {
        const token = localStorage.getItem("token");

        if (!token) {
          setError("Authentication token not found. Please log in.");
          setLoading(false);
          return;
        }

        const res = await axios.get("http://127.0.0.1:5000/applications/my",
{
          headers: {
            Authorization: `Bearer ${token}`,
```

```
        },
      });

      const fetchedApps = res.data.applications || [];

      // Sort by date (most recent first)
      const sorted = fetchedApps.sort(
        (a, b) => new Date(b.applied_on) - new Date(a.applied_on)
      );

      setApplications(sorted);
    } catch (err) {

      console.error("Error fetching applications:", err);
      setError("Failed to load applications.");
    } finally {
      setLoading(false);
    }
  };

  fetchApplications();
}, []);

// Format applied date
const formatDate = (dateStr) => {
  try {
    return new Date(dateStr).toLocaleDateString();
  } catch {
    return "N/A";
  }
};

return (
  <div className="container">
    <h2 className="page-title">My Applications</h2>

    {loading && <p>Loading applications...</p>}
    {error && <p className="error-message">{error}</p>}
```

```
        {!loading && !error && applications.length === 0 && (
          <p>No applications submitted yet.</p>
        )}

        {!loading && !error && applications.length > 0 && (
          <div className="applications-list">
            {applications.map((app) => (
              <div key={app.id} className="job-card">
                <h3>{app.job_title}</h3>
                <p><strong>Department:</strong> {app.department}</p>
                <p>
                  <strong>Status:</strong>{" "}
                  <span className={`status-badge
${app.status?.toLowerCase() || "pending"}`}>
                    {app.status || "Pending"}
                  </span>
                </p>

                <p><strong>Applied On:</strong>
{formatDate(app.applied_on)}</p>
              </div>

            ))}
          </div>
        )}
      </div>
    );
  };

  export default MyApplications;
```

Sidebar.js

```
import React, { useState, useEffect } from "react";
import { Link, useNavigate } from "react-router-dom";
import {
  FaBars,
  FaChartBar,
  FaUpload,
  FaSignOutAlt,
  FaClipboardCheck,
  FaUserCheck,
  FaBriefcase,
  FaClipboardList,
} from "react-icons/fa";
import "./Sidebar.css";

const Sidebar = () => {
  const navigate = useNavigate();
  const [isCollapsed, setIsCollapsed] = useState(
    () => JSON.parse(localStorage.getItem("sidebarCollapsed")) || false
  );
  const [userRole, setUserRole] = useState(null);

  useEffect(() => {
    const rawRole = localStorage.getItem("role");
    if (rawRole) {
      const normalized = rawRole.trim().toLowerCase();
      let finalRole = normalized;

      // Normalize custom role labels if needed
      if (normalized === "data analyst") finalRole = "analyst";
      else if (normalized === "hr manager") finalRole = "hr";

      setUserRole(finalRole);
      console.log("□ Sidebar role:", finalRole);
    } else {
      console.warn("□ No role found in localStorage.");
    }
  }, []);
```

```
useEffect(() => {
  localStorage.setItem("sidebarCollapsed", JSON.stringify(isCollapsed));
}, [isCollapsed]);

const toggleSidebar = () => setIsCollapsed((prev) => !prev);

const handleLogout = () => {
  localStorage.clear(); // Clear all stored user session data
  navigate("/login", { replace: true });
};

const renderAdminLinks = () =>
  userRole === "admin" && (
    <>
      <li>
        <Link to="/admin-approval">
          <FaUserCheck className="icon" />
          {!isCollapsed && <span>Admin Approval</span>}

        </Link>
      </li>
      <li>
        <Link to="/admin-applications">
          <FaClipboardList className="icon" />
          {!isCollapsed && <span>Job Applications</span>}
        </Link>
      </li>
    </>
  );

const renderLinksByRole = () => {
  switch (userRole) {
    case "admin":
    case "analyst":
    case "hr":
      return (
        <>
          <li>
            <Link to="/dashboard">
```

```
            <FaChartBar className="icon" />
            {!isCollapsed && <span>Dashboard</span>}
          </Link>
        </li>
        <li>
          <Link to="/upload">
            <FaUpload className="icon" />
            {!isCollapsed && <span>Upload Data</span>}
          </Link>
        </li>
        <li>
          <Link to="/predict">
            <FaClipboardCheck className="icon" />
            {!isCollapsed && <span>Predictor</span>}
          </Link>
        </li>
      </>
    );

  case "employee":
    return (
      <>
        <li>
          <Link to="/job-vacancies">

            <FaBriefcase className="icon" />
            {!isCollapsed && <span>Job Vacancies</span>}
          </Link>
        </li>
        <li>
          <Link to="/upload">
            <FaUpload className="icon" />
            {!isCollapsed && <span>Upload Data</span>}
          </Link>
        </li>
        <li>
          <Link to="/predict">
            <FaClipboardCheck className="icon" />
            {!isCollapsed && <span>Predictor</span>}
```

```
            </Link>
          </li>
          <li>
            <Link to="/my-applications">
              <FaClipboardList className="icon" />
              {!isCollapsed && <span>My Applications</span>}
            </Link>
          </li>
        </>
      );

    case "users":
      return (
        <>
          <li>
            <Link to="/user-dashboard">
              <FaChartBar className="icon" />
              {!isCollapsed && <span>User Dashboard</span>}
            </Link>
          </li>
          <li>
            <Link to="/job-vacancies">
              <FaBriefcase className="icon" />
              {!isCollapsed && <span>Job Vacancies</span>}
            </Link>
          </li>
          <li>
            <Link to="/my-applications">
              <FaClipboardList className="icon" />
              {!isCollapsed && <span>My Applications</span>}
            </Link>
          </li>
        </>
      );

    default:
      return null;
  }
};
```

```
  return (
    <div className={`sidebar ${isCollapsed ? "collapsed" : ""}`}>
      <div className="sidebar-header">
        <h2>{isCollapsed ? "EA" : "Employee Attrition"}</h2>

        <button
          className="toggle-btn"
          onClick={toggleSidebar}
          aria-label={isCollapsed ? "Expand Sidebar" : "Collapse Sidebar"}
        >
          <FaBars />
        </button>
      </div>

      <ul className="sidebar-menu">
        {renderLinksByRole()}
        {renderAdminLinks()}
      </ul>


      <button
        className="logout-btn"
        onClick={handleLogout}
        aria-label="Logout"
      >
        <FaSignOutAlt className="icon" />
        {!isCollapsed && <span>Logout</span>}
      </button>
    </div>
  );
};

export default Sidebar;
```

ProtectedRoutes.js

```
 import { Navigate, Outlet, useLocation } from "react-router-dom";
import { useContext } from "react";
import AuthContext from "../context/AuthContext";

const ProtectedRoutes = ({ requiredRole }) => {
  const { isAuthenticated = false, userRole = "" } = useContext(AuthContext);
  const location = useLocation();

  const currentPath = location.pathname;
  const normalizedRole = userRole?.toLowerCase() || "";

  console.log("□ ProtectedRoutes Check:", {
    isAuthenticated,
    userRole,
    requiredRole,
    currentPath,
  });

  if (!isAuthenticated) {
    console.warn("□ Not authenticated. Redirecting to /login");
    return <Navigate to="/login" replace />;
  }

  const allowedPathsForUsers = [
    "/job-vacancies",
    "/apply",
    "/user-dashboard",
    "/my-applications",
  ];

  const allowedPathsForEmployee = [
    "/job-vacancies",
    "/apply",
    "/user-dashboard",
    "/my-applications",
    "/predict",
    "/upload",
  ];
```

```
const allowedPathsForAnalyst = [
  "/dashboard",
  "/analytics",
  "/insights",
  "/report",
];

if (
  normalizedRole === "users" &&
  !allowedPathsForUsers.some((path) => currentPath.startsWith(path))
) {
  console.warn("□ 'users' role trying to access a restricted route.");
  return <Navigate to="/job-vacancies" replace />;
}

// □ Restrict 'employee'
if (
  normalizedRole === "employee" &&
  !allowedPathsForEmployee.some((path) => currentPath.startsWith(path))
) {
  console.warn("□ 'employee' role trying to access a restricted route.");
  return <Navigate to="/job-vacancies" replace />;
}

if (
  normalizedRole === "analyst" &&
  !allowedPathsForAnalyst.some((path) => currentPath.startsWith(path))
) {
  console.warn("□ 'analyst' role trying to access a restricted route.");
  return <Navigate to="/dashboard" replace />;
}

if (requiredRole) {
  const allowedRoles = new Set(
    requiredRole.split(",").map((role) => role.trim().toLowerCase())
  );



  if (!allowedRoles.has(normalizedRole)) {
    console.warn(
      `□ Role mismatch. Required: [${[...allowedRoles].join(", ")}], Found:
```

```
${normalizedRole}. Redirecting to /dashboard`
      );



      return <Navigate to="/dashboard" replace />;
    }
  }

  return <Outlet />;
};

export default ProtectedRoutes;
```

7. TESTING

7.1 Introduction:

In this software testing has been defined as the process of analysing a software item to detect the differences between existing and required conditions and to evaluate the features of the software item. Software testing is the process used to assess the quality of computer software. It involves operation of a system or application under controlled conditions and evaluating the results. The controlled conditions should include both normal and abnormal conditions.
Testing should intentionally attempt to make things go wrong to determine if things happen when they should.

There are two types of software testing:

•Black box testing-internal system design is not considered in this type of testing. test are based on requirements and functionality.
•White box testing-this testing is based on knowledge of thaw internal logic of an applications code. Tests are based on coverage of code statements, branches, paths and conditions White box testing is applicable at the unit, integration and system levels of the software testing process.

7.2 Testing objectives

•Finding defects which may get created by the programmer while developing the software.

•Gaining confidence in and providing information about the level of attrition risk and percentage

•To prevent defects

•To make sure that the end results meets the company and hr requirements.

•To ensure that it satisfies the SRS that is System Requirement Specification.

7.3 Testing steps

7.3.1 Unit Testing

Unit testing focuses efforts on the smallest unit of software design. This is known as module testing. The modules are tested separately. The test is carried out during programming stage itself. In this step, each module is found to be working satisfactory as regards to the expected output from the module

7.3.2 Integration Testing

Data can be lost across an interface. One module can have an adverse effect on another, sub functions, when combined, may not be linked in desired manner in major functions.
Integration testing is a systematic approach for constructing the program structure, while at the same time conducting test to uncover errors associated within the interface. The objective is to take unit tested modules and builds program structure. All the modules are combined and tested as a whole

7.3.3 Validation

At the culmination of the integration testing, Software is completely assembled as a package. Interfacing errors have been uncovered and corrected and a final series of software test begin in validation testing. Validation testing can be defined in many ways, but a simple definition is that the validation succeeds when the software functions in a manner that is expected by the customer. After validation test has been conducted, one of the three possible conditions exists.

•The function or performance characteristics confirm to specification and are accepted.

•A deviation from specification is uncovered and a deficiency lists is created

•Proposed system under consideration has been tested by using validation test and found to be working satisfactory.

7.3.4 Output Testing

After performing the validation testing, the next step is output testing of the proposed system, since no system could be useful if it does not produce the required output in a specific format. The output format on the screen is found to be correct. The format was designed in the system design time according to the user needs. For the hard copy also; the output comes as per the specified requirements by the user. Hence output testing did not result in any correction for the system

7.3.5 User acceptance testing

User acceptance of a system is the key factor for the success of any system. The system under consideration is tested for the user acceptance by constantly keeping in touch with the prospective system users at the time of developing and making changes whenever required.
This is done in regard to the following point:

•Input screen design.
•Output screen design.

7.3.1.1 Login

Input	Test Condition	Test Output	Comments
Login	If the admin/HR Manager/Data Analyst/Employee/Users email_id is not entered	Please fill out this field.	Username is required
	If the password is not entered	Please fill out this field.	Password is required
	If the username and password are not valid	invalid username and password	Give valid username and password
	If the User email_id and password are valid.	Successfully logged in.	Appropriate password and email is entered and the User is now logged in.
	If the Admin email_id and password are valid.	Successfully logged in.	Appropriate password and email is entered and the Admin is now logged in to the admin dashboard.

7.3.1.2 HR Manager/Data Analyst/Employee / Users Registration Form :

Input	Test Condition	Test Output	Comments
Register	If the username is not entered	Please fill out this field.	User name is required
	If the email is not entered	Please fill out this field.	email address is required
	If the password . is not entered	Please fill out this field.	password required
	If the confirm password is not entered	Please fill out this field.	Type password once again
	If these above fields are valid	Registeredsuccessfully. redirecting tologin……	User registered

7.3.1.3 DataUpload :

Input	Test Condition	Test Output	Comments
Upload files	If the user try to ulpload the wrong file type	Invaid filetype. Only csv ,xls and xlsx are allowed	Upload csv , xls and xlsx files
	If user selected the correct file type	File uploaded successfully.file preview is shown	File uploaded and a priview of uploaded file is shown

7.3.1.4 Predictor :

Input	Test Condition	Test Output	Comments
Predict	If the feilds are not entered	Please fill out this field.	values is required
	If the above field are filled	Predicting…..	Prediction is shown

7.3.1.5 Apply job :

Input	Test Condition	Test Output	Comments
Apply	If fields are not filled	Please fill out this feild	Fill the fields
	If all fields are filled	Application submitted successfully	Applied for job

7.3.2 Integration testing :

7.3.2.1 Admin Login :

Test No.	Test Description	Expected Output	Expected Output
1	Login	The site is redirected to admin Dashboard	Success

7.3.2.1 User/Employee register page and login :

Test No.	Test Description	Test Output	Expected Output
1.	Register	The entered data is stored in database and the site is redirected to login page	Success
2.	Login	The site is redirected to job applications	Success

7.3.2.1 HR Manager/ Data Analyst register page and login :

Test No.	Test Description	Test Output	Expected Output
1.	Register	The entered data is stored in database and the site is redirected to login page	Success
2.	Login	The site is redirected to Dashboard	Success

7.3.2.2 Admin dashboard :

Test No.	Test Description	Expected Output	Expected Output
1.	Admin dashboard	The site is redirected to Admin dashboard	Success
2.	Upload Data	The site is redirected to Upload Data page	Success
3.	Predictor	The site is redirected to predictor page	Success
4.	Admin Approval	The site is redirected to Admin approval Page	Success
5.	Job Applications	The site is redirected to job applications page	Success

7.3.2.3 HR Manager/Data Analyst Dashboard

Test No.	Test Description	Expected Output	Expected Output
1.	Dashboard	The site is redirected to Dashboard	Success
2.	Upload Data	The site is redirected to Upload data Page	Success
3.	Predictor	The site is redirected to Predictor Page	Success

7.3.2.4 Employee Dashboard

Test No.	Test Description	Expected Output	Expected Output
1.	Job Vacancies	The site is redirected to Job Vacancies	Success
2.	Upload Data	The site is redirected to Upload data Page	Success
3.	Predictor	The site is redirected to Predictor Page	Success
4.	My Applications	The site is redirected to My Applications Page	Success

7.3.2.7 Users Dashboard

Test No.	Test Description	Expected Output	Expected Output
1.	Job Vacancies	The site is redirected to Job Vacancies	Success
2.	User Dashboard	The site is redirected to User Dashboard Page	Success
3.	My Applications	The site is redirected to My Applications Page	Success

7.3.3 System Testing:

System Testing is the testing of a complete and fully integrated software product. Usually, software is only one element of a larger computer-based system. Ultimately, software is interfaced with other software/hardware systems. System Testing is actually a series of different tests whose sole purpose is to exercise the full computer-based system.

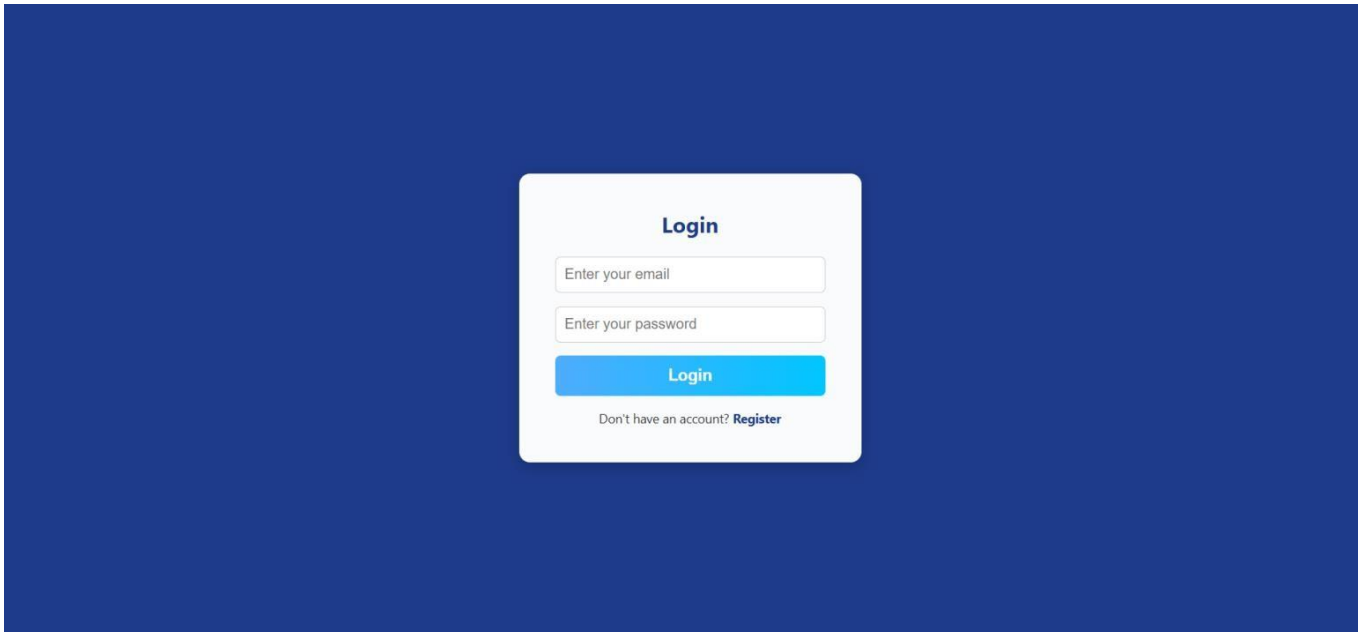
7.3.2.5 System testing tables:

Sl. No	Test condition	Test report
1.	System lading	Successful
2.	System run procedure	Successful
3.	File I/O operation	Successful
4.	Database communication	Successful
5.	Server/client interaction	Successful
6.	Memory usage	Normal
7.	System processor usage	Normal
8.	Authentication/Authorization	Successful

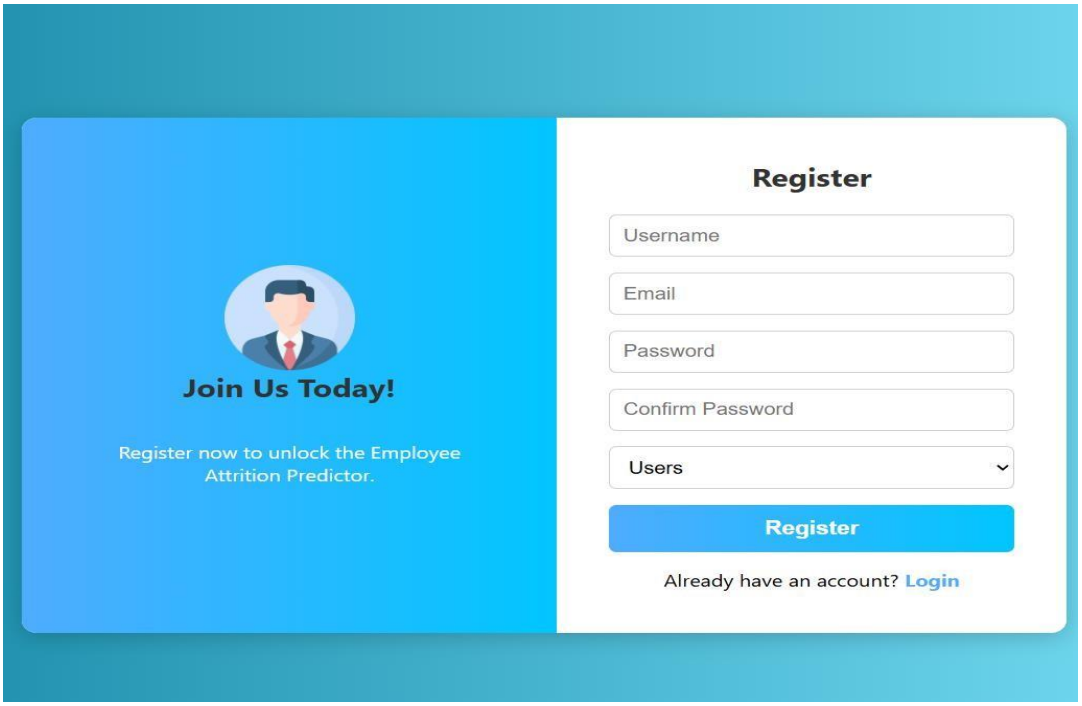
8. USER INTERFACE:

8.1 Screenshots:

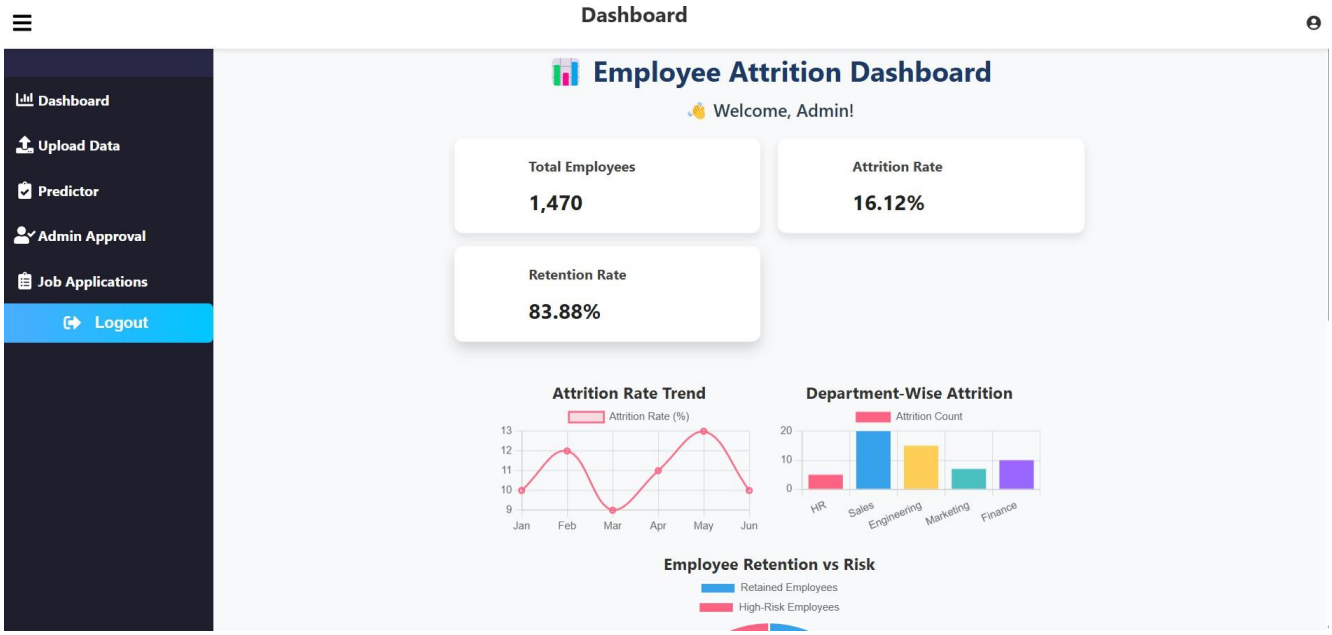
8.1.1 Login :

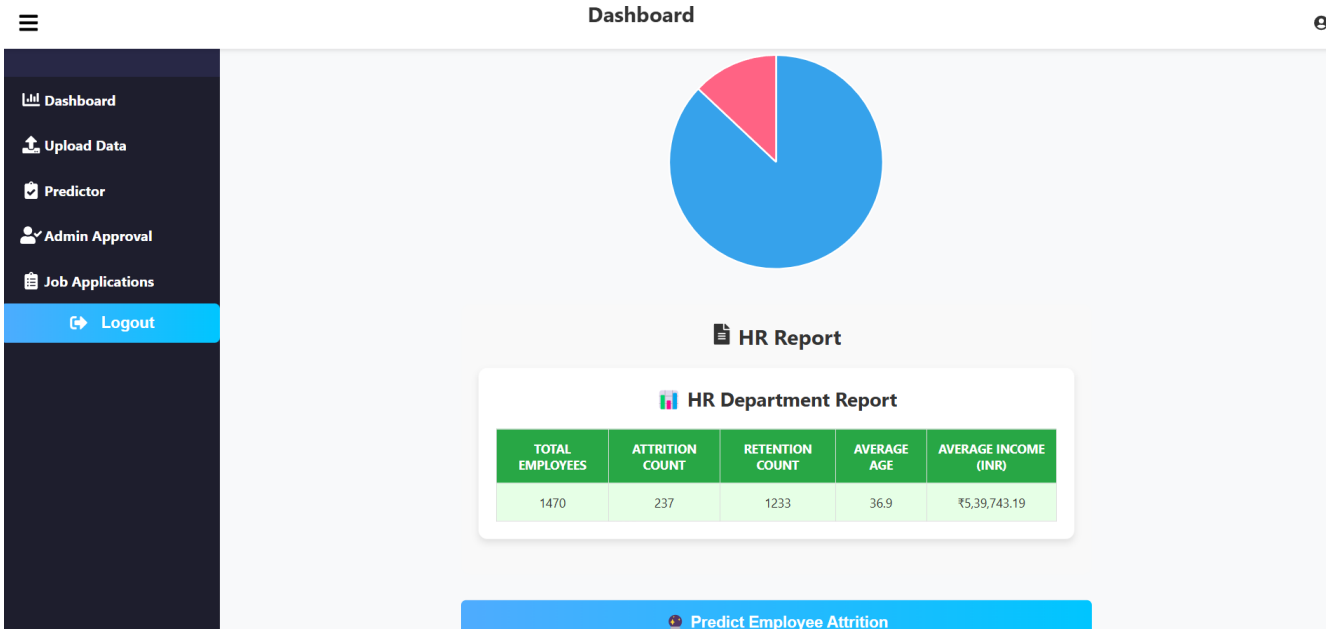


8.1.2 Register page:

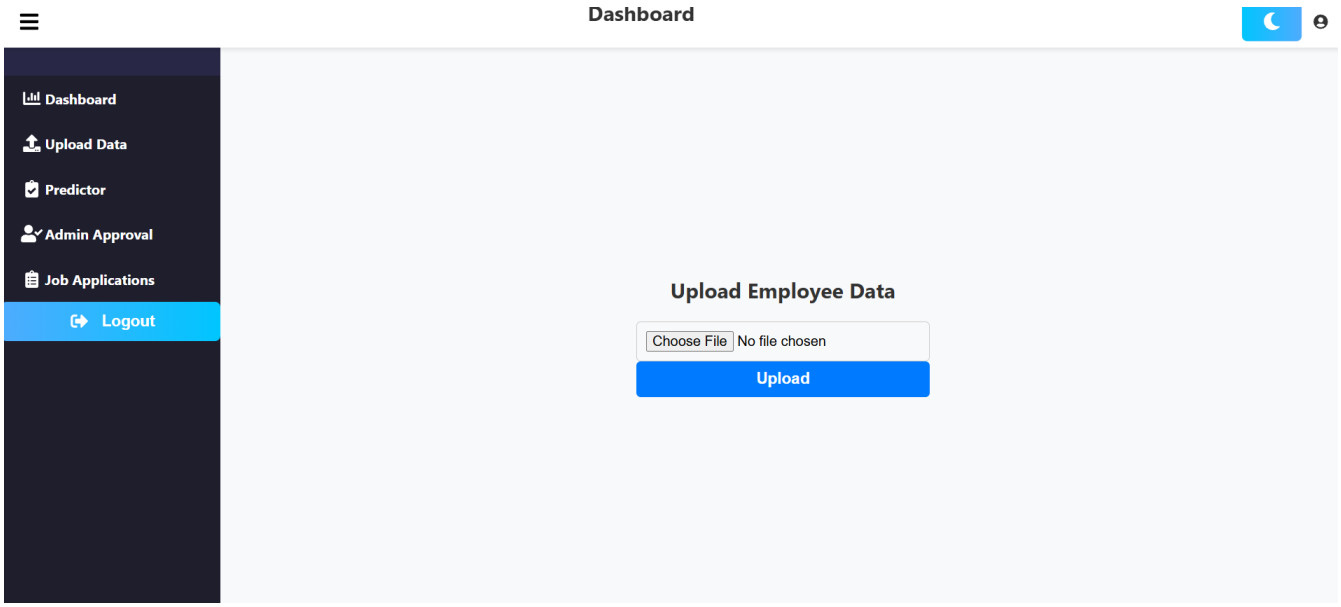


8.1.3 Admin Dashboard:

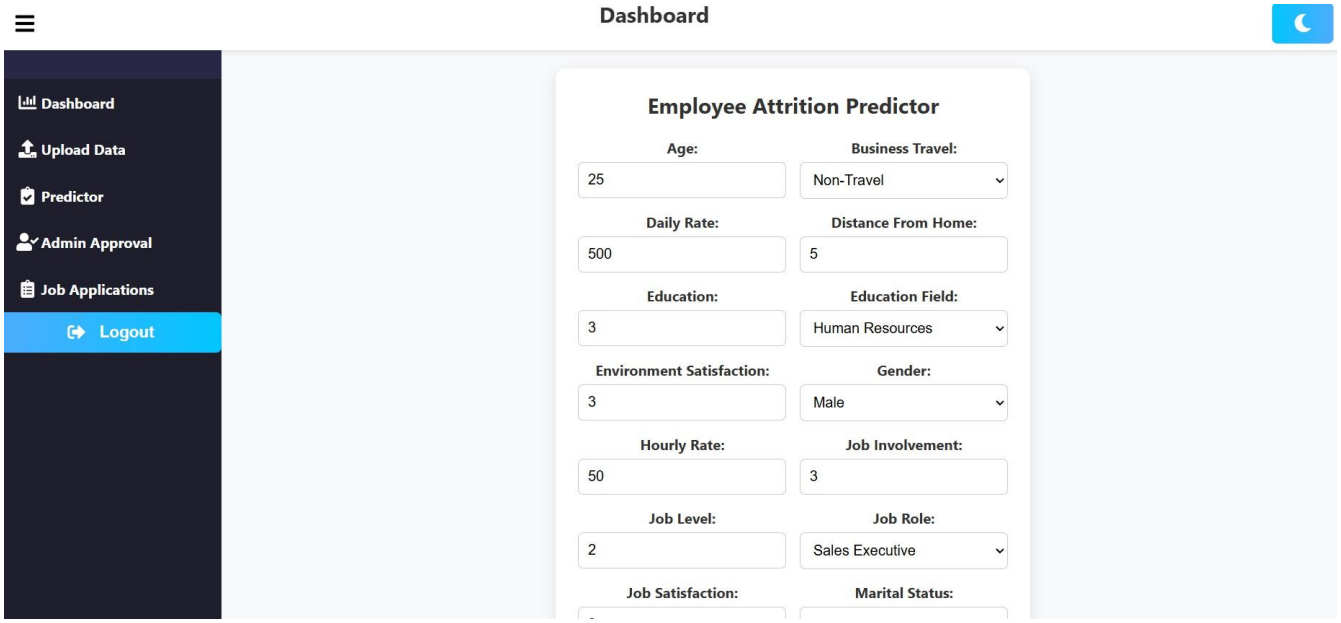


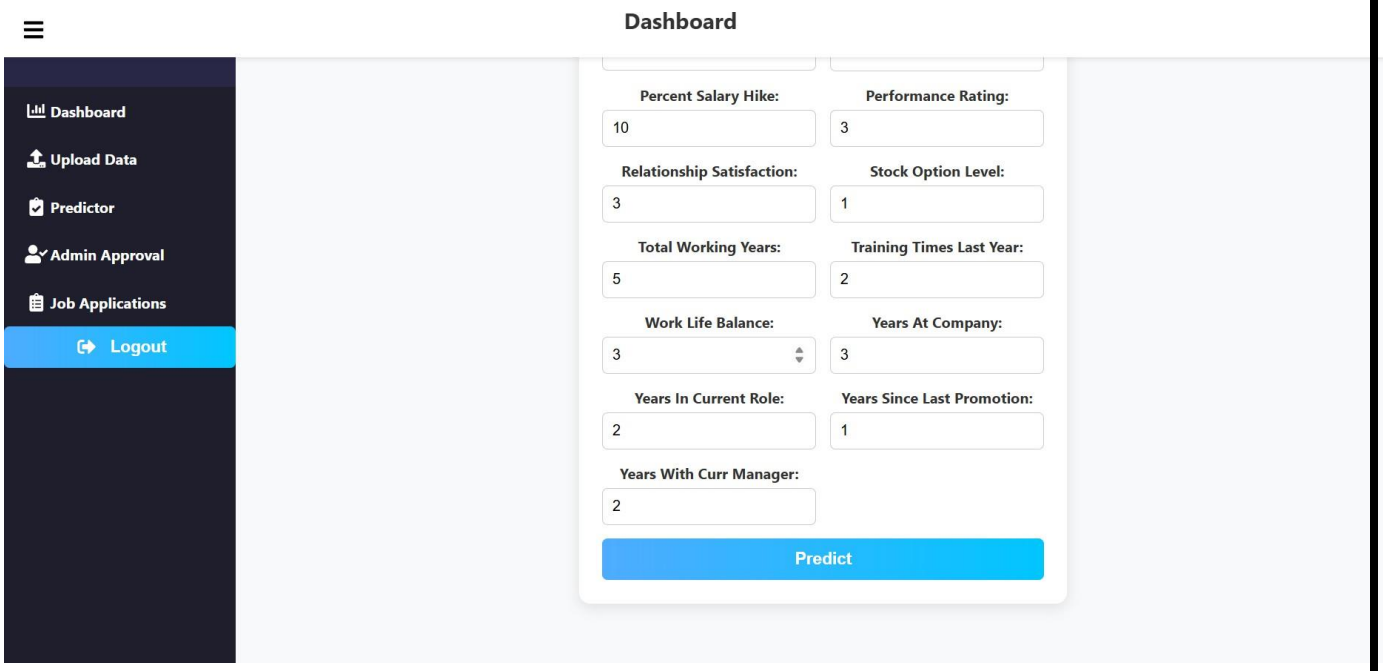


8.1.4 Upload Data Page:

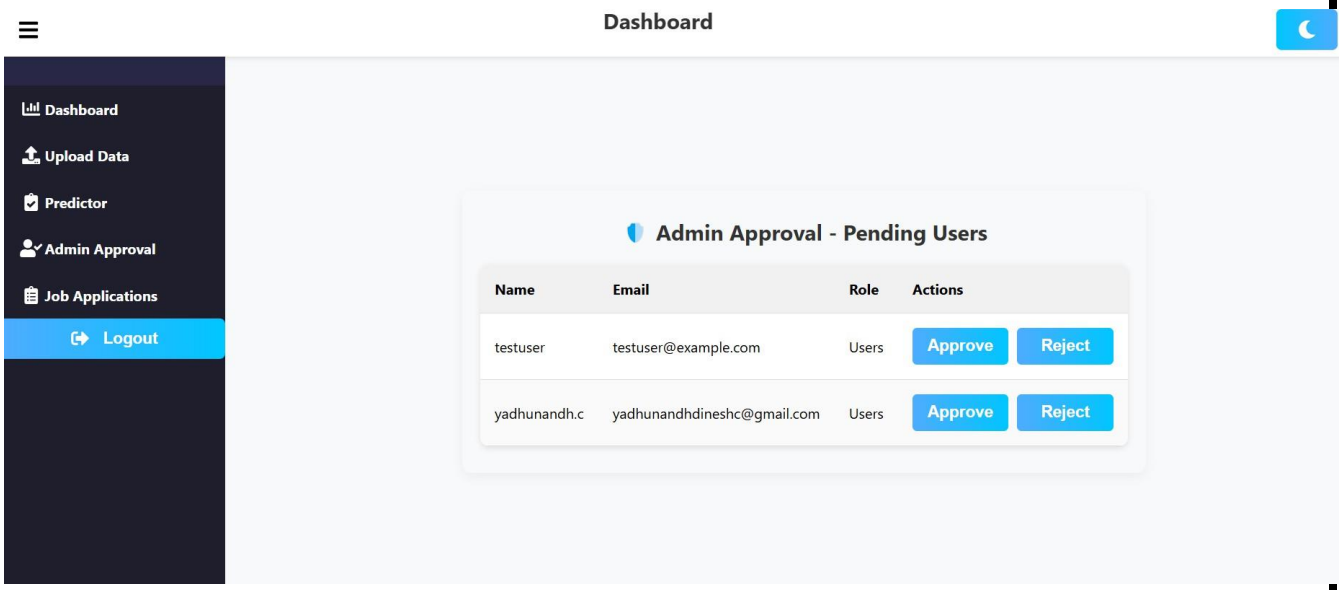


8.1.6 Predictor Page:

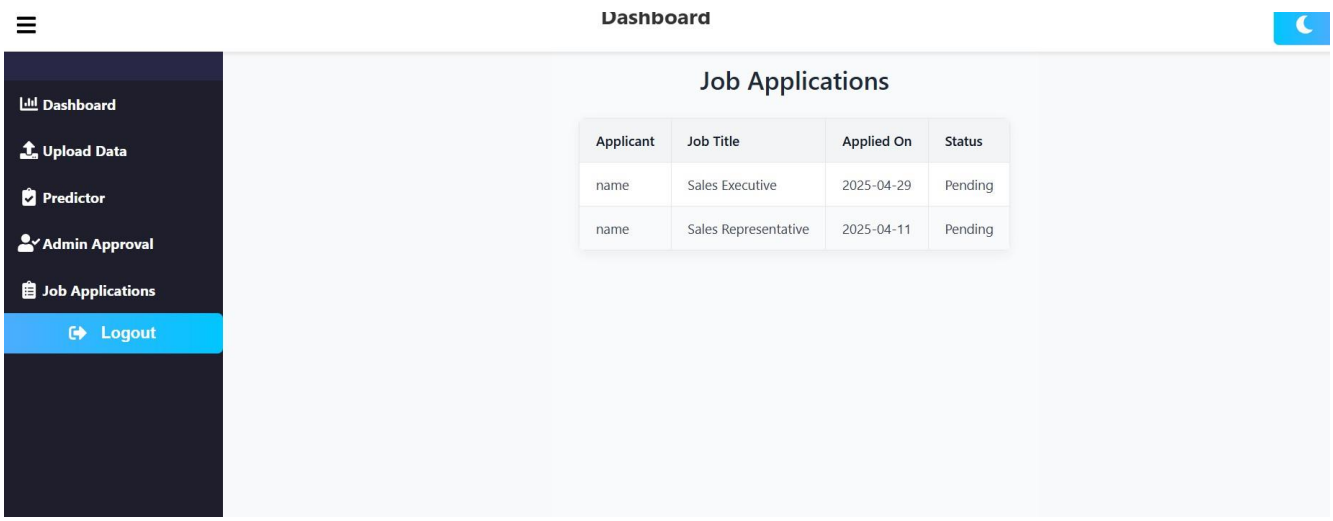




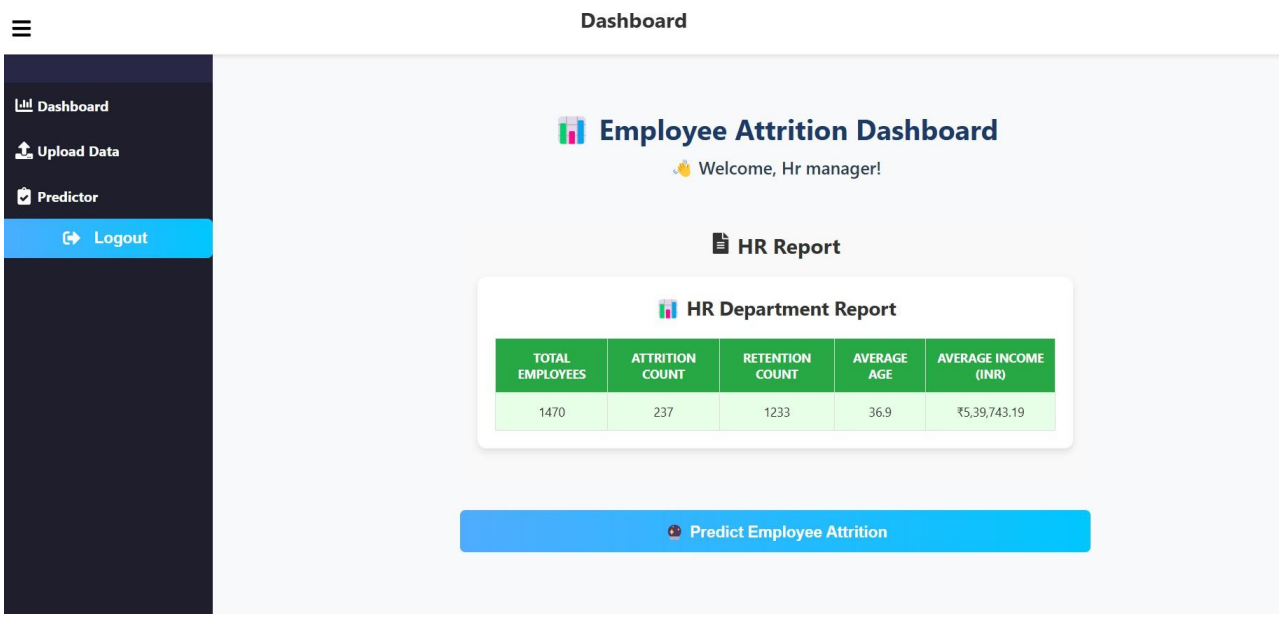
8.1.7 Admin Approval page



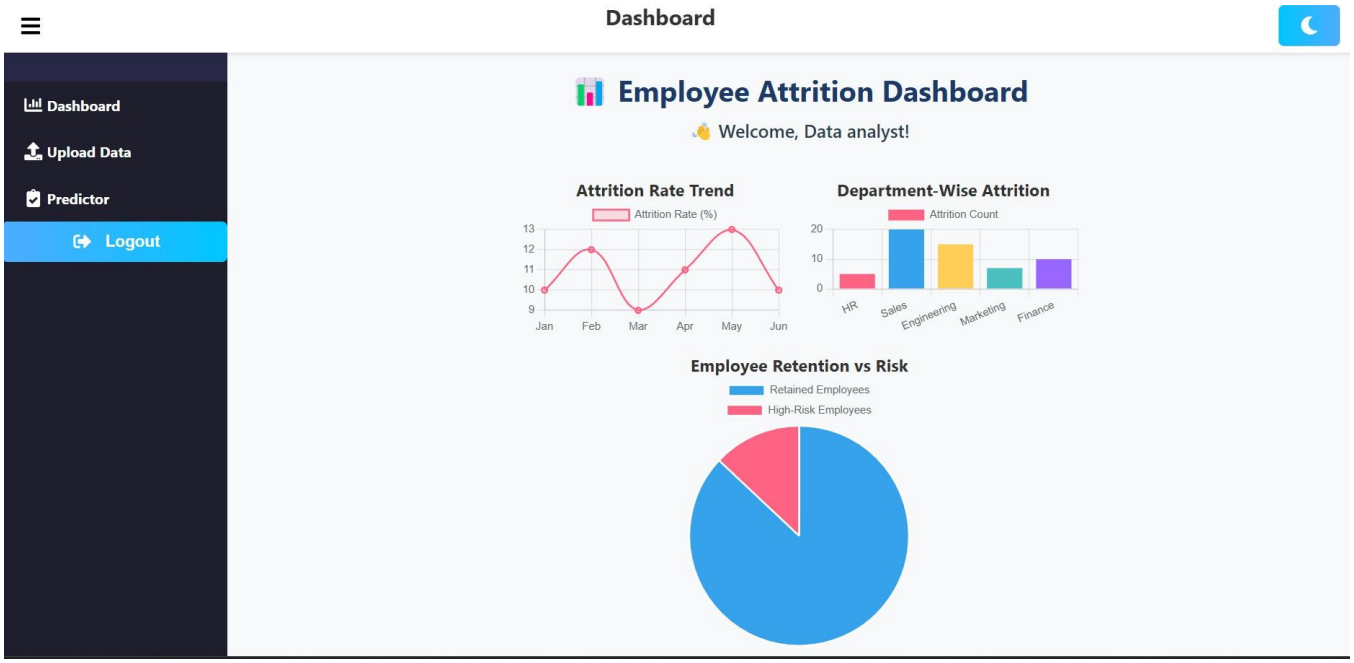
8.1.8 Job Applications page:



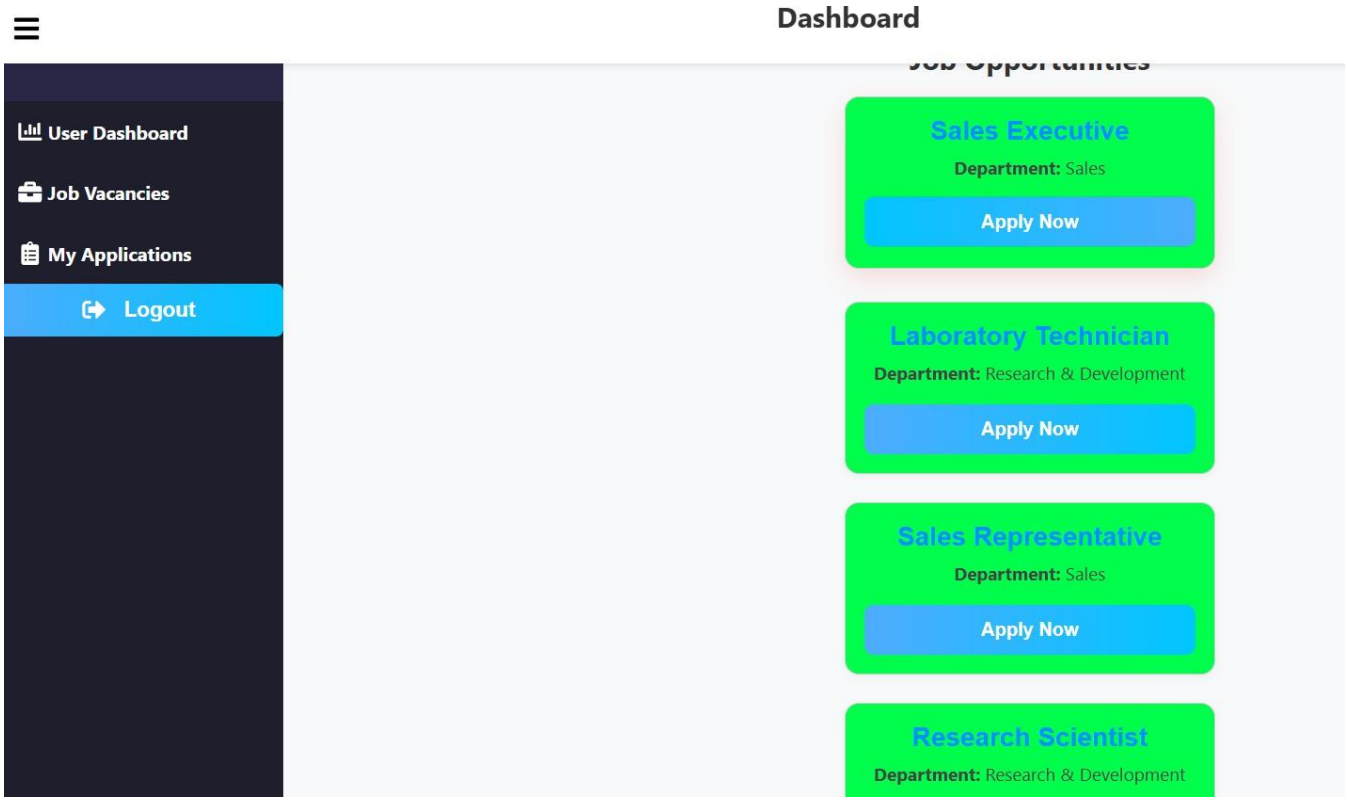
8.1.9 HR Dashboard:



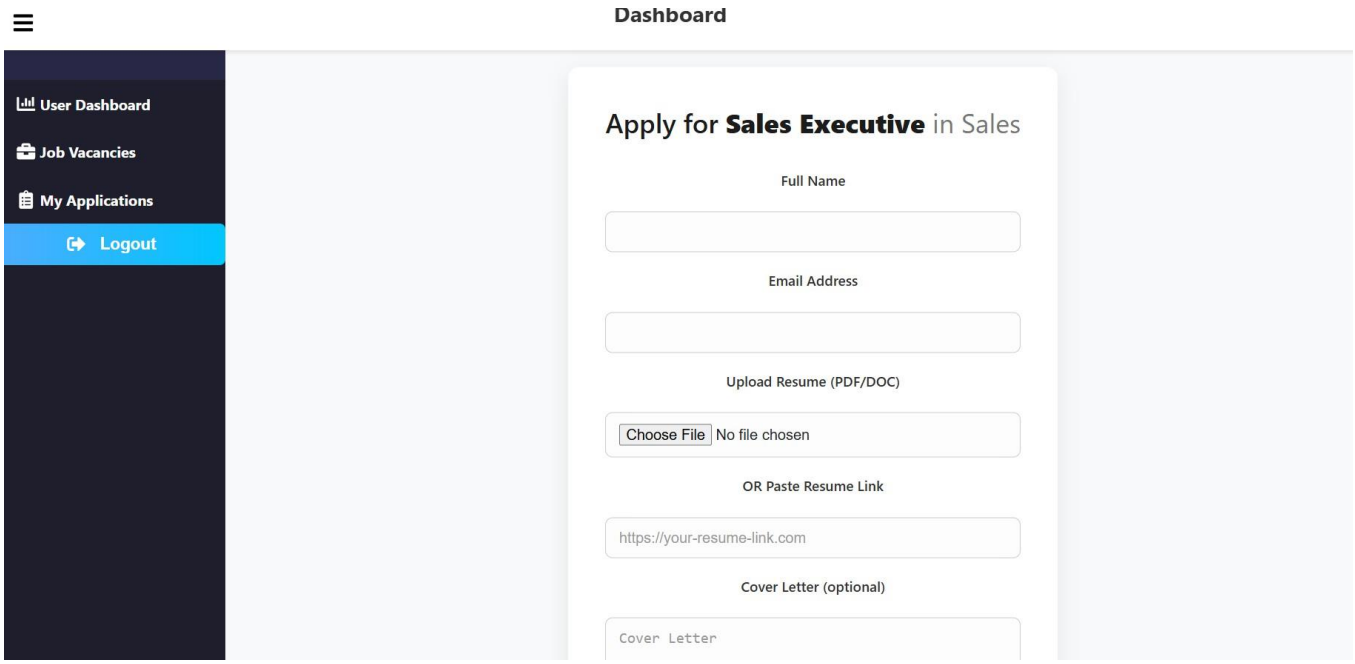
8.1.10 Data Analyst Dashboard:



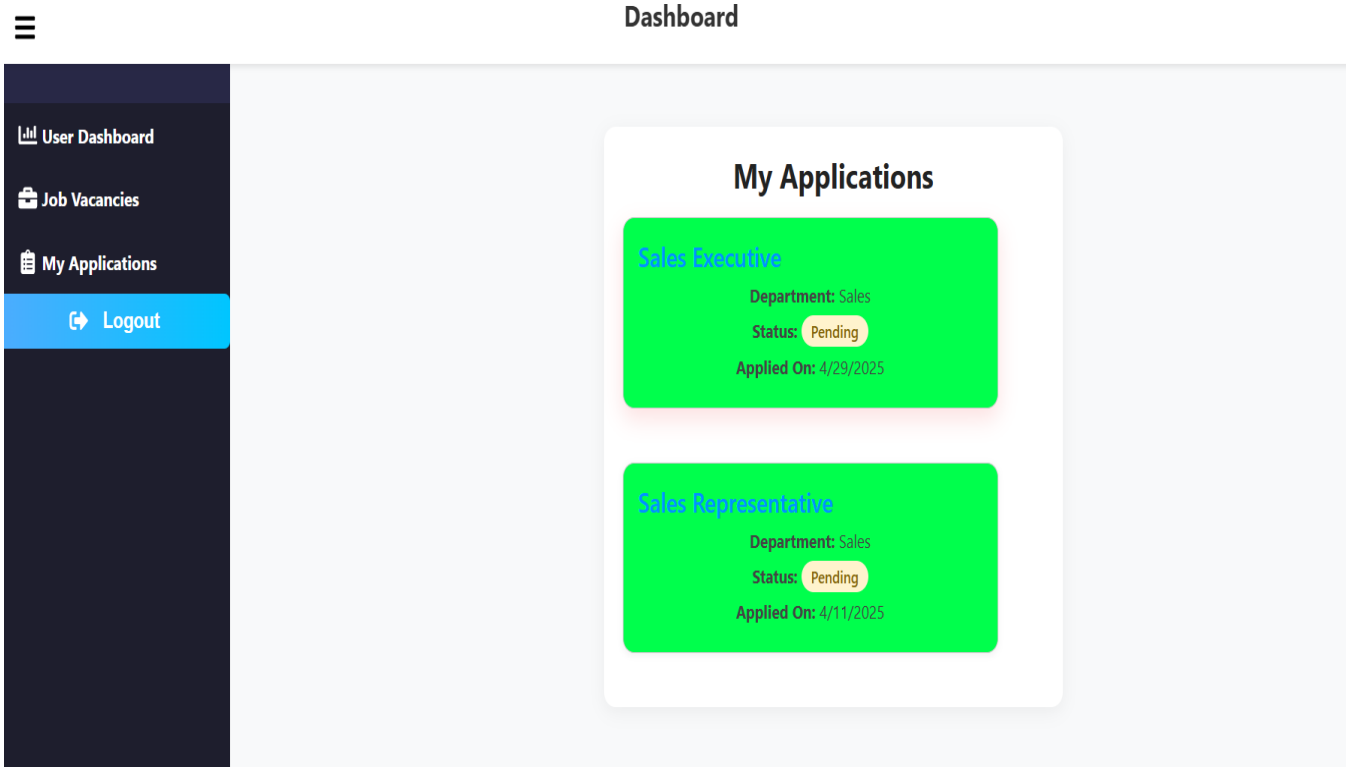
8.1.11 Job Vacancies Page:



8.1.12 Apply Job Page:



8.1.13 My Applications Page:



9. USER MANUAL

9.1 Introduction

The goal of Employee attrition predictor is to is to help organizations predict the likelihood of employee attrition using machine learning models. By analyzing key HR metrics, the system will provide actionable insights to HR departments, enabling them to take proactive measures to retain employees and reduce hiring costs.In the modern world, it helps to provide organization with actionable insights into employee retention by analyzing key factors such as job satisfaction, performance,work environment,and other HR metrics.This system also contains various activities, including uploading data, predicting individual risk percentage, viewing attrition trends, displaying job vacancies, applying for jobs etc.

9.2 Hardware Requirements:

- Intel i5or higher
- 8 GB RAM or higher
- 250 GB SSD or higher

9.3 Software Requirements:

- Languages: Python(flask for backend) ,Javascript (react for frontend)
- User interface design: Html, CSS
- Scripting language: JavaScript
- Operating system: Windows
- Visual Studio Code
- Database (Back End): MySQL
- Apache server
- Browsers**:** Firefox, Chrome or any other browser
- Machine Learning Library : Scikit-learn, TensorFlow
- Visualization Tools : Matplotlib,Seaborn,Ploty

10. CONCLUSION

The “Employee Attrition Predictor” project has been a remarkable example of efficiency , and reliability at organizations. Employee attrition predictor is to is to help organizations predict the likelihood of employee attrition using machine learning models. By analyzing key HR metrics, the system will provide actionable insights to HR departments, enabling them to take proactive measures to retain employees and reduce hiring costs.In the modern world, it helps to provide organization with actionable insights into employee retention by analyzing key factors such as job satisfaction, performance,work environment,and other HR metrics.This system also contains various activities, including uploading data, predicting individual risk percentage, viewing attrition trends, displaying job vacancies, applying for jobs etc . This software is a fully independent product and not a part of any other system. The users of the system are categorized as admin, Hr manager , Data analyst,Employee and Users. This is the a software for a complete attrition Predictor . It provides a simple database rather than complex ones for high requirements and it provides a good and easy graphical user interface (GUI) to both new as well as experienced users of the system

Future scope:

- One potential future scope is expand the project internationally .
- This system enable organizations to take proactive steps to retain valuable talent, manage workforce planning, and optimize recruitment strategies
- This system Collect employee-related data (job satisfaction, salary, experience, etc.).
  and Predict the probability of an employee leaving the organization.
- It Process and analyze data using machine learning algorithms and Provide visualization and insights to HR teams

11. BIBLIOGRAPHY

11.1 Book References:

- "HR Analytics in-Depth " by Subhashini Sharma Tripathy and Reuben Ray
- "Hands-on Machine Learning with Scikit-Learn,Keras and TensonFlow" by Aurelien Geron
- "Elements Of Statistical Learning" by Hasty,Tibshirani and Friedman
- "Predictive Analytics :The Power to Predict Who Will Click,Buy,Cheat And Die" by Eric Siegel
- "Data Mining For Dummies" by Kristina D.Lattmann

11.2 web references:

- Scikit-learn Documentation (https://scikit-learn.org/)
- Flask Documentation (https://flask.palletsprojects.com/)
- TensorFlow Documentation (https://www.tensorflow.org/)
- Kaggle Dataset (https://www.kaggle.com/pavansubhasht/ibm-hr-analytics-attrition-dataset)
- IEEE Research Paper (https://ieeexplore.ieee.org/document/8949565)
- Science -Direct Research Paper(https://www.sciencedirect.com/science/article/pii/S1877050920317191)
- GitHub Repository (https://github.com/yourusername/employee-attrition-prediction)
- Research Gate (https://www.researchgate.net/publication/123456789)
- Google Colab Notebook (https://colab.research.google.com/github/yourusername/employee-attrition-prediction/blob/main/notebook.ipynb)
- React.js Documentation (https://reactjs.org)

CHAPTER-12
PLAGIARISM

DrillBit

The Report is Generated by DrillBit Plagiarism Detection Software

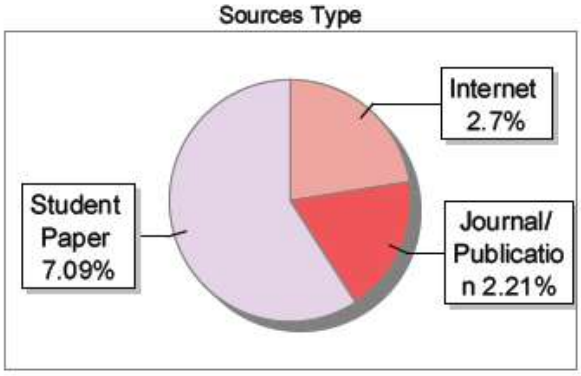
Submission Information

Author Name	Yadhunandh. C and Abhaydev A. K
Title	Employee Attrition Predictor
Paper/Submission ID	3600567
Submitted by	librarian@srinivasuniversity.edu.in
Submission Date	2025-05-10 14:07:55
Total Pages, Total Words	65, 5028
Document type	Project Work

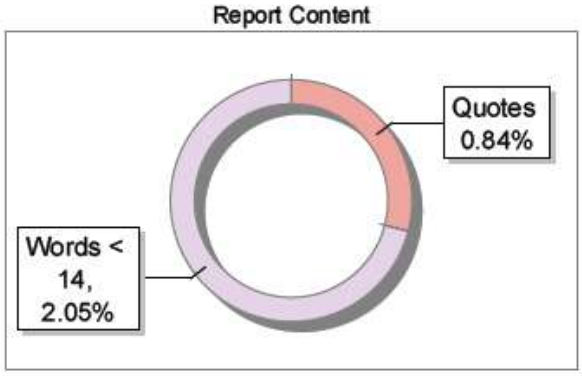
Result Information

Similarity **12 %**

Sources Type



Report Content



Exclude Information

Quotes	Not Excluded
References/Bibliography	Excluded
Source: Excluded < 14 Words	Not Excluded
Excluded Source	**0 %**
Excluded Phrases	Not Excluded

Database Selection

Language	English
Student Papers	Yes
Journals & publishers	Yes
Internet or Web	Yes
Institution Repository	Yes

A Unique QR Code use to View/Download/Share Pdf File

DrillBit

DrillBit Similarity Report

12	22	B
SIMILARITY %	MATCHED SOURCES	GRADE

A-Satisfactory (0-10%)
B-Upgrade (11-40%)
C-Poor (41-60%)
D-Unacceptable (61-100%)

LOCATION	MATCHED DOMAIN	%	SOURCE TYPE
1	Submitted to Visvesvaraya Technological University, Belagavi	4	Student Paper
2	Submitted to Visvesvaraya Technological University, Belagavi	1	Student Paper
3	Submitted to Visvesvaraya Technological University, Belagavi	1	Student Paper
4	Submitted to Visvesvaraya Technological University, Belagavi	1	Student Paper
5	REPOSITORY - Submitted to Exam section VTU on 2024-07-31 16-20 909028	1	Student Paper
6	ijoes.vidyapublications.com	1	Publication
7	Secured Communication using Data Dictionary through Triple DES- www.ijcaonline.org	<1	Publication
8	engagedly.com	<1	Internet Data
9	www.slideshare.net	<1	Internet Data
10	Secured Communication using Data Dictionary through Triple DES- www.ijcaonline.org	<1	Publication
11	pt.slideshare.net	<1	Internet Data
12	randomnerdtutorials.com	<1	Internet Data
13	iitk.ac.in	<1	Internet Data

14	pdfcookie.com	<1	Internet Data
15	sjcit.ac.in	<1	Publication
16	docplayer.net	<1	Internet Data
17	qdoc.tips	<1	Internet Data
18	Four-Year Follow-Up of High versus Low Intensity Summer Treatment for Adolescent by Sibley-2020	<1	Publication
19	moam.info	<1	Internet Data
20	mrynet.com.br	<1	Internet Data
21	sifisheriessciences.com	<1	Publication
22	The fourth law of thermodynamics steepest entropy ascent by Beretta-2020	<1	Publication